基于 RISC-V CPU 与 NPU 的异构视觉处理系统

摘 要

针对嵌入式智能视觉应用中通用控制、密集计算与连续视频处理的不同需求，设计了基于紫光同创 PG2L100H 国产 FPGA 的 TriHaRiscv 异构视觉处理系统。自主设计的 RISC-V CPU 采用适配 FPGA 同步存储与时序约束的六级顺序流水线，支持综合时配置单、双发射。前端结合指令队列与跨取指包配对供给指令，利用动态分支预测减少控制流等待，通过操作数前递、记分牌和互锁处理数据相关与资源冲突；配置独立指令与数据缓存，并共享乘法和访存单元，权衡处理性能、资源开销与实现复杂度。

针对 YuNet 人脸检测网络构建定点 NPU，采用广播式并行乘加阵列和独立深度卷积通路，利用片上特征与权重复用、DMA 搬运及硬件计算图执行完成模型运算。CPU 负责整帧任务调度、检测框解码和非极大值抑制，NPU 执行卷积与辅助算子，视频子系统承担图像预处理、帧缓存读写和显示合成，各模块经共享 DDR 协同工作。

系统接收 800×600 HDMI 视频，实现全画面人脸检测及 RAW、BOX、AVATAR 三种显示模式。三帧槽通过提交与释放握手维持图像和结果对应，使采集、推理与显示重叠运行；透明头像依据主脸位置与尺寸跟随缩放，采用无损行区间纹理缓存和流式像素合成，减少片上存储需求并避免整帧合成图写回。

在 100 MHz 独立 CPU 板测中，片上存储配置的单、双发射 CoreMark/MHz 分别为 3.055 和 4.093。完整应用采用 50 MHz 计算时钟，整机综合占用 45,716 个 LUT，内部后布线最差建立、保持裕量分别为 +0.100 ns 和 +0.131 ns。完整模型的数字联合仿真通过三模式逐像素核对，仿真结果生成率约 2.77 次/秒；实板完成人脸检测、头像跟随、透明合成及无脸清除与恢复的功能验收。

关键词：RISC-V；FPGA；双发射处理器；异构计算；神经网络加速；智能视觉

目 录

摘 要 **I**

第一部分 作品概述 **1**
1.1 功能与特性 . . . 1
1.2 应用领域 . . . 1
1.3 主要技术特点 . . . 1
1.4 主要性能指标 . . . 2
1.5 主要创新点 . . . 2
1.6 设计流程 . . . 3

第二部分 系统组成及功能说明 **4**
2.1 系统整体介绍 . . . 4
2.1.1 系统架构 . . . 4
2.1.2 模块功能 . . . 5
2.2 CPU 设计 . . . 6
2.2.1 CPU 微架构 . . . 6
2.2.2 指令提取 . . . 8
2.2.3 指令译码 . . . 13
2.2.4 指令执行 . . . 13
2.2.5 存储访问 . . . 17
2.2.6 指令写回 . . . 18
2.2.7 流水线控制 . . . 18
2.2.8 存储系统 . . . 19
2.2.9 中断和异常 . . . 21
2.3 NPU 设计 . . . 22
2.3.1 NPU 微架构 . . . 22
2.3.2 PE 计算单元 . . . 24
2.3.3 卷积计算通路 . . . 25
2.3.4 算子控制器 . . . 26
2.3.5 片上缓冲 . . . 26
2.3.6 DMA 控制器 . . . 26
2.3.7 卷积窗口生成 . . . 27
2.3.8 辅助算子单元 . . . 27
2.3.9 定点后处理单元 . . . 28
2.3.10 寄存器接口 . . . 28
2.4 系统互连与存储 . . . 29
2.4.1 总线结构 . . . 29
2.4.2 存储组织 . . . 29
2.4.3 访问仲裁 . . . 30
2.4.4 DDR 带宽预算 . . . 30
2.4.5 共享数据访问约定 . . . 31
2.5 图像采集与显示链路 . . . 31
2.5.1 图像采集 . . . 32

2.5.2 输入预处理 . . . 33
2.5.3 帧缓存管理 . . . 33
2.5.4 图像叠加 . . . 34
2.5.5 显示输出 . . . 35
2.6 模型部署与应用软件 . . . 36
2.6.1 模型选型 . . . 36
2.6.2 量化方案 . . . 37
2.6.3 模型导出 . . . 39
2.6.4 算子映射 . . . 39
2.6.5 检测后处理 . . . 40
2.6.6 裸机运行时 . . . 41
2.6.7 帧调度 . . . 42
2.6.8 异常处理 . . . 42
2.7 FPGA 实现 . . . 42
2.7.1 目标平台 . . . 44
2.7.2 资源统计 . . . 45
2.7.3 工程配置 . . . 45
2.7.4 时钟复位 . . . 46
2.7.5 板级接口 . . . 46
2.7.6 实现约束 . . . 46

第三部分 完成情况及性能参数 **47**
3.1 整体介绍 . . . 47
3.2 工程成果 . . . 47
3.2.1 硬件成果 . . . 47
3.2.2 软件成果 . . . 47
3.2.3 功能验证 . . . 47
3.2.4 应用演示 . . . 48
3.3 特性成果 . . . 50
3.3.1 CPU 性能 . . . 50
3.3.2 CPU 流水线效率 . . . 55
3.3.3 NPU 性能 . . . 57
3.3.4 系统性能 . . . 58
3.3.5 FPGA 实现指标 . . . 60

第四部分 总 结 **62**
4.1 可扩展之处 . . . 62
4.2 心得体会 . . . 62

第五部分 参考文献 **64**

附 录 **65**
A 仿真环境与验证方法 . . . 65
A.1 工具配置 . . . 65
A.2 测试平台 . . . 66
B CPU 仿真验证 . . . 67
B.1 指令集验证 . . . 67
B.2 单元级仿真验证 . . . 70

B.3 模块级仿真验证 74
B.4 系统级仿真验证 79
C NPU 仿真验证 85
C.1 计算正确性 85
C.2 分块计算 86
C.3 DMA 传输 87
C.4 任务控制 88
C.5 辅助算子 90
D 系统互连与存储仿真验证 94
D.1 CPU 主端控制与外设访问 94
D.2 共享存储与地址译码 94
E 图像采集与显示链路仿真验证 97
E.1 四类系统级场景 97
E.2 采集与槽所有权 98
E.3 共享 DDR 背压与采显联合 98
E.4 紧凑显示组件回归 99
F 系统联合仿真验证 101
F.1 完整模型推理 101
F.2 连续帧处理 101
F.3 运行配置 102
F.4 结果核对 103

第一部分 作品概述

1.1 功能与特性

TriHaRiscv[1]是基于盘古 100pro+ 国产 FPGA 的 RISC-V CPU 与 NPU 异构系统，完成 YuNet 人脸检测与同帧头像叠加。

- CPU 采用六级顺序流水线，可配置单/双发射，支持 RV32IM、Zicsr 和 Zifencei。
- CPU 运行于机器模式，支持异常处理、中断及周期计数。
- I/D-Cache 各 16 KiB、两路组相联，D-Cache 采用写回、写分配策略。
- BHT、BTB 和返回地址栈提供动态分支预测，双队首指令 FIFO 支持跨取指包配对。
- NPU 支持普通、逐点和深度卷积及辅助算子，采用 8 位定点乘法、32 位累加。
- CPU 提交整帧任务，NPU 执行计算图，DMA 搬运权重、特征图和结果。
- 配置 32 bit AXI4-Lite 控制接口和 128 bit AXI4 DMA 数据接口。
- 集成 Boot ROM、SRAM 及 1 GB DDR3，区分 CPU 可缓存私有区与 DMA 不可缓存共享区。
- UART 装载模型，I^2C 初始化 HDMI 芯片，LED 指示运行状态。
- 支持 800×600 HDMI 输入输出、全画面人脸检测及原图、检测框、透明头像显示。
- 视频硬件完成双线性缩放与补边，生成 320×320 UINT8 BGR NCHW 模型输入。
- CPU 完成定点解码、NMS 和主人脸选择，三帧槽保持采集、推理与显示的帧对应关系。

1.2 应用领域

系统面向交互式展示终端和嵌入式智能视觉设备。HDMI 视频源输入后，系统对完整画面进行人脸检测，按显示模式输出原图、检测框或透明头像。头像随主人脸的位置和大小变化，无人脸时输出原图。FPGA 完成图像预处理、推理加速和像素合成，CPU 负责帧任务控制、检测后处理及显示记录提交。模型和头像装载完成后，系统在板上独立运行。该平台还可用于 RISC-V 处理器微架构评估、CPU 与加速器协同控制，以及共享存储、跨时钟域和视频时序的工程验证。

1.3 主要技术特点

- **可配置顺序处理器**：CPU 采用 IF1、IF2、ID、E1、E2、WB 六级流水线；通过双队首指令 FIFO、跨取指包配对和两条整数执行通路实现可选双发射，共享 LSU 与

[1]名称取自团队名“哈哈哈”，即 Triple Ha。

乘法器以控制资源开销。

- **面向模型的定点加速**：针对 YuNet 框检测部署图，CPU 提交整帧任务，NPU 读取 79 条算子记录并按依赖顺序执行；广播式并行 MAC 阵列和独立深度卷积通路支持 U8/S8 激活、S8 权重和定点重定标。
- **分层存储与数据搬运**：利用片上多存储体缓冲和 DMA 复用特征、权重与中间结果，CPU、NPU 和 Video 经共享 DDR 交换数据。
- **同帧视频显示**：视频硬件生成 BGR NCHW 模型输入，并在原图扫描时叠加检测框或透明头像；通过帧号、槽状态和释放确认维持采集、推理与显示的对应关系。

1.4 主要性能指标

表1-1列独立 CPU 板测；表1-2列整机数据。

表 1-1 100 MHz CPU 独立板测（核级 LUT）

发射	存储	CoreMark/MHz	LUT	CoreMark/LUT
双	SRAM	4.093	9677	0.04229
双	Cache ＋ DDR	3.152	9707	0.03247
单	SRAM	3.055	6139	0.04977
单	Cache ＋ DDR	2.497	6507	0.03838

表 1-2 系统性能

指标	数值或现象	统计口径
CPU/NPU	50 MHz	应用配置
LUT/FF	45716/41848	整机综合
DRM/APM	113.5/64	整机综合
建立裕量	+0.100 ns	内部后布线
结果完成率	约 2.77 fps	融合仿真
头像叠加	三模式、头像跟随、透明合成	板级功能观察

DRM 按 36 Kbit 等效块计。板级新图帧率尚未测得。

1.5 主要创新点

1. 设计可配置单/双发射六级 RISC-V CPU，结合跨取指包指令配对、双执行通路和共享运算单元，提供不同的性能与资源配置。
2. 围绕固定 YuNet 计算图设计广播式 MAC 与独立深度卷积通路，由 NPU 完成图遍历和层内分块，利用片上数据复用控制外存访问和 FPGA 资源开销。
3. 构建采集、推理和显示的同帧管理机制，采用三帧槽、显示边界切换与流式头像合成，避免整帧合成结果再次写回 DDR。

1.6 设计流程

开发流程如图1-1所示。先分析人脸检测需求与 FPGA 资源，再并行开发 CPU、NPU 和 Video：CPU 完成指令执行与软件控制，NPU 完成 YuNet 算子和量化适配，Video 完成采集与显示。各模块经独立验证后，统一地址、寄存器和帧接口，完成系统集成。融合仿真核对完整模型与同帧像素，FPGA 板测检查三模式显示与头像跟随，并分别统计 CoreMark、资源、时序和结果间隔。

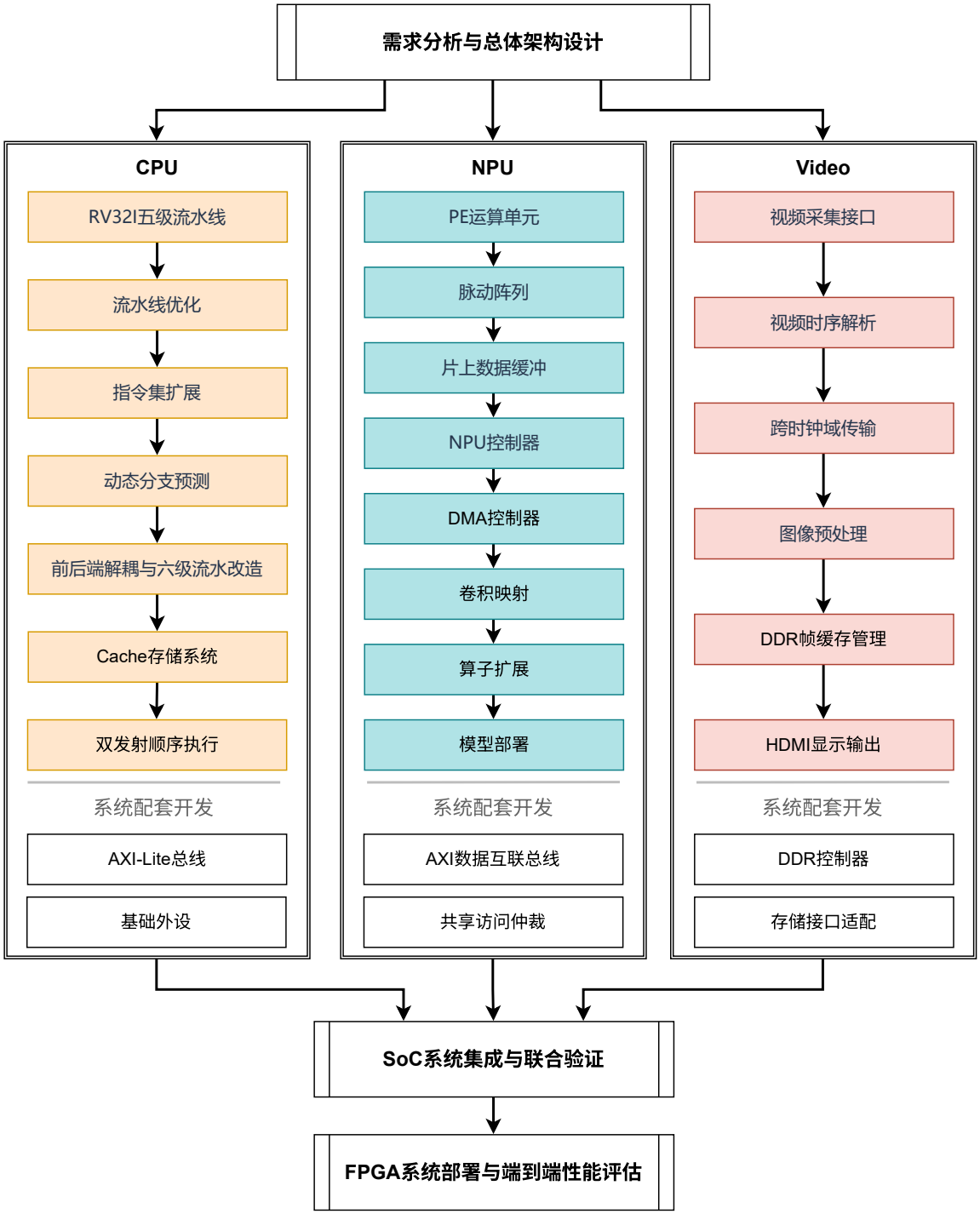


图 1-1 TriHaRiscv 设计流程

第二部分 系统组成及功能说明

2.1 系统整体介绍

2.1.1 系统架构

系统由 TriHaRiscv SoC 与视频子系统组成，整体架构如图2-1所示。SoC 以支持单/双发射配置的六级 RISC-V CPU 为控制核心，集成 NPU、片上存储和访问互连；视频子系统在 FPGA 顶层与 SoC 并列连接，完成图像采集、输入预处理和同帧全屏显示。CPU 执行裸机程序，向 NPU 提交绑定输入帧和工作区的整帧任务，处理检测输出并提交显示参数；NPU 读取模型程序，依次执行 YuNet 的卷积及辅助运算。

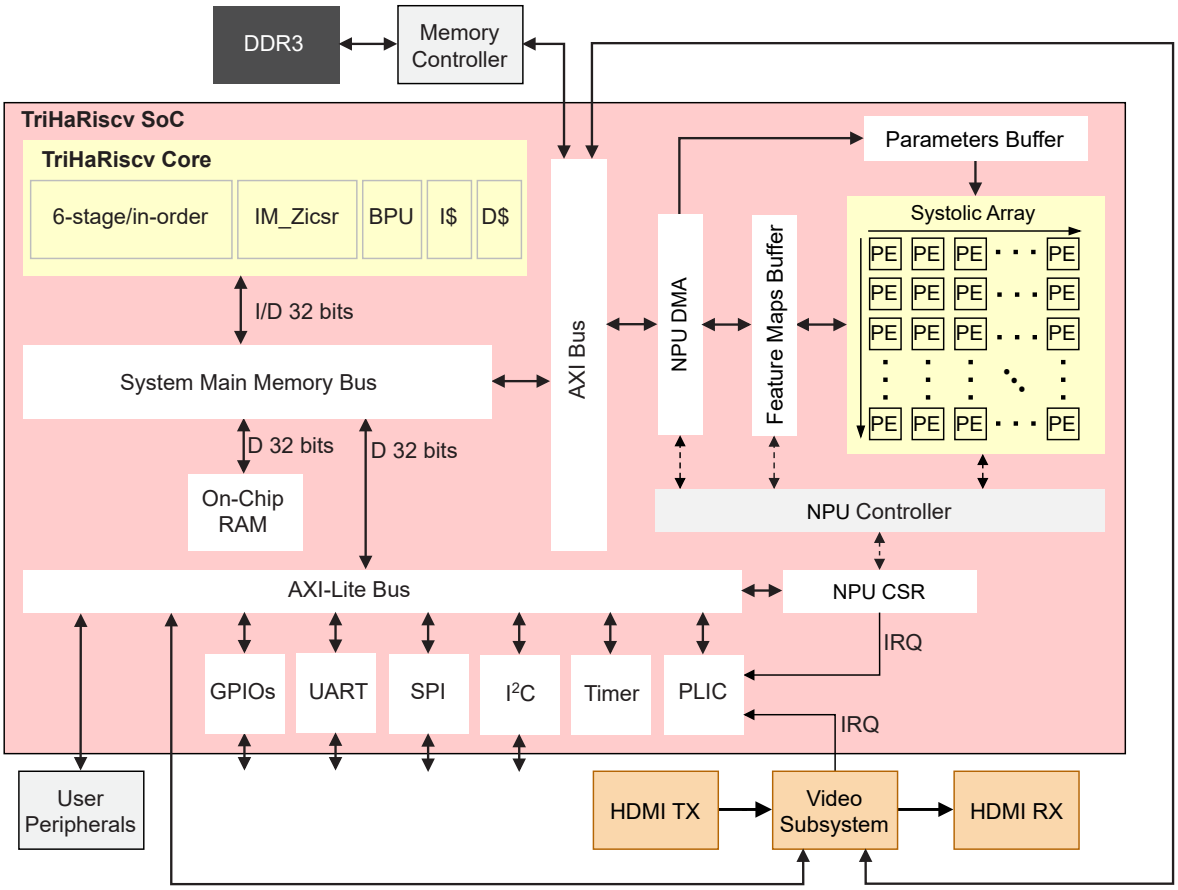


图 2-1 TriHaRiscv 异构 SoC 系统架构图

控制通路采用 32 bit AXI4-Lite，连接 SoC 内部的 NPU 任务与状态寄存器，并通过外部控制接口访问 Video CSR。视频 DMA 作为独立 AXI 主设备接入数据互连，与 CPU 访存桥和 NPU DMA 共享 DDR 访问通路，逻辑数据接口宽度为 128 bit。Video 与 NPU 的事件中断汇入 CPU 外部中断输入，应用通过寄存器轮询管理任务。Boot ROM 和片上 SRAM 支持 CPU 启动，DDR 按访问属性划分为 CPU 私有区和共享区。DDR

控制器、视频子系统及 HDMI 接收、发送封装均位于 SoC 之外，由 FPGA 顶层完成连接。

输入的 800×600 RGB888 画面生成原图和 320×320 UINT8 BGR NCHW 检测输入。CPU 对每个选定的输入帧提交一次网络推理任务，NPU 依次执行各层算子并输出九个检测头，CPU 完成定点解码、NMS 和主人脸选择。视频 DMA 将两条采集支路的数据写入 DDR，显示端预取原图，经行缓存、检测框或头像合成与时序生成模块输出一路 800×600 全屏 HDMI 画面。帧控制模块在显示边界整体更新原图地址、检测框及头像参数，保持图像与检测结果的帧号一致；无人脸时输出对应原图。

2.1.2 模块功能

系统按 CPU、NPU、总线、存储、外设和视频六类模块组织，视频模块在 FPGA 顶层与 SoC 连接，SoC 内部模块组成如图2-2所示。硬件模块通过总线和事件接口连接，裸机软件负责整帧任务调度、帧所有权管理及异常记录。

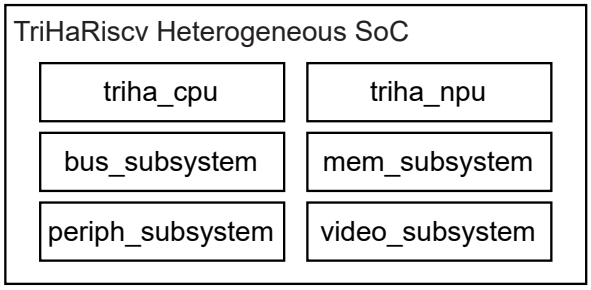


图 2-2 TriHaRiscv SoC 模块图

CPU 经总线模块读取模型输出、配置计算任务并提交显示记录。NPU 通过 DMA 访问权重、输入张量和中间张量；Video 通过独立 DMA 写入采集数据、读取原图和头像素材。存储模块组织 ROM、SRAM 和 DDR 访问，板级控制逻辑完成模型与素材装载、接口初始化和运行状态指示。各模块的主要功能与接口见表2-1。

表 2-1 SoC 模块功能表

模块	主要功能	主要接口
`triha_cpu`	执行程序、处理异常中断，运行任务调度、检测后处理和帧管理软件	64 bit 取指返回、32 bit 数据接口、中断输入
`triha_npu`	读取模型图并执行普通/深度卷积及辅助算子，管理分块、缓冲与 DMA	AXI4 数据、AXI4-Lite 整帧任务接口、完成/错误中断
`bus_subsystem`	地址译码、访问仲裁、CPU 接口桥接和错误响应	CPU 访存、AXI4 及 AXI4-Lite 接口
`mem_subsystem`	组织 ROM、SRAM、DDR 窗口及访问属性	片上存储接口、DDR 访问接口
板级控制逻辑	完成模型与头像串口装载、HDMI 初始化及按键处理	UART、I^2C、复位与 LED

表 2-1 SoC 模块功能表（续）

模块	主要功能	主要接口
`video_subsystem`	完成全帧采集、预处理、输入格式化、全屏显示、检测框及头像合成	像素流、AXI4 DMA、控制寄存器、帧事件

2.2 CPU 设计

2.2.1 CPU 微架构

CPU 采用 RV32IM、Zicsr 与 Zifencei 指令集[1]，支持机器模式异常处理，承担裸机程序执行、外设控制、NPU 任务配置和检测后处理。指令长度固定为 32 bit，整数寄存器与地址宽度均为 32 bit。六级顺序流水线支持单发射和双发射配置。双发射时，每周期最多发射两条相邻指令，程序顺序靠前的指令使用 A 槽，靠后的指令使用 B 槽；B 槽只有在 A 槽可接纳时才发射。顶层参数 `DUAL_ISSUE` 取 0 选择单发射，取 1 选择双发射，并向内核传递为 `SUPPORT_DUAL_ISSUE`。该参数在综合时确定硬件结构，不通过软件在运行时切换。单发射配置不生成第二执行单元，寄存器堆使用 2 读 1 写接口；双发射配置使用两条执行通路和 4 读 2 写接口。两种配置共用六级流水边界、指令集、异常接口和存储协议。本节结构图以双发射配置展示完整数据通路，单发射配置每周期仅接纳 A 槽指令。程序通过物理地址访问存储空间，启动入口映射到 `0x0000 0000` 处的 Boot ROM。

取指前端、双候选译码、寄存器堆及前递网络向两条执行通路提供指令和操作数。两路整数运算单元分别处理 A、B 槽指令，LSU 和乘法器各由一个实例共享，除法与 CSR 仅由 A 槽使用。I-Cache 与 D-Cache 分别服务取指和数据访问，本地 ROM、SRAM 及不可缓存设备由地址路由选择访问路径。独立 TCM 配置直接接入片上存储，不实例化 I/D Cache 及 DDR 控制器；缓存配置通过 I/D Cache 访问 DDR，两类性能结果分别统计。CPU 主要配置见表2-2。

人脸检测应用采用 50 MHz 单发射控制配置，程序、数据和栈位于 32 KiB 片上 SRAM，CPU 通过不可缓存访问路径读取 DDR 中的检测输出。该配置设置 `MINIMAL_CONTROL_CPU=1` 和 `CACHE_ENABLE=0`，保留整数乘除法、机器模式 CSR 和异常处理。相对于通用配置，分支预测、BHT 与 RAS 使能置零，Load/MUL 及提前 MUL 结果旁路置零；BTB/BHT 规模参数由 512 项缩为 2 项，但预测模块不参与该配置的取指。相关指令按结果就绪条件推进。应用由 CPU 处理控制和后处理任务，NPU 执行计算图，片上 SRAM 承担程序与数据访问。整机以功能和时序验收为优先，采用与资源、时钟跨域及布线约束配套的 50 MHz 工作点，计算域 Slow 角建立裕量为 +2.287 ns。本节按通用配置介绍 CPU 各模块；独立 CPU 的 100 MHz CoreMark 测试采用各自构建，结果不代入该应用配置。

表 2-2 CPU 主要配置

项目	配置
指令与模式	RV32IM、Zicsr、Zifencei，M-mode
流水线	IF1、IF2、ID、E1、E2、WB；顺序双发射，可配置为单发射
通用寄存器	32 个 32 bit 寄存器；单发射 2 读 1 写，双发射 4 读 2 写，`x0` 恒为零
取指接口	32 bit 地址、64 bit 返回数据，8 B 对齐取指包
数据接口	32 bit 地址与数据、4 bit 写字节使能
指令 FIFO	六条指令容量，每项 119 bit 载荷，双队首缓存，可跨包配对
分支预测	512 项 BTB、512 项两位 BHT、4 项 RAS
执行资源	两路 ALU、共享 LSU 和 MUL、A 槽专用 DIV 和 CSR
L1-Cache	I/D 各 16 KiB，两路组相联、256 组、32 B 行；D-Cache 写回、写分配
结果前递	两条通路的 E1、E2 及 WB；按结果类型和就绪条件选择

图2-3给出六级流水线的模块划分、两条执行通路与共享资源，窄寄存器符号及带内侧竖线的状态框标出寄存位置。ID 级包含寄存器读取、相关性检查、前递选择和组合 ALU 运算，整数结果在 ID/E1 边界保存；E1 使用已寄存的分支操作数解析通用分支，保持整数结果并发出访存请求。因此，六级划分不意味着全部算术组合逻辑都位于 E1，ID 的时序路径同时包含操作数选择与 ALU 计算。图中实线表示数据通路，虚线表示控制关系，浅色分区区分流水级或功能区域；寄存状态用窄条或带内侧竖线的方框表示，此约定同样用于后续结构图。

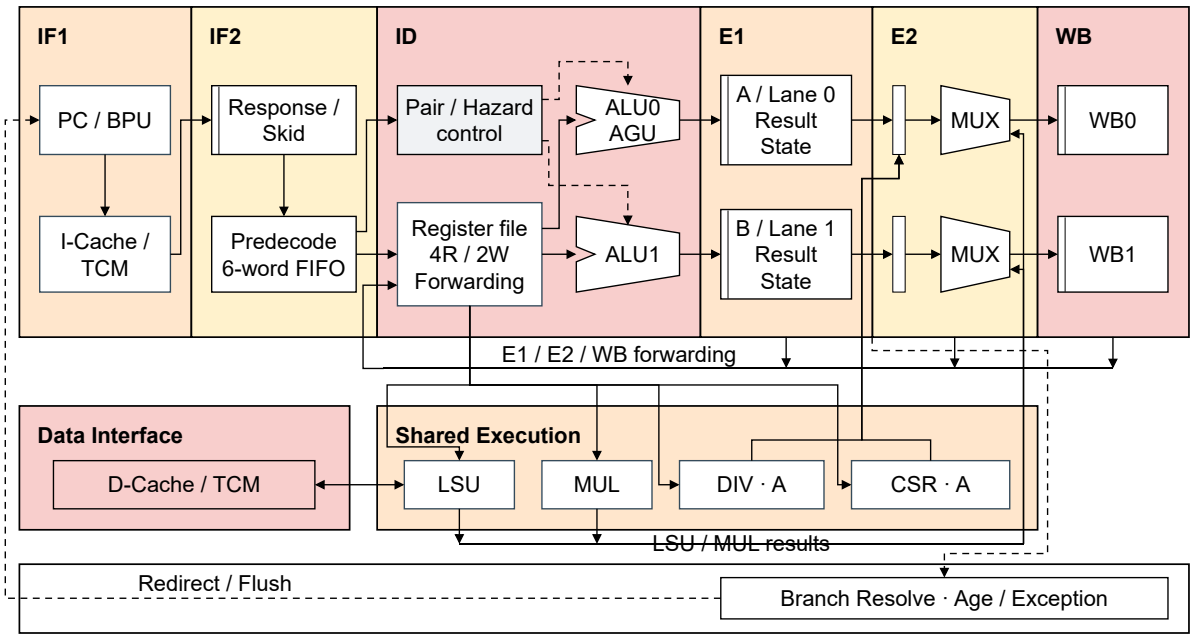


图 2-3 六级 CPU 微架构

表 2-3 CPU 实际流水级及寄存内容

流水级	功能与边界
IF1	选择顺序、预测或重定向地址，提交取指请求；接纳时保存 PC 及预测上下文
IF2	接收返回包，匹配上下文，经响应缓冲和预译码写入指令 FIFO
ID	读取双队首，执行 4 路寄存器读取、前递及配对检查；计算 ALU 结果、访存地址和特定零比较分支的早期条件
ID/E1 边界	保存两条通路的指令状态、ALU 结果、分支操作数、访存请求信息及乘法输入
E1	解析通用分支，发出访存请求，提供整数结果前递及满足条件的低位乘积前递
E2	获得常规乘法结果，接收访存完成或等待响应，选择各通路后续写回值
WB	根据有效位、异常及目的寄存器使能完成最多两路写回，并提供指令完成观测

各级状态包含 PC、指令类型、有效位、目的寄存器及异常信息。暂停保持相关状态，恢复或异常控制撤销后续指令的有效性。Cache 未命中、总线背压及迭代除法会增加停留周期；双发射表示结构上限，实际吞吐率还受前端供给、数据相关和指令配对限制。表2-3区分组合运算与寄存位置，避免将 ALU 结果寄存边界与分支解析阶段混为一谈。

2.2.2 指令提取

取指前端由 PC 选择、请求控制、预测器、请求上下文寄存器、响应缓冲和指令 FIFO 组成，如图2-4所示。请求地址按 8 B 对齐；每个 64 bit 返回包包含两个 32 bit 指令位置。入口 PC 的第 2 位保留在请求上下文中，用于确定第一条有效指令。

取指器跟踪一笔未返回请求。忙条件为“已有请求且本周期尚无有效响应”，因此前一响应到达时，可以在下层接纳和 FIFO 就绪均满足的情况下提交下一请求。表2-4给出一周期返回、无背压时的接口时序；它说明接口允许同周期接续，不表示所有缓存状态均能达到每周期一个包。缓存查询延迟、未命中、重定向以及 FIFO 背压仍会降低实际供给率。

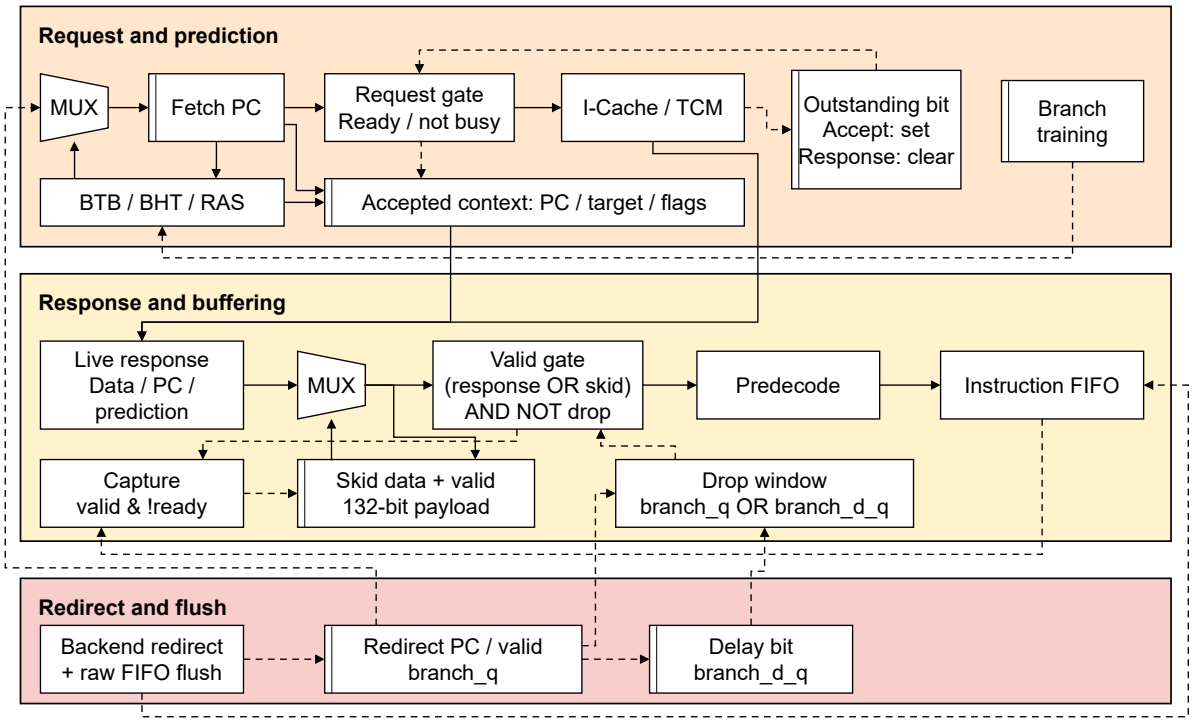


图 2-4 取指前端的请求、返回与恢复通路

表 2-4 取指接口允许的理想接续时序

周期	本周期接纳请求	本周期响应与上下文
T_0	包 0	无；沿后保存包 0 上下文
T_1	包 1	包 0，使用沿前的包 0 上下文
T_2	包 2	包 1，使用沿前的包 1 上下文
T_3	包 3	包 2，使用沿前的包 2 上下文

132 bit 响应缓冲保存返回指令、PC、预测及错误状态。后端可接纳时直接传递，背压时保存完整返回包，解除背压后再送入 FIFO。已被接纳的旧取指请求不能由冲刷撤销总线事务，前端必须接收并丢弃其返回。后端原始重定向在时钟沿清空 FIFO，已寄存的重定向与延后一拍状态共同形成响应丢弃窗口；缓冲中的旧包同步失效。表2-5列出有旧请求在途时的事件关系，具体恢复请求周期由旧响应与下层接纳共同决定。

表 2-5 重定向与旧响应处理

事件	前端动作	对后端可见的指令
旧请求已接纳	保持旧 PC 及请求上下文	已在 FIFO 中的有效候选
收到原始重定向	清空 FIFO，寄存目标 PC	本沿取消旧队列状态
恢复窗口内旧响应返回	丢弃旧响应，清除旧缓冲有效位	不向 FIFO 传递错误路径返回包
恢复地址请求可接纳	提交目标 PC，保存新上下文	等待新包返回
目标包有效返回	经预译码重新入队	仅提供恢复路径候选

分支预测

分支预测器由分组 BTB、两位计数器 BHT、RAS 及下一 PC 选择逻辑组成，如图2-5所示。BTB 共 512 项，低字、高字各使用 256 项存储体，PC 第 10 至 3 位索引行，第 2 位区分包内位置，高位地址用于标签匹配。从低字入口查询时，低字命中优先，只有低字未命中才采用高字命中；从高字入口查询时仅采用高字结果。

BHT 含 512 个两位饱和计数器，按 BTB 选中的指令位置选择方向预测，计数值 2 或 3 表示跳转。训练按实际分支 PC 索引，跳转递增、不跳转递减，端点饱和。这是一种先选择 BTB 命中位置、再判定该位置方向的结构：低字命中但预测不跳转时，不再重新选择高字的预测跳转。因此该策略可能错过同包高字的提前跳转机会，后端仍按实际分支结果恢复 PC。

RAS 含 4 个 32 bit 返回地址，以 `x1` 识别调用与返回，调用地址为该指令的 $PC+4$。预测调用或返回在 PC 通路可推进时更新推测栈指针；另一个参考指针在调用、返回的分支请求事件中参与修正。执行通路解析真实分支目标，恢复控制在预测错误时纠正程序流。

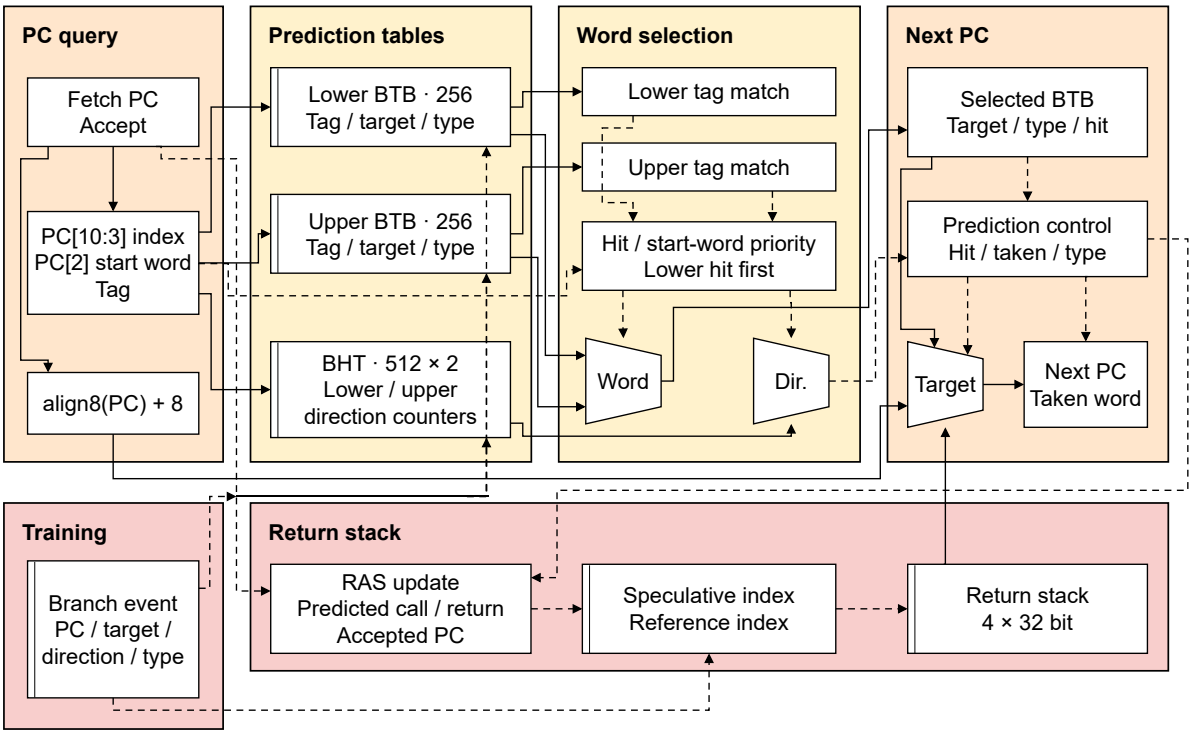


图 2-5 分支目标、方向与返回地址预测结构

有效返回预测优先使用 RAS 地址，其余跳转预测使用 BTB 目标；无跳转预测时采用 align8$(PC)+8$。包内跳转标志区分低字、高字和无跳转，随请求上下文保存。低字预测跳转时，入队端不保留同包高字。执行端的实际方向、目标、PC 及类型通过分支事件寄存器训练预测器，BHT 覆盖预测正确和错误的方向事件，BTB 在恢复时分配或修正条目。

指令 FIFO

图2-6中的 FIFO 按指令字组织，而非按取指包组织。六项环形存储提供六条指令容量，两个队首寄存器缓存最早的两个候选，二者不增加逻辑容量。每项 119 bit 载荷包含指令及取指状态、PC、预测下一 PC、预译码信息和 CSR 提示。取指入口位于包高字时，入队逻辑直接将高字作为第一条有效指令；不需要在发射端先丢弃一个低字。

入队根据入口位置和包内预测标志写入一条或两条指令。就绪条件为当前占用数不超过 4，预留最多两条输入的容量；本周期出队释放的容量不提前计入接纳余量。Head0 始终对应程序顺序最靠前的候选 A，Head1 对应后继候选 B；只有两者 PC 相差 4 且其他配对条件成立，才允许共同发射。因此队首可以跨越取指包边界，示例见表2-6。

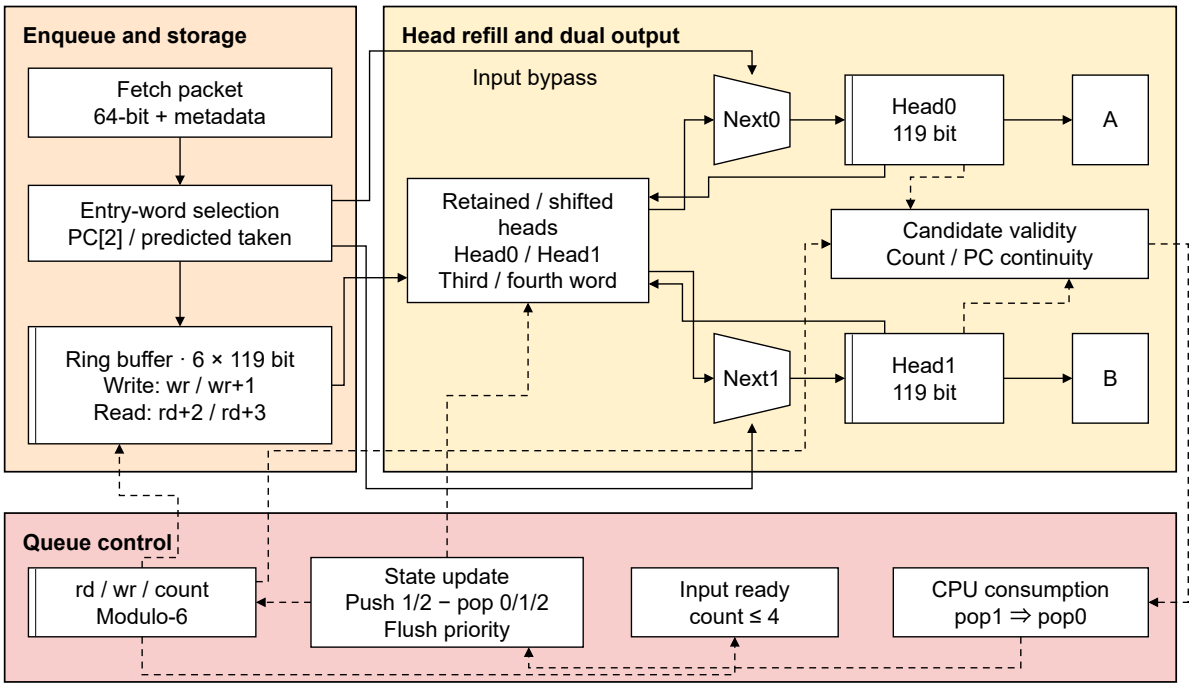


图 2-6 六项指令 FIFO 与双队首顺序出队

表 2-6 跨取指包配对示例

事件	队首 A、B	更新结果
包 P 与 $P+8$ 已入队	P、$P+4$	环中后续为 $P+8$、$P+12$
仅 A 可发射	P 出队，B 保留	新队首为 $P+4$、$P+8$
新队首可配对	$P+4$、$P+8$	同周期出队，虽来自不同取指包仍保持程序顺序
发生恢复冲刷	所有候选失效	计数和队首状态复位；载荷 RAM 无需逐项清零

CPU 的出队约束为 pop1 ⇒ pop0：B 出队必须伴随 A 出队。队首更新分别选择保留或前移的 Head0/Head1、环形存储中 rd+2/rd+3 处的后续项，以及本周期输入旁路。两路 Next 选择器形成下一周期队首载荷；队首数据、占用数及指针共同决定候选有效性。更新时冲刷优先于入队、出队；正常周期按实际入队数和出队数更新占用及队首。恢复沿可能同时存在队列出队条件的组合计算，但最终执行请求已被恢复控制屏蔽，队列状态以冲刷为准。

2.2.3 指令译码

译码与发射结构如图2-7所示，为 A、B 通路提供指令与就绪操作数。两条候选各提供两个寄存器索引，逻辑 4 读 2 写寄存器堆与 E1/E2/WB 前递网络并行提供操作数。寄存器 `x0` 读零且不建立写入依赖。同一源寄存器对应多个前序写入指令时，按程序顺序选择最近的写入结果；该结果尚未就绪时阻止发射。同一对中的 A 写 B 读相关以及 A/B 同目的寄存器冲突都会禁止 B 发射，保留 B 为后续周期的队首。

2.2.4 指令执行

双执行通路与共享资源如图2-8所示。两路 ALU 的组合结果分别在 ID/E1 边界锁存。各通路的 E2 寄存状态继续保存结果、类型与有效信息；A 槽在进入 E2 时选择 ALU、DIV 或 CSR 结果，随后由 E2 结果选择逻辑接入 LSU 或 MUL 返回，并送入 WB。EX0 支持完整整数运算，EX1 支持加减、逻辑、比较和立即数移位，不接收寄存器变量移位。

整数 ALU 的数据通路如图2-9所示。输入根据指令选择寄存器值、PC 或立即数，加减、比较、逻辑及移位结果经选择后在 ID/E1 边界写入结果寄存器，两条执行通路分别使用独立实例。暂停时保持结果寄存器，指令有效性由流水控制管理，寄存器中的数据本身不表示有效指令。

LSU 与 MUL 各自独立选择 A 或 B 的指令和操作数，因此 LSU 与 MUL 可在同周期分别接收一条指令；单个共享单元不能同时接收 A 与 B。DIV 和 CSR 只使用 A 槽，执行期间通过在途状态串行化相关控制。表2-7给出资源层面的允许组合，所有组合还

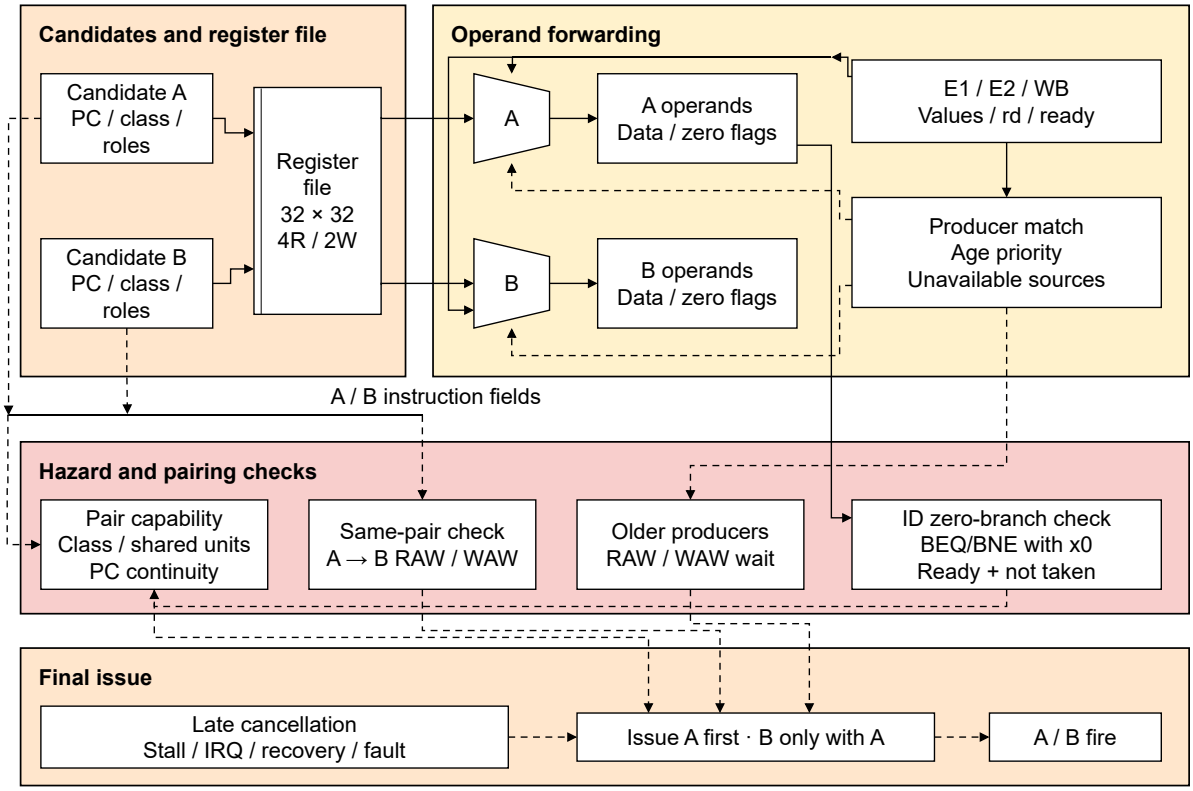


图 2-7 双候选译码、操作数前递与发射控制

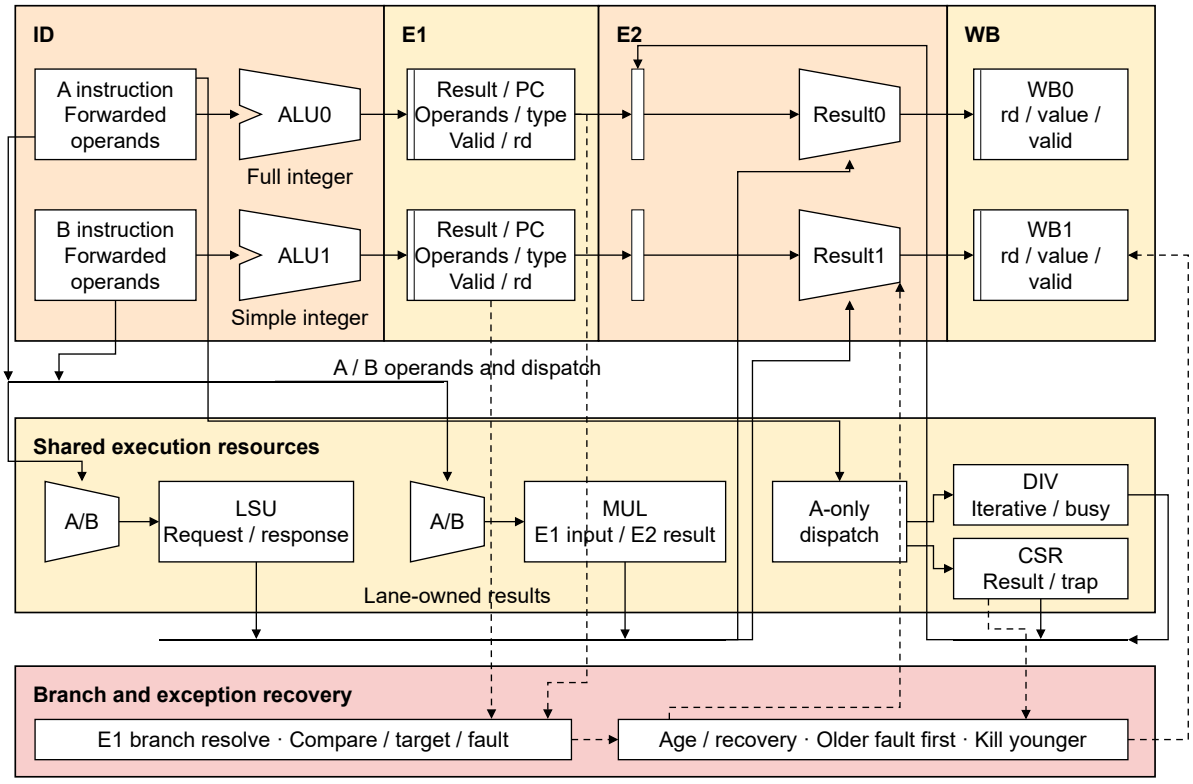


图 2-8 双执行通路、共享资源与恢复控制

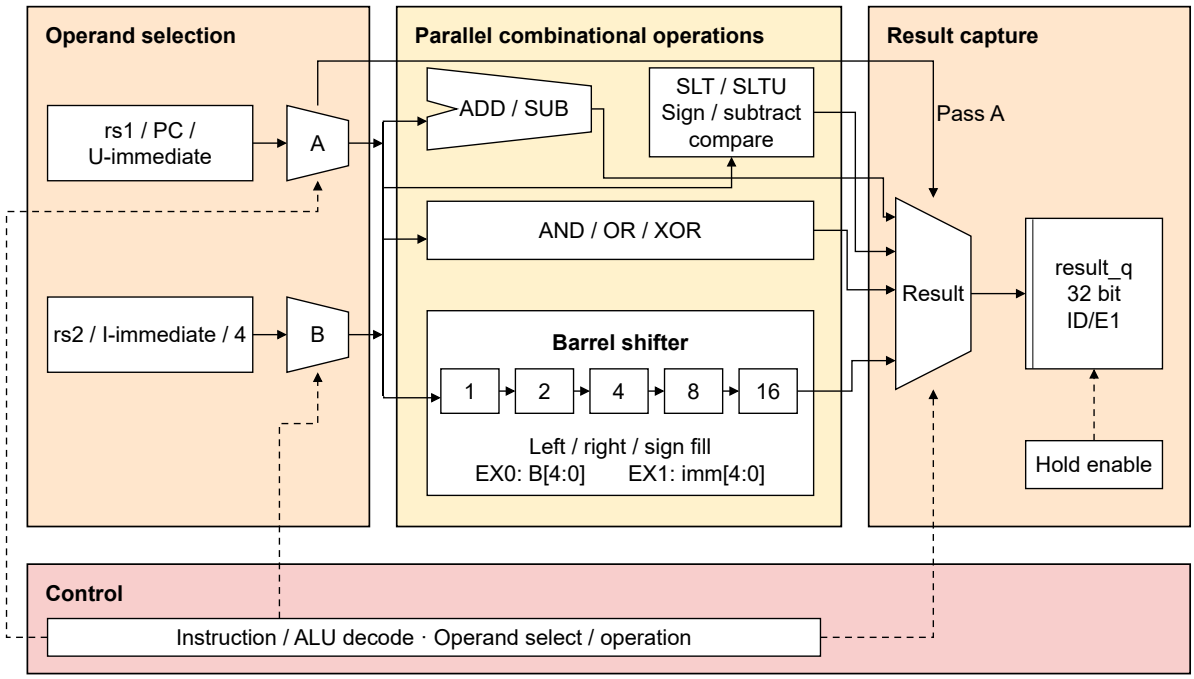


图 2-9 整数 ALU 组合通路及 ID/E1 结果寄存边界

需满足 PC 连续、操作数就绪、无相关冲突且 A 槽指令被接纳。

表 2-7 双发射配对规则

A 槽类型	允许的 B 槽类型	附加约束
整数 ALU	简单 ALU、分支、LSU、MUL	B 须属于 EX1 或共享单元支持的类型
LSU	简单 ALU、分支、MUL	不允许第二条 LSU
MUL	简单 ALU、分支、LSU	不允许第二条 MUL
未提前判定的分支	简单 ALU	B 仅作推测运算，A 恢复或故障时取消 B
ID 确认不跳转的 BEQ/BNE 零比较	简单 ALU、LSU、MUL	A、B 有效且无取指或非法指令错误；零标志对应就绪的前递操作数
DIV 或 CSR	无	仅 A 槽发射

ID 中的早期分支检查只处理 BEQ/BNE 且至少一个源寄存器为 `x0` 的情况。另一操作数的零标志与数据使用相同的前递来源和优先级；BEQ 比较值非零、BNE 比较值为零时，可以在 ID 确认不跳转。该判断只确定是否允许配对，最终发射仍检查尚未完成的前序写入指令、同对相关、全局暂停、异常及中断取消条件。未就绪加载或乘法的暂态零标志不能单独授权 B，普通双操作数分支也不使用此早期通路。

A 分支尚未在 ID 确定不跳转时，B 仅可为简单 ALU。A 在 E1 解析出恢复或故障时，`cancel1_e1` 取消 B 向 E2 推进，避免其写回。允许 B 进入 LSU 的零比较分支组合由 ID 条件提前授权，不依赖同周期尚未产生的 E1 结果；LSU 实际有效脉冲还由最终发射和中断屏蔽共同限定。已经被存储接口接纳的 Store 没有通用撤销机制，因而禁止错误路径 Store 发出必须发生在接纳之前。

A 为 Load、B 为分支时，共享暂停保持两条指令的状态及其对应执行通路，分支恢复不能越过尚未完成的 A 槽加载访问。前序访存指令的异常取消后续指令，CSR 异常或返回重定向优先于普通分支恢复。通用条件分支的比较与目标生成使用 E1 寄存的操作数、PC 和指令；跳转链接值为 $PC+4$，目标按 4 B 指令边界检查。分支解析、队列冲刷和后续指令取消共同恢复程序流，不能仅依靠预测器保持正确执行。

乘法器支持 `MUL/MULH/MULHSU/MULHU`，输入在 E1 保存，常规结果在 E2 保存。启用相应早期旁路时，`MUL` 且两个输入高 16 位均为零，可从 E1 输入形成组合低位部分积供后继相关指令使用；完整结果仍通过 E2 正常写回。高位乘法和不满足范围条件的低位乘法继续等待常规结果，不使用早期值。除法器按迭代方式生成商和余数，包含除零及有符号溢出的规定结果；完成前保持在途状态。CSR 采用专用读改写和异常控制通路，整数寄存器写回值与 CSR 副作用分别管理。

2.2.5 存储访问

LSU 结构见图2-10。地址生成将基址与 I 型或 S 型立即数相加，访问控制译码字节宽度、符号扩展及缓存属性。字节访问允许任意字节地址，半字与字访问分别要求低 1 位、低 2 位为零；不对齐时禁止外部读写，使用内部完成事件报告异常。Store 按有效地址低位排列数据并生成 4 bit 字节使能。请求信息在 ID/E1 边界保存，由 E1 向 D-Cache 或 TCM 提交。

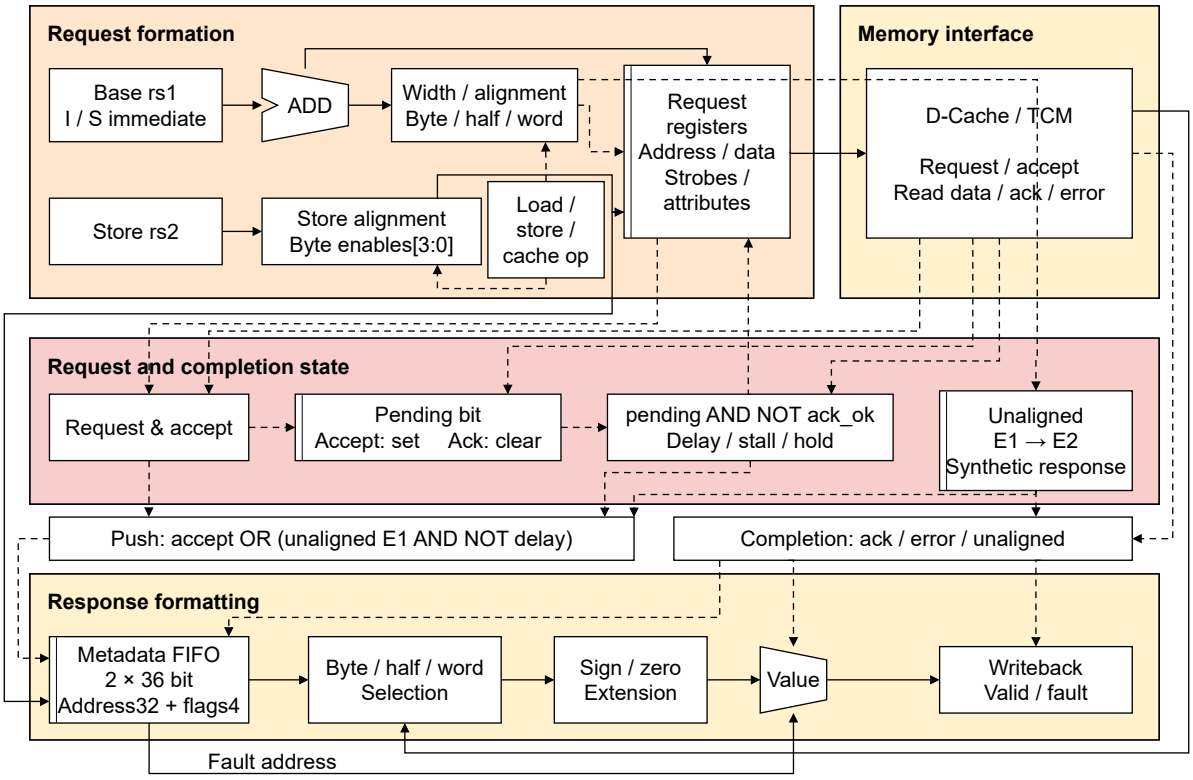


图 2-10 LSU 请求、数据处理与完成通路

请求接纳和访问完成是两个事件：`mem_accept` 接纳请求，`mem_ack` 报告响应，`mem_error` 报告访问错误。未接纳期间保持请求；接纳后设置 pending，等待完成时阻止继续接纳新的外部事务。成功完成与下一次接纳可在同一周期交接，见表2-8。表中状态取时钟沿前后，例子假定下一请求已准备且没有异常、背压或其他流水阻塞。

表 2-8 LSU 成功响应与下一请求接纳的交接

周期	沿前状态	接口事件	沿后状态
T_0	无待完成访问	接纳请求 A	pending=1，队首为 A 元数据
T_1	A 等待响应	无 ack，无新接纳	保持 A 对应的执行通路及流水状态
T_2	A 等待响应	ack(A) 与 accept(B)	A 完成并出队，B 入队；pending 保持 1

表 2-8 LSU 成功响应与下一请求接纳的交接（续）

周期	沿前状态	接口事件	沿后状态
T_3	B 等待响应	无 ack，无新接纳	保持 B 对应的执行通路，不重复接纳 B
T_4	B 等待响应	ack(B)，无新接纳	B 出队，pending 清零

请求元数据 FIFO 深度为 2，每项保存 32 bit 地址和加载、字节、半字、符号扩展四个标志，共 36 bit。外部请求被接纳时元数据入队；非对齐 E1 事件在未被延迟时也入队，以便内部完成事件取得相应故障地址。外部 ack 或非对齐 E2 事件弹出队首。这一容量服务元数据保存与同周期出入队，不表示总线允许两笔独立未完成事务。请求标签固定为零；元数据不包含发射槽、目的寄存器或 PC，Pipe0/Pipe1 通过其在途类型、有效位和共享暂停，保持返回结果与对应流水线的关联。响应周期按旧队首格式化 A 数据，时钟沿再弹出 A 并写入 B；pending 更新中接纳优先于完成清除，保持新请求的等待状态。

加载返回选择目标字节、半字或整字并执行符号或零扩展，成功结果送入对应流水线，并在满足旁路条件时用于前递。访问错误或内部非对齐完成携带故障地址及异常原因，取消正常目的寄存器更新。请求寄存器接纳后撤销该次请求的有效状态，下一事务只有在新的有效发射和请求握手成立时才接纳。已接纳事务继续完成接口协议，恢复控制负责抑制被取消指令的正常写回。

可缓存 Store 在 D-Cache 完成选中字节更新与脏位置位后得到响应，脏数据通过替换或维护写回 DDR；不可缓存 Store 等待下层写事务完成。共享 DMA 数据使用不可缓存区，根据成功写响应、访问顺序及所有权移交状态确定读取方何时可访问数据。

2.2.6 指令写回

双发射配置的两条通路分别保持 E1、E2 和 WB 的指令状态，在 WB 按有效位、异常与目的寄存器使能提供最多两路寄存器写入；单发射配置仅由 A 通路写回。同对指令保持 A 先 B 后的程序顺序，前序指令异常和恢复控制取消不应提交的后续指令。Store 及分支等无通用寄存器写入的指令仍有完成事件，不能以寄存器写使能代替退休数量。仿真在相同计时窗口采样两条通路的有效完成信息和周期数，IPC 按该窗口内完成指令总数除以周期数计算。

2.2.7 流水线控制

前递网络从两条通路的 E1、E2 及 WB 中选择程序顺序最近的前序写入结果；E1 整数结果可立即前递，Load 及普通 MUL 需等待对应完成阶段。记分牌与源角色信息过滤无效寄存器字段，对结果尚未就绪的前序写入指令及同对 RAW/WAW 冲突施加互锁。LSU 等待、迭代除法、CSR 在途和资源冲突均可阻止新指令发射，暂停期间队首

状态及在途指令与执行通路的对应关系保持稳定。

异常、CSR 返回和分支纠正向前端提供恢复地址，队列冲刷取消旧候选，最终执行使能阻止同周期新副作用，通过通路有效位取消后续指令的写回。这些控制按指令的程序顺序协同工作；已经被外部系统接纳的 Store 不能通过清除流水有效位回滚。双发射关键接口的定向仿真配置、用例及结果列于附录的 CPU 验证部分。

2.2.8 存储系统

图2-11给出指令与数据通路、本地存储及外部 AXI 接口之间的关系。存储系统按物理地址选择本地存储、缓存 DDR 或不可缓存访问。Boot ROM 和片上 SRAM 通过本地接口访问；`0x8000 0000` 至 `0x81FF FFFF` 为 CPU 私有可缓存 DDR，`0x8200 0000` 至 `0x87FF FFFF` 为不可缓存共享 DDR，MMIO 按设备访问处理。缓存子系统同时检查地址范围和请求缓存属性，使共享区及设备寄存器始终绕过缓存。地址路由在请求接纳时保存请求与响应的对应关系，已接纳请求的目标与响应关系保持到事务结束。

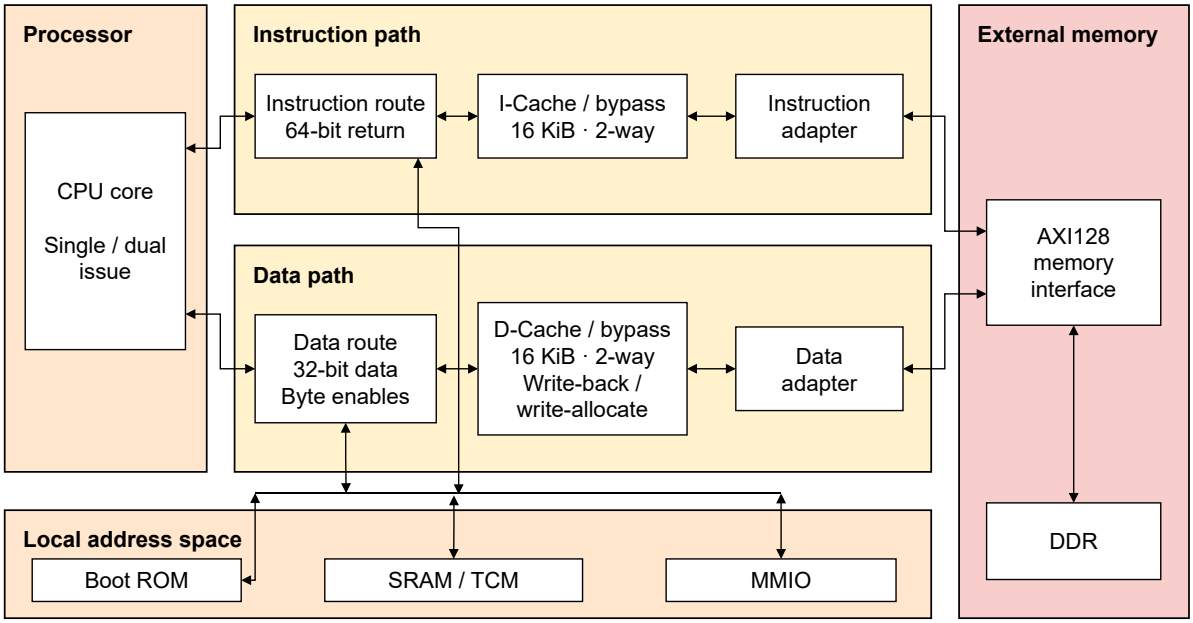


图 2-11 CPU 指令、数据与本地及外部存储接口

I-Cache 和 D-Cache 各为 16 KiB，采用两路组相联、256 组和 32 B 缓存行，数据阵列使用同步 RAM。32 bit 地址划分为 19 bit 标签、8 bit 组索引和 5 bit 行内偏移。查询根据组索引读取两路标签及数据，标签匹配且有效时命中，否则选择替换行并启动填充。替换控制使用全局交替状态选择路，D-Cache 在目标组存在无效行时优先使用无效路。下级 DDR 读通路以 128 bit 返回数据，一条缓存行由两个 16 B 数据拍组成。填充控制接收完整且无错误的响应后才建立有效缓存行；失败填充保持新行无效，并向原访问返回错误。I-Cache 从命中行中选择 64 bit 取指包，D-Cache 向加载存储单元提供 32 bit 数据。

D-Cache 采用写回、写分配策略。存储命中时按字节使能更新行内数据并设置脏

位，完成响应由缓存内部产生。存储未命中时，先处理被替换行，再读入目标缓存行，合并本次写数据并标记为脏行；后续对同一行的多次存储可在片上合并。加载命中从数据阵列取得选中字，成功的可缓存加载响应可与满足条件的下一请求接纳重叠，返回数据通过寄存保持。缓存未命中、维护操作和错误处理由阻塞式状态机串行完成。

替换脏行时，控制器将 32 B 行拆为 8 个 32 bit 字，逐字通过下级写通路提交，并等待各字的成功响应。只有 8 个字全部写回成功，才清除原行脏状态并继续替换或维护。任一写响应失败时，原行保持有效和脏状态，终止后续填充并上报错误。重试以整行为单位重新写回，以覆盖错误发生前已经到达 DDR 的部分数据。

缓存维护操作控制私有缓存数据的写回与失效，具体语义见表2-9。其中“清理”表示将脏数据写回并保留缓存行，“丢弃”表示直接取消有效性，“清理并失效”表示先写回再取消有效性。`0x7C1` 至 `0x7C3` 位于机器模式自定义读写 CSR 区，用于专用缓存维护，避开标准 PMP 地址。维护仅由寄存器形式的 `CSRRW` 触发；单行维护以源寄存器提供目标地址，按缓存行匹配。这三个命令 CSR 读回零，其他 CSR 指令形式不产生缓存维护动作。`0x7C0` 独立用于内部计时比较。

表 2-9 CPU 缓存维护操作

操作	功能	缓存侧完成条件
CSR `0x7C2`	清理匹配的 D-Cache 行，保留有效性	脏行全部写响应成功；未命中或干净行无需写回
CSR `0x7C3`	丢弃匹配的 D-Cache 行，包括其中的脏数据	行失效完成，不产生写回
CSR `0x7C1`	遍历 D-Cache 全部组和路，清理并失效	全部处理完成，或在首个写回错误后产生缓存错误响应
`FENCE.I`	排空先前数据事务，清理并失效 D-Cache，再请求 I-Cache 失效	D-Cache 维护成功且 I-Cache 失效请求脉冲结束后恢复数据接纳；可缓存取指等待 I-Cache 失效完成

表中前三项的完成和错误条件指缓存接口响应，`FENCE.I` 一项列出后续访问的接纳条件。维护指令按 CSR 通路进入 WB，维护事务由加载存储单元和缓存继续跟踪，指令到达 WB 不等同于脏行已经成功写回。维护错误也不经普通加载存储指令的 E2 异常条件上报，软件接口应区分 CSR 指令完成与缓存事务结果。

CPU 私有区用于程序、栈及内部工作数据，DMA 使用的帧缓存、共享张量和模型数据放置在不可缓存共享区，不建立可缓存别名。软件在共享区写入完成后执行 `fence rw,rw`，再将缓冲区标记为就绪并移交使用权；读取方根据完成事件及所有权状态读取数据。普通访问屏障用于约束访问顺序，脏数据写回由专用缓存维护负责。

`FENCE.I` 属于 Zifencei 扩展[1]。该指令发起后，缓存控制暂时停止接纳新的 DDR 取指和数据请求，排空已接纳的数据事务，完成 D-Cache 清理失效后向 I-Cache 发送失效请求。包含该指令的软件使用 `rv32im_zicsr_zifencei` 目标配置。D-Cache 维护

成功且 I-Cache 失效请求脉冲结束后，数据通路恢复接纳；可缓存取指继续由 I-Cache 自身状态机阻塞，直到逐组失效完成。该操作通过缓存侧接纳控制约束后续取指，不向核心提供独立的维护完成响应。内部清理若遇到写回错误，缓存子系统锁存协议故障并停止接纳请求，SoC 路由器将后续普通 DDR 访问转换为错误响应，交由协调复位恢复。CPU、缓存及外部事务控制共同管理复位；需要保留私有脏数据的计划复位应先完成缓存清理，再开始新的缓存有效状态。

2.2.9 中断和异常

CPU 在机器模式下处理异常和中断，机器 CSR 保存处理器状态、使能位、待决位及异常返回信息[2]，主要接口见表2-10。CSR 指令支持寄存器和立即数两种操作数形式，提供交换、按位置位和按位清位操作，各寄存器的可写字段按实现掩码更新。CSR 访问同时检查地址是否实现、地址编码要求的特权级及只读属性。访问未实现地址、越权访问或写入只读 CSR 时产生非法指令异常；`mepc` 保存该指令 PC，`mtval` 保存指令字。`CSRRS/CSRRC` 的源寄存器为 `x0`，或相应立即数形式的立即数为零时只读不写，可用于读取只读计数器。固定 WARL 字段与未实现地址分别处理：`misa` 具有读写地址编码，写入不改变实现固定的 ISA 字段；缓存维护命令 CSR 则按前述约定读回零。

表 2-10 CPU 主要 CSR 接口

寄存器	地址	功能
`mstatus`	`0x300`	中断使能及陷入、返回状态
`misa`	`0x301`	固定返回 `0x40001100`，表示 RV32IM
`mie`、`mip`	`0x304`、`0x344`	机器中断使能及锁存的待决状态
`mtvec`	`0x305`	四字节对齐的机器异常、中断直接入口
`mscratch`	`0x340`	机器处理程序暂存数据
`mepc`	`0x341`	陷入时保存的返回 PC
`mcause`、`mtval`	`0x342`、`0x343`	事件原因及对应异常信息
`cycle`、`time`	`0xC00`、`0xC01`	读取同一周期计数器的低 32 位
`timeh`	`0xC81`	读取周期计数器的高 32 位
内部计时比较	`0x7C0`	比较低 32 位周期值，写入后重新布防
`mvendorid`	`0xF11`	厂商标识，实现为只读零
`marchid`	`0xF12`	架构标识，实现为只读零
`mimpid`	`0xF13`	实现版本标识，实现为只读零
`mhartid`	`0xF14`	处理器编号，固定为 0

CPU 提供一根外部 IRQ 输入，接收核外中断请求并在 `mip` 中形成 MEIP 待决状态。机器软件中断通过 CSR 操作设置 MSIP；内置计时比较器在周期计数低 32 位等于 `0x7C0` 中的比较值时设置 MTIP，单次触发后撤销布防。下一次计时事件由软件写入新的比较值重新设置。外部事件与内部计时事件均使用锁存的待决位，处理程序在处置

中断源后清除相应状态。

中断受 mstatus.MIE 全局使能和 mie 分项使能控制。多个中断同时满足受理条件时，按机器外部中断、机器软件中断、机器计时中断的次序选择，即 MEI > MSI > MTI，原因号分别为 11、3 和 7。中断在流水线允许推进且没有受抑制的在途操作时进入异常控制路径，抑制被接管指令的正常副作用，并将中断状态传递至 WB 异常处理边界。

同步异常包括非法指令、ECALL、EBREAK、指令访问错误、指令地址不对齐、加载或存储访问错误以及数据地址不对齐。异常原因与指令 PC 随流水线传递。mtval 在数据访问及数据对齐异常时记录有效地址，在非法指令时记录指令字；取指类异常记录异常 PC，分支目标不对齐时记录分支指令 PC。中断、ECALL 和 EBREAK 对应的 mtval 为零，软件结合 mcause 判断其含义。

陷入时，异常控制写入 mepc、mcause 和 mtval，将原 MIE 保存到 MPIE 并清除 MIE，然后将取指地址重定向至 mtvec。软件将 mtvec 配置为四字节对齐的统一入口，由处理程序根据原因寄存器分派事件。MRET 从 mepc 恢复 PC，将 MPIE 恢复到 MIE 并置 MPIE 为 1；程序继续在机器模式下运行。异常返回前，处理程序完成现场恢复，并根据异常类型决定保留或调整返回地址。

周期计数器按 CPU 时钟逐拍递增，流水线暂停期间继续计时。软件读取 64 bit 计数时依次读取高位、低位和高位，若两次高位不一致则重读，避免低位进位造成计时撕裂。板级基准使用该计数接口计算运行时间，指令完成数和流水线事件则由验证环境独立统计。

2.3 NPU 设计

2.3.1 NPU 微架构

NPU 由计算图执行器、PE 计算通路、算子控制器、片上缓冲、DMA、卷积窗口生成、辅助算子、定点后处理和整帧寄存器接口组成。激活按算子记录选择 UINT8 或 INT8，权重为对称 INT8，完整卷积和与 Bias 采用 INT32 数据格式。CPU 通过 MMIO 提交整帧任务，绑定模型程序、工作区和本帧输入输出槽；NPU 读取模型程序，解析算子及张量引用，依次完成各层计算和写回。CPU 在整网执行及检测后处理期间持有帧槽和模型工作区，在九个检测头发布后读取结果，完成框解码、NMS 和主人脸选择。

整体结构如图2-12所示。DMA 经 AXI4 连接外部存储，将输入特征图和权重搬入多存储体片上缓冲，并将输出特征图写回。数据流控制模块（Data Streamer）按算子配置组织特征、权重、Bias 及重定标参数输入，以及计算结果的输出通路，使供数顺序与计算单元所需的操作数顺序一致。首层直接读取 Video 写入的模型输入槽，中间张量保存在共享 DDR 工作区，CPU 只配置地址和任务身份。

卷积核心采用可配置并行 MAC 阵列（Configurable PE Array）。普通卷积使用激活、权重广播和本地累加，PE 之间不逐级转发操作数或部分和，属于广播式并行结构。

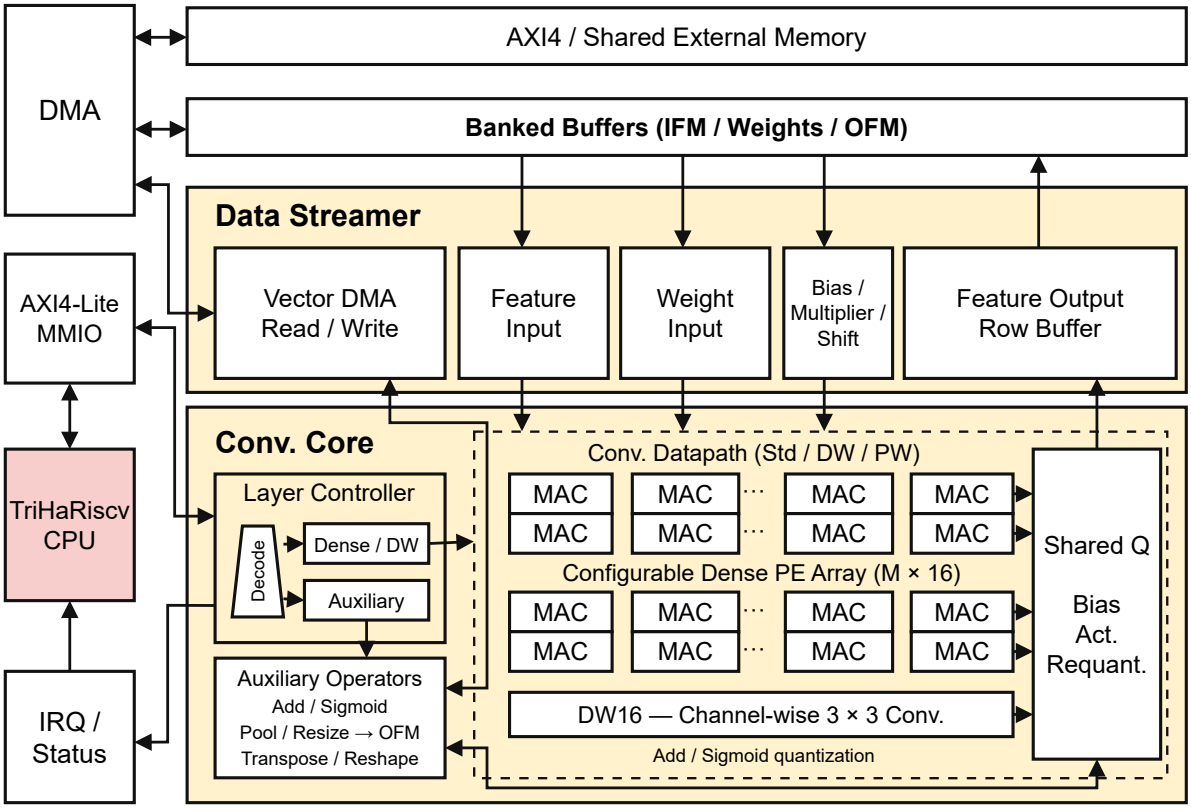


图 2-12 NPU 微架构

深度卷积使用独立的 16 通道乘加通路。两条卷积通路共用定点后处理单元，累加结果经过 Bias、重定标和饱和后送回输出缓冲。Add 与 Sigmoid 通过请求和返回接口使用同一量化通路；池化、缩放和张量重排按各自的数据路径完成读写。图中的 Host CSRs 表示 CPU 经 MMIO 访问的 NPU 控制状态寄存器，数据流控制与卷积核心分别管理搬运和计算参数；整帧控制层组织计算图执行，并通过状态及中断接口报告任务结果。

单个算子的执行依次经历参数锁存、输入与权重搬入、窗口供数、计算、重定标和输出写回。算子执行期间保持参数稳定，计算图执行器依据层完成状态推进下一算子。整图执行结束、DMA 无在途事务且外部服务适配器允许发布后，寄存器接口保持整帧完成记录。CPU 核对命令号、帧号、会话标识及发布状态，完成后处理后通过 ACK 释放记录；整帧执行期间共享工作区保持独占。

NPU 路线采用硬件与模型协同设计，流程如图2-13所示。设计输入包括 FPGA 计算与存储资源约束，以及 YuNet ONNX 模型的算子类型、张量形状和数据量分析，据此确定计算并行度、缓冲组织和算子映射。硬件侧分别完成卷积核心与 DMA、数据流控制、缓冲模块的 RTL 设计，集成后进行仿真、综合和布局布线。模型侧准备校准数据，完成 INT8 校准、层与分块映射及模型量化，导出与硬件配置匹配的权重和参数。两条流程在 FPGA 功能验证阶段汇合，结合检测精度与执行时延评价设计结果。

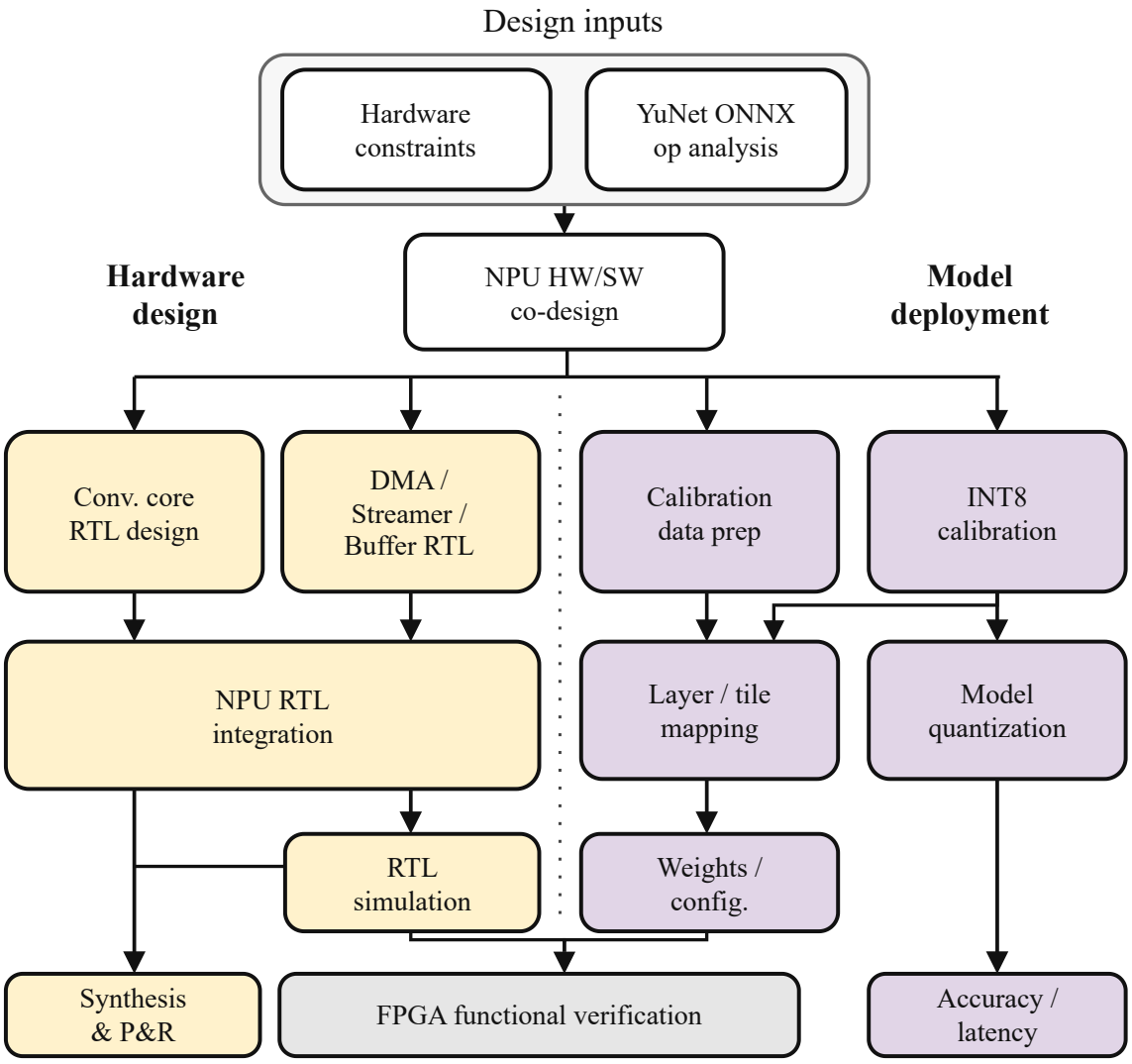


图 2-13 硬件与模型协同设计流程

2.3.2 PE 计算单元

乘法输入由激活类型决定：UINT8 激活零扩展，INT8 激活符号扩展，权重始终符号扩展。首层直接执行 U8×S8，像素不减 128；内部 S8 激活执行 S8×S8，融合 ReLU 后的 U8 激活仍执行 U8×S8。阶段类型见表2-16。各阶段零点均为 0，卷积补边使用数值 0，不需要零点补偿项。

普通卷积阵列含 M 个空间位置、每位置 16 个输出通道，每个被接受的 K 向量完成 $16M$ 次乘加。每个 APM 的两条 SIMD 通路处理同一激活与相邻两个输出通道权重，阵列使用 $8M$ 个 APM。板级配置取 $M = 2$，对应 32 次乘加/有效计算拍；模块默认 $M = 16$，对应 256 次乘加/有效计算拍。空间并行度由综合参数 `M_LANES` 设置，量化并行度由 `QUANTIZER_LANES` 独立设置为 1 或 2。应用配置采用单量化通路，CPU 与 NPU 共同运行于 50 MHz 计算时钟。运行时的形状、步长、有效掩码和地址来自算子记录。供数、后处理和写回开销由控制器另行承担。

每个累加器固定对应一个空间位置和输出通道，沿 $K = C_{\text{in}}K_hK_w$ 归约。普通卷积限定 $K \le 128$、权重范围为 $[-127, 127]$，原始和的绝对值不超过 $128 \times 255 \times 127 = 4\,145\,280$，因此 APM 内部使用 24 位有符号累加，结束后符号扩展到 INT32 接口，再加 Bias。深度卷积为 16 个通道各保留一个 INT32 累加器。有效掩码屏蔽尾通道和尾空间位置，输入有效与计算使能共同控制状态更新。

2.3.3 卷积计算通路

卷积计算通过片上保存与复用输入、权重和部分和，减少重复的外存访问[3]。普通卷积沿输入通道和卷积窗口完成归约，沿输出通道和输出空间位置分配并行工作。1×1 卷积省去空间窗口展开，3×3 卷积由窗口收集器把当前空间块整理到激活缓冲 A。权重缓冲 B 按“输出通道块、K 索引、16 通道”排列，每个 K 索引同时读出一组激活和 16 个权重。激活广播到 16 个输出通道，同一权重广播到 M 个空间位置；部分和始终保留在对应 PE 内。完整 K 结束后才后处理，不在 DDR 中生成完整 Im2Col 矩阵，也不采用跨 K 块的部分和回装。

以首层普通 3×3 卷积为例，输入为 $3 \times 320 \times 320$，输出为 $16 \times 160 \times 160$，步长 2、四边各补 1，$K = 27$。取板级 $M = 2$ 时，一个计算块覆盖同一输出行的两个相邻位置和全部 16 个输出通道。DMA 先把 432 B 有效权重及 Bias、重定标系数搬入片上缓冲；输入 ROI 保留各通道所需的三行，步长 2 换行时保留一行并补入两行。窗口收集器按 (c, k_y, k_x) 顺序生成 27 个 A 向量，每向量含两个有效激活。每个有效计算拍读取一个 A 向量和一个 B 向量，32 个本地累加器分别归约 27 项，得到 32 个输出，计 864 次有效乘加。边界越界位置的激活为 0。权重保留供全层各空间块复用；当输出通道超过 16 时，同一 A 块还可供后续输出通道块复用。

以其后的首个深度 3×3 卷积为例，输入、输出均为 $16 \times 160 \times 160$，步长 1、四边各补 1，输入为 S8。每个空间位置对应一个 16 通道块，ROI 的 16 个通道存储体同时提供同一窗口位置的 16 个激活，B 提供 16 个对应权重。8 个 SIMD APM 完成每拍 16 次乘法，9 个窗口位置依次进入各通道的 INT32 累加器，共 144 次有效乘加；此处不做跨通道归约。通道块沿一整行扫描，九组权重反复读取；换到下一行时三行 ROI 轮换，只补入新的一行。相邻窗口复用 ROI 中的像素，深度卷积直接读取 ROI，不经过普通卷积的 A 展开。DW16 在首个核采样点以零为累加起点，中间采样点更新对应通道的部分和，最后一个合法采样点到达后提交结果。通道掩码屏蔽尾部无效项，任务标签和有效位随计算流水传递，暂停期间保留尚未交付的结果。

上述两层均把量化后的结果写入 O 缓冲。16 通道、160 像素宽、U8 输出时，O 可容纳 6 个完整输出行，形成 $16 \times 6 \times 160$ 的写回 Tile，最后一个 Tile 为 4 行。计算块的 M 个位置与写回 Tile 的行数分别由并行度和缓冲容量决定。计算结果经量化写入 O 缓冲，当前写回 Tile 完成后才复用该缓冲，以上乘加数只描述有效算术工作量。

2.3.4 算子控制器

计算图执行器按模型记录生成层任务，算子控制器锁存形状、核尺寸、步长、补边、张量地址及量化参数，按算子类型选择普通卷积、深度卷积或辅助运算通路。普通卷积按输出行、空间块、16 输出通道块和完整 K 循环；深度卷积按输出行、16 通道块、空间位置和窗口位置循环。执行前检查形状、通道关系、数据类型、参数范围及存储边界；普通卷积的 K 超过 128 或输出行超过缓冲容量时拒绝任务。

控制状态与 DMA 握手衔接，背压时保持地址、数据和计数状态。当前写回 Tile 的计算与量化全部结束后启动搬出，完成响应通过检查后才复用 O 缓冲；最后一个 Tile 完成后报告算子结束。层核结束后仍需等待 DMA 与外部适配器完成事务，层状态交回计算图执行器。跨层依赖、张量引用及下一算子的启动由计算图执行器管理，执行错误随整帧完成状态返回 CPU。

2.3.5 片上缓冲

片上 ROI 按 16 个通道存储体组织，每个存储体为 1024×32 bit，逻辑容量 64 KiB。A、B、O 分别包含 4 个存储体，每个存储体为 1024×32 bit，各 16 KiB；输入行暂存为单存储体 4 KiB，Bias 与重定标参数另存于小型参数存储。这些数值是逻辑容量，物理存储块占用还受端口和映射方式影响。部分和驻留在计算通路的累加寄存器内。

ROI 在整张特征图满足存储体容量时采用驻留方式，否则滚动保留卷积所需的输入行。A 按 K 索引存放空间向量，B 按 K 索引提供 16 输出通道权重，O 通过存储体轮换和双端口接收量化输出并恢复连续 NCHW 写回顺序。DMA 装载、窗口整理、计算和写回分阶段交接缓冲所有权，行复用减少重复读数。输入行命中驻留缓冲时直接向窗口生成器供数；权重读取以请求标签对应返回数据，响应受阻时保持数据与标签稳定。参数缓冲按输出通道保存 32 位 Bias、32 位重定标乘数和 6 位移位数，每次选出八个通道供定点后处理使用。通道有效位记录已装载的参数，本组参数成功装载后置位 `param_valid`，权重读取据此启动。

设输出通道数为 C_o、输出宽度为 W_o、每元素字节数为 b，O 缓冲按下式检查单行容量：

$$\left\lceil \frac{C_o}{4} \right\rceil \left\lceil \frac{W_o b}{4} \right\rceil \le 1024$$

输出行写入器按张量顺序组织数据，依据有效位置、通道和元素宽度生成字节使能，保护尾块相邻的无效区域。输入 FIFO 清空时复位占用计数和载荷有效标志，重新装载完成后才对外提供有效数据，防止旧任务字节进入新任务。行复用、窗口供数和输出尾块的组件验证见附录C.2。

2.3.6 DMA 控制器

DMA 内部采用 256 bit 块服务，每块 1 至 16 个 32 B 数据字，按 AXI4 的 4 KiB 边界约束[4] 及尾部字节拆分。SoC 适配器把一个数据字转换为先低后高的两个 128 bit

AXI4 拍，每个 INCR 突发最多 16 拍。块服务串行持有一个读或写命令，适配器在当前突发完成后再发出下一子事务。输入取数、输出行写回和向量访存共用这一块传输服务。Vector DMA 为 Add、Sigmoid、Transpose 和 Reshape 提供元素块读写接口。本地数据字与 AXI 数据拍的宽度不同，适配器按转换后的拍数重新划分突发，同时检查最终 DDR 地址的边界。

读返回按目标缓冲写入，写事务等待 B 通道成功响应。输出行写回还请求同端口读回，按字节掩码比对后返回块完成。该读回用于检查搬运结果，算子完成时的数据可见性由 DMA 和外部服务适配器共同确认。背压不能改变已提交的事务信息，错误响应与任务标识一并上报。卷积窗口在片上生成。独立读写桥的功能仿真与完成握手验证见附录C.3。

2.3.7 卷积窗口生成

窗口生成器根据输入尺寸、有效通道、步长和补边计算输入地址，从片上缓冲提取 3×3 或 1×1 操作数。图像边界使用量化零值填充，通道对齐区由有效掩码屏蔽。深度卷积供数保持通道对应关系，普通卷积供数按输入通道归约顺序排列。

首层输入在 DDR 中为 B、G、R 三个连续平面，每行 320 B、每平面 102400 B，总计 307200 B。视频输入的上下各 40 行在 NCHW 写入时填 0；卷积窗口超出 320×320 张量时由供数逻辑另行补 0。两种补边均对应零点 0，但发生在不同阶段。通道块尾部另以有效掩码屏蔽。输入行读取模块按需装入行片，窗口生成器依据核位置、步长和补边选择本拍激活。普通卷积将采样值组织为 M 个空间位置的向量；深度卷积按通道组提供同一核采样点的激活；池化和缩放按采样坐标取值。

2.3.8 辅助算子单元

辅助算子单元完成 Add、Sigmoid、MaxPool、Resize、Transpose 和 Reshape，算子控制器按记录选择向量执行或空间采样。Add 通过向量端口读取两路输入，调用共享量化器完成尺度系数乘法，在宽中间域求和后统一舍入与限幅，再写回输出。Sigmoid 先将 S8 输入重定标为查表索引，读取模型包中的概率表，输出 U16 Q0.16 数据。CPU 直接读取概率头用于后处理。Add 与 Sigmoid 的请求、返回均使用有效与接收握手；辅助系数读返回先保存于接收寄存器。运算暂停期间保持已接收的系数，恢复后再更新供算子使用的参数。

MaxPool 按 2×2 窗口、步长 2、无补边取最大值，输出尺寸向下取整。Resize 采用 2 倍最近邻上采样，按非对称坐标映射向下取整选择源位置。两者保持输入编码，采样结果进入 O 行缓冲，经行写入器提交 DMA。Transpose 按部署记录支持的维序生成读写地址，Reshape 保持元素顺序，按目标地址和长度搬运。这两类操作通过向量端口读写原编码数据，不引入新的量化边界；支持的形状和维序由算子准入检查限定。辅助算子的组件状态波形及数值比较结果见附录C.5，各用例按其量化通路配置统计。

2.3.9 定点后处理单元

定点后处理接收 INT32 累加结果，完成 Bias 相加、整数系数乘法、移位、舍入与饱和。融合 ReLU 的特征图输出为 UINT8，非 ReLU 特征图输出为对称 INT8，检测框回归输出为 INT16；输出类型由算子参数选择。权重采用逐输出通道尺度，激活采用逐张量尺度，相关系数随模型包装载。

BN 在线下融合到权重与 Bias，运行时使用融合后的整数参数。完整 K 归约后只加一次 Bias、重定标一次。重定标采用正 INT31 乘数、64 位整数乘积及 1 至 62 位右移，按最近偶数舍入后依输出类型饱和。UINT8 范围为 $[0, 255]$，对称 INT8 为 $[-127, 127]$，INT16 回归值为 $[-32767, 32767]$。最近偶数舍入同样处理负半值，例如 $-9/2 = -4.5$ 舍入为 -4，随后再执行目标类型的限幅。量化接口每组包含八个元素，内部以一条或两条物理通路分时处理，并保留位置标签、有效掩码和输出类型。Sigmoid 概率采用 U16 Q0.16 编码，尺度为 2^{-16}，数值 1 限幅至 65535；其索引重定标与卷积共用量化硬件。检测框解码与 NMS 由 CPU 执行，与本单元的张量重定标分开。卷积累加、尾通道和定点后处理的组件验证见附录C.1。

2.3.10 寄存器接口

整帧寄存器空间基址为 `0x2000 0000`，通过 32 位 AXI4-Lite 提供版本、能力、控制、状态、完成/错误中断及会话信息。偏移 `0x100` 开始的 32 个 32 位字保存整帧任务，偏移 `0x200` 开始的 20 个 32 位字保存完成记录。CPU 写入程序、工作区、输入输出区域、几何参数及命令号、帧号和会话标识，执行 I/O 屏障后写 START；接口在接收启动时原子锁存任务。任务中的地址以共享窗口 `0x8200 0000` 为基准提交字节偏移，计算图执行器据此绑定本帧数据。内部存储服务使用相对 DDR 基址的地址，外部服务适配器加 `0x8000 0000` 形成总线地址。

完成发布同时检查整帧执行结束、局部 DMA 空闲和外部服务适配器的发布许可。接口进入 `HELD` 状态后保持完成记录，CPU 核对执行状态、命令号、帧号、会话标识 `epoch` 及 `published` 标志。应用配置的完成记录给出九头输出的地址与 26,368 B 跨度，CPU 在记录保持期间读取输出并执行检测后处理。驱动通过轮询收取完成记录，硬件同时提供可使能的完成与错误中断。

CPU 完成后处理并构建显示记录后，以匹配的命令号、帧号和会话标识 ACK；确认 NPU 返回 IDLE 后，软件释放整图工作区并将帧置为待显示状态。IRQ 写 1 清除仅清除通知状态，活跃任务与待确认结果均锁定任务区；原图槽继续按显示引用管理。任务执行错误、身份不符或会话变化使本次推理失效，软件保存错误并停止提交后续任务。附录C.4给出单算子 MMIO 与输入 FIFO 的组件控制验证；整帧接口与应用循环的联合验证依据板级配置记录。

2.4 系统互连与存储

2.4.1 总线结构

CPU 采用 32 bit 地址、64 bit 取指返回和 32 bit 数据访问接口，经本地存储路由连接 Boot ROM、SRAM、NPU、Video 及 DDR。应用采用片上程序与数据存储配置，DDR 访问通过不可缓存通路接入共享互连。控制寄存器采用 32 bit AXI4-Lite，数据 DMA 采用 128 bit AXI4，通道握手与响应遵循 AXI 协议[4]，在 DDR 封装处适配厂商 IP 的用户位宽和时钟。AXI 互连按事务所有者串行转发读写请求，完成当前事务后再接纳下一请求。

跨时钟域通路在明确的桥接边界使用异步 FIFO 或握手机制。运行错误使系统停止接纳新任务，已接纳的事务和缓冲引用保持归属，重新启动通过协调复位建立统一会话。

2.4.2 存储组织

CPU 启动存储包括 8 KiB Boot ROM 和 32 KiB SRAM。板载 DDR3 物理容量为 1 GB，软件映射其中 128 MiB 窗口：32 MiB 划为 CPU 私有区，96 MiB 作为不可缓存共享区。应用代码、数据及栈均位于 SRAM。系统地址映射按下表组织，地址属性同时用于 CPU 访存检查、驱动和链接配置。

表 2-11 系统地址映射表

区域	基址	容量或用途
Boot ROM	`0x0000 0000`	8 KiB，启动代码
片上 SRAM	`0x0001 0000`	32 KiB，应用代码、数据及栈
NPU 控制	`0x2000 0000`	整帧任务、完成记录及状态
Video 控制	`0x2001 0000`	采集、显示提交、释放及状态
CPU 私有 DDR	`0x8000 0000`	32 MiB，CPU 私有地址区
共享 DDR	`0x8200 0000`	96 MiB，不可缓存

共享区中，帧槽 A、B、C 的基址分别为 `0x8200 0000`、`0x8220 0000` 和 `0x8240 0000`，每槽 2 MiB。原图位于槽内偏移 0，有效数据量为 1,440,000 B，槽内余量按固定地址布局保留。从 `0x8260 0000` 开始的 2 MiB 为头像素材区，区首保存 256×256 RGBA8888 头像，有效数据量为 262,144 B。

检测输入区从 `0x8280 0000` 开始，三槽基址依次为 `0x8280 0000`、`0x8285 0000` 和 `0x828A 0000`，每槽预留 320 KiB，有效张量为 307200 B。检测输出区从 `0x8290 0000` 开始，预留 2 MiB。输出槽 A、B、C 分别从 `0x8290 0000`、`0x8294 0000` 和 `0x8298 0000` 开始，每槽 256 KiB。320×320 输入对应 2100 个预测位置，每位置包含 6 个 16 bit 值，有效头数据为 25,200 B，各头分配跨度合计 26,368 B。输出头位于槽内偏移 0，其余空

间按公共布局保留；整帧完成信息通过 MMIO 寄存器返回，CPU 读取输出头形成检测结果。

模型区预留 8 MiB，基址为 `0x82B0 0000`。NPM3 程序占 10,240 B，放在区首；CPU 解码表从 `0x82B1 0000` 开始，长度 1,572,864 B。张量与参数工作区从 `0x8330 0000` 开始，预留 64 MiB；本部署模型实际使用 4,876,544 B。参数镜像位于工作区偏移 4,768,256，长度 108,288 B，保存权重、Bias、重定标及辅助算子参数，前部空间用于中间张量。共享区余下 13 MiB 用于装载、测试和扩展。跨层张量按生命周期分配，帧槽、输入输出槽及模型工作区分别管理。CPU 提交整帧任务时，以共享窗口基址 `0x8200 0000` 计算程序、工作区及输入输出槽的相对地址；NPU 计算图执行器解析模型内的张量引用。内部存储服务地址相对 DDR 基址 `0x8000 0000`，服务适配器在发起总线访问时加该基址一次。

2.4.3 访问仲裁

CPU、NPU 和视频 DMA 共享 DDR，互连采用轮询方式选择事务所有者。仲裁器记录每笔事务的发起主端，并将响应、背压和错误返回到原请求方。

带宽统计分别使用实际采集率和显示扫描率。原图及模型输入的写流量按被接纳的完整帧计数，全屏显示读流量按原图扫描率及行缓冲的实际读取量计算。

2.4.4 DDR 带宽预算

每帧采集写入 1,440,000 B 原图和 307,200 B 输入张量，共 1,747,200 B。全屏显示每次扫描读取 1,440,000 B 原图；即使重复显示同一帧，该次扫描仍产生读流量。

40 MHz 像素时钟配合 1056×628 总时序，扫描频率为

$$f_{\mathrm{scan}} = \frac{40 \times 10^{6}}{1056 \times 628} \approx 60.31654\ \mathrm{Hz}$$

以下带宽预算采用该扫描频率。设接纳帧率为 f_{in}，则视频有效载荷带宽为

$$B_{\mathrm{video}} = 1{,}747{,}200 f_{\mathrm{in}} + 1{,}440{,}000 f_{\mathrm{scan}} \quad \mathrm{B/s}$$

表 2-12 两支路采集与全屏显示的 DDR 有效载荷预算

接纳帧率 (fps)	采集写入 (MB/s)	显示读取 (MB/s)	读写合计 (MB/s)	占原生端口峰值 (%)
24	41.933	86.856	128.789	3.220
30	52.416	86.856	139.272	3.482
60	104.832	86.856	191.688	4.792

表中 MB 按 10^{6} B 计，24、30 和 60 fps 为容量预算场景，实际应用吞吐由完整处理链路决定。接纳帧率表示进入处理链路的完整帧数，有效输出帧率表示完成检测并形成可用结果的帧数，两者需分别计数。

DDR3 控制器采用 HMIC_S IP，用户端口和初始化接口依据厂商指南配置。DDR3 配置为 32 bit、1000 MT/s，对应 500 MHz 外部 CK；厂商 IP 原生用户端口为 256 bit、125 MHz，两种口径的理论数据峰值均为 4,000 MB/s。共享 AXI4 数据通路为 128 bit、125 MHz，理想连续传输上限为 2,000 MB/s，以上三种视频载荷分别占其 6.439%、6.964% 和 9.584%。原生端口的读写共享物理存储带宽，不能把两个方向的峰值相加。事务串行仲裁、桥接缓冲、写后读回校验、刷新、行切换和读写换向均会增加占用，因此载荷占比仅用于预算，剩余百分比不是已经测得的可用吞吐余量。该桥在写响应前读回核验已写数据；仅按采集有效字节等量读回一次计算，三种场景还需增加 41.933、52.416、104.832 MB/s 原生读流量，视频及其写校验的合计下限为 170.721、191.688、296.520 MB/s，突发对齐与其他开销需继续计入。

NPU 读取输入、权重和中间张量并写回输出的流量另计；采集端 307,200 B 写入不包含 NPU 随后读取该输入的流量。总预算还需加入 CPU 访存、NPU 计算图执行及启动装载流量，不能只以视频载荷判断整机带宽达标。头像在启动阶段从 DDR 读入片上纹理缓存，连续显示时由缓存供数；全尺寸素材装载与缓存填充单独计入启动流量，不作为每次扫描的 DDR 读流量。应用配置及原始验证记录集中见`docs/verification/soc/face_photo_report_20261104/`。

2.4.5 共享数据访问约定

CPU 应用代码、栈及检测候选工作区位于片上 SRAM，DMA 共享张量、帧数据和 MMIO 使用不可缓存访问。CPU 通过共享 DDR 读取九头输出和解码表，保持与 NPU 写回结果一致的数据视图。写入方完成数据写入并执行所需屏障后发布就绪事件，读取方完成读取后才交还缓冲区。

采集完成等待两条写入分支的成功响应，NPU 完成等待整图执行结束、DMA 无在途事务及外部服务适配器的发布许可，显示切换等待安全帧边界，旧槽释放等待所有显示 DMA、行缓存预取和 FIFO 引用排空。CPU 在完成记录保持期间读取检测头并完成后处理，构建显示记录后 ACK 整帧任务；确认 NPU 返回 IDLE 后归还整图工作区，原图槽继续按显示引用管理。访问属性、总线完成和帧所有权共同决定数据何时可读、可写。

2.5 图像采集与显示链路

图像采集与显示链路连接外部视频源、DDR3 存储器和系统计算模块，完成视频接收、输入预处理、帧缓存读写及显示合成，整体结构如图2-14所示。MS7200 将 HDMI 输入转换为并行 RGB888 像素，FPGA 从同一源帧生成原图和模型输入两类数据并写入 DDR；显示端按行读取已提交的原图，根据模式绘制同帧检测框或合成透明头像，经 MS7210 输出 800×600 全屏画面。

像素处理和数据搬运由 Video 硬件及 DMA 完成，CPU 负责帧调度、检测后处理

和显示提交，NPU 从输入槽读取张量并执行模型计算图。应用提供 RAW、BOX 和 AVATAR 三种模式，分别显示原图、检测框和透明头像；原图引用、检测及头像参数在有效图像边界整体切换。每个采集帧写入 1,440,000 B 原图和 307,200 B 模型输入，共 1,747,200 B。

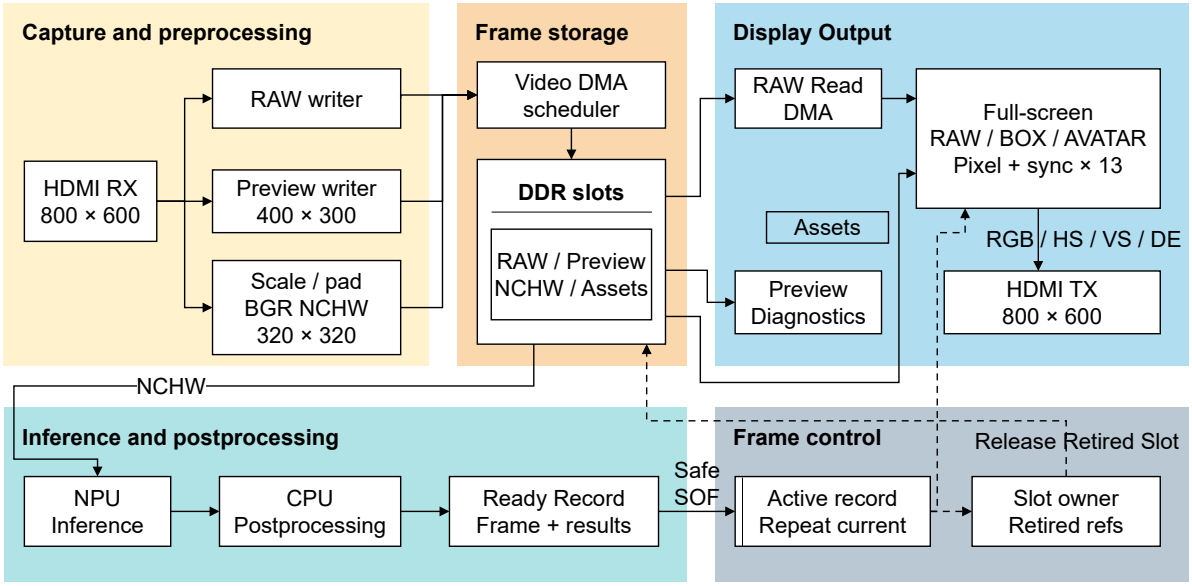


图 2-14 视频子系统结构框图

2.5.1 图像采集

HDMI 接收端提供 24 bit RGB888 像素，以及输入像素时钟、行同步 HS、场同步 VS 和数据有效 DE 信号。HS、VS 用于识别行场边界，DE 限定有效图像区；消隐期间的数据不写入图像缓冲区。板级 I^2C 控制器先配置 MS7210 发送端，再配置 MS7200 接收端。时钟锁定和芯片初始化完成后，入口继续检查输入光栅，并经过默认 128 个连续有效帧的预热过程；DDR 初始化、时钟和接口状态满足要求且模型及素材装载封存后，CPU 进入应用循环并提交采集任务。输入与采集参数见表2-13。

表 2-13 视频输入与采集参数

项目	配置	功能
输入有效区	800×600	每帧 480000 个有效像素
像素格式	RGB888，24 bit	三个颜色分量均为 8 bit 无符号数
输入像素时钟	标称 40 MHz	接收随源输入的像素与同步信号
核心处理时钟	125 MHz	预处理、DMA 与存储访问控制
边界与有效信号	HS、VS、DE	行场边界及有效区标记
入口跨域缓冲	2048 项，48 bit/项	像素、行尾及帧结束信息成包传递
完整帧条件	几何、计数及校验正确	与两路 DMA 写响应及端口排空状态共同判定完成

采集请求锁存槽号、帧号及预处理版本，入口等待下一完整有效帧的起点，再接收该帧。像素包携带 RGB 数据和行尾标记，帧结束包携带有效性、CRC 及入口缓冲占用信息。核心域独立核对行尾位置、帧尾状态和原图 CRC，并分别统计原图 480000 个像素和模型有效图像区 76800 个像素。完整帧检查、两路 DMA 写入结果和端口排空条件全部满足后才发布成功完成事件，完成记录同时给出任务标识、有效字节数及预处理版本。

接收像素域与核心域通过异步 FIFO 连接。核心域仅在原图和模型输入两条路径均可接收当前像素时共同推进，保持两类输出来自同一源帧。HDMI 源按外部像素时钟连续发送，FIFO 用于吸收有限速率波动；入口溢出、尺寸或同步异常、CRC 错误、DMA 故障及 DDR 就绪丢失均锁存首次检测到的错误原因。发生故障后停止发布有效完成结果，已接纳的存储事务继续排空；CPU 记录错误并停止提交后续任务，保留帧池状态和相关所有权。

2.5.2 输入预处理

原图分支保持 800×600 和 RGB888 格式，用于全屏显示；模型分支生成 320×320 UINT8 BGR NCHW 张量。两条路径共享帧标识，分别组织输出和 DMA 写入事务。

模型分支将 800×600 图像按横纵均为 $2/5$ 的比例缩小为 320×240，再在顶部和底部各补 40 行零值，保持有效图像的 4:3 宽高比。固定 5:2 双线性结构以五个源位置为一组生成两个目标位置，水平和垂直方向交替使用 1:3 及 3:1 的权重。行缓存保存水平加权和，再完成垂直组合；分级截断和最终舍入顺序与整数参考一致。补边在写入路径中显式生成，每次任务均写入全部补零区域。

输入张量形状为 $[1, 3, 320, 320]$，内存按 B、G、R 三个平面顺序存放，数据类型为 UINT8，尺度为 1，零点为 0。Video 保留 0 至 255 的像素数值，模型内部的量化及算子重定标由模型包定义。每个平面包含 102400 B，每帧共 307200 B；行跨度为 320 B，平面跨度为 102400 B。像素坐标为 (x, y)、通道编号为 c 时，输入槽内字节偏移为

$$\Delta A = c \times 102400 + y \times 320 + x, \qquad c \in \{0, 1, 2\}$$

其中 $c = 0, 1, 2$ 分别对应 B、G、R，$0 \le x, y < 320$。原图按交错 RGB 字节打包，模型输入按通道平面和行组织，两路写 DMA 经 128 bit 接口和轮转仲裁器共享存储写端口。CPU 核对完成事件中的帧标识、字节数和预处理版本，并按共享存储访问约定确认可见性，再发布输入就绪并启动 NPU。

2.5.3 帧缓存管理

帧缓存放置在 DDR3，片上保留跨域 FIFO、缩放行缓存和显示行缓冲。公共配置分配三个 2 MiB 帧槽和三个 320 KiB 模型输入槽；每个帧槽保存原图，同槽号的独立输入槽保存模型张量。有效数据量与预留容量分开管理，两类数据的组织见表2-14。

表 2-14 图像数据与缓存组织

数据对象	尺寸与格式	有效数据量	槽内布局
原图	800×600 RGB888	1,440,000 B	帧槽偏移 0，行跨度 2400 B
模型输入	320×320×3 UINT8 BGR NCHW	307,200 B	独立输入槽，行跨度 320 B
固定头像	256×256 RGBA8888	262,144 B	独立素材区，行跨度 1024 B

Video 的采集写入和显示读取分别接入共享 DDR 互连，与 CPU、NPU 访问通路共同参与事务轮询。仲裁器锁存事务来源，保持请求与响应的对应关系，完成当前事务后再选择下一主端。桥接层完成 128 bit 访问与 HMIC 256 bit 用户接口之间的转换，行缓冲预取用于衔接存储访问与连续像素输出。

槽号标识物理位置，帧号关联原图、模型输入和结果，代次标识共同复位后的运行会话，槽复用时分配新的帧号；预处理版本限定输入数值含义。CPU 帧池依次维护空闲、采集、就绪、计算、待显示、显示提交等待、正在显示和等待释放状态。CPU 在计算完成记录保持期间读取输出头并完成后处理，构建显示记录后向 NPU 确认任务完成，再释放整图工作区；显示端独立保持对当前原图和叠加参数的引用。

原图槽在计算、后处理及显示引用期间保持所有权。CPU 将同一槽号、帧号和代次的原图引用、检测结果、头像参数及所选模式组成就绪记录，显示记录中的结果帧号与原图帧号一致。显示端仅在匹配的安全 SOF 边界将就绪记录整体转为活动记录，首个有效像素实际输出后确认实际提交。在下一份完整记录到达前，HDMI 重复扫描当前活动记录，而不追逐最新采集帧。因此图像内容更新率可以低于扫描刷新率，同帧匹配不要求 NPU 每个扫描周期均产生新结果。

显示命令接收后进入等待生效状态；首像素反馈返回核心域后，所有权管理模块更新活动记录并发布 COMMIT 事件。新帧生效后，旧槽进入等待释放状态，相关读事务完成、行缓冲和像素流水中的引用解除后发布 RELEASE 事件。CPU 核对提交事件中的槽号、帧号、代次、模式和提交序号，核对释放事件中的槽号、帧号和代次，再通过 ACK 寄存器确认已处理事件。释放 ACK 成功后，硬件和软件帧池才将旧槽恢复为空闲。无空闲槽时，应用等待释放事件后继续分配采集任务。

2.5.4 图像叠加

CPU 完成检测框解码、非极大值抑制和原图坐标裁剪后，将最多 16 个保留框写入显示记录。主人脸从这些框中按面积最大、分数最高、候选序号最小的顺序确定。检测框坐标采用 800×600 原图坐标，左上角包含在矩形内，右下角作为不包含的边界。

RAW 模式输出已提交帧的原图，采集、推理和帧调度继续运行。BOX 模式绘制全部有效保留框，边框在矩形内部宽两像素，主人脸框为红色，其余框为绿色。AVATAR 模式在主人脸位置合成固定头像。有效无脸记录撤销框和头像，使三种模式均显示同

帧原图。每个输出像素核对原图槽号、帧号和代次与活动显示记录的一致性，使叠加参数与对应原图共同生效。

头像素材采用 256×256 RGBA8888，颜色和透明度均为 8 位，字节顺序为 R、G、B、A，颜色使用非预乘形式。CPU 根据主人脸框的左上角 (x_b, y_b)、宽 w、高 h 计算正方形覆盖区域。固定素材的覆盖系数为 1.5，锚点为 $(128, 160)$，其定位过程为

$$S = \left\lceil \frac{384 \max(w, h)}{256} \right\rceil, \qquad c_x = x_b + \left\lfloor \frac{w}{2} \right\rfloor, \quad c_y = y_b + \left\lfloor \frac{h}{2} \right\rfloor$$

$$x_0 = c_x - \left\lfloor \frac{128S}{256} \right\rfloor, \qquad y_0 = c_y - \left\lfloor \frac{160S}{256} \right\rfloor, \qquad x_1 = x_0 + S, \quad y_1 = y_0 + S$$

目标矩形为 $[x_0, x_1) \times [y_0, y_1)$。CPU 保留伸出屏幕的矩形部分，显示端只裁剪可见覆盖范围，使画面边缘处的纹理比例保持一致。

Video 将覆盖区内像素 (x, y) 映射到素材坐标 (u, v)，采用向下取整的最近邻采样：

$$u = \left\lfloor \frac{256(x - x_0)}{S} \right\rfloor, \qquad v = \left\lfloor \frac{256(y - y_0)}{S} \right\rfloor$$

坐标映射采用定点流水，每个像素时钟可接收一组坐标。设素材颜色通道为 F、原图通道为 B、透明度为 a，逐通道合成结果为

$$C = \left\lfloor \frac{aF + (255 - a)B + 127}{255} \right\rfloor$$

$a = 0$ 时保持原图，$a = 255$ 时输出素材颜色，中间值按整数规则舍入。输入预处理的 5:2 双线性缩放与此处的头像最近邻采样分别处理模型输入和显示纹理。

固定头像采用无损行区间缓存。每行保存最左至最右非零 RGBA 像素之间的完整连续区间，区间外恢复为全零 RGBA；区间内部零值及透明像素中的非零 RGB 数据均保留。该素材共保存 34,331 个 RGBA 字，片上像素载荷为 137,324 B，另设行区间表和控制状态。装载器封存模型及素材后，头像读取器从 DDR 读取完整 262,144 B 数据并校验 CRC，再按行区间写入缓存；最后一次物理写入完成后才发布素材就绪。缓存身份为素材编号 1、代次 2，与显示记录共同参与提交校验，运行期间内容保持稳定。更换固定头像时，素材数据、身份及行区间表配套生成，并重新构建固件和位流。

合成通路以 16 项、每项 84 位的环形 RAM 保存原图颜色、坐标、同步信息及帧号，与纹理读取响应对齐。上下文写入后第 9 拍同步读取，第 10 拍用于混合，原存储项到第 16 拍才再次写入；复位后的填充计数屏蔽旧内容。RGB、HS、VS、DE、坐标与帧标识沿像素流水同步传递，合成像素直接送往 HDMI 发送端，省去整帧叠加图的 DDR 写回。

2.5.5 显示输出

显示读取器依据已提交记录中的原图地址、行跨度和帧标识按行获取 RGB888 数据。每行 2400 B，对应 150 个 128 bit 数据拍；核心域预取下一行，像素域读取当前行，

双行缓冲衔接突发访问和连续扫描。原图和头像读取均采用 16 B 对齐、最多 16 拍的 INCR 突发，并按剩余数据量及 4 KiB 页边界缩短当前事务；读仲裁器锁存来源直至响应完成。新帧数据准备完成后在有效图像起点切换，准备不足时保留已有有效帧。扫描期间检测供数不足和读响应错误，并锁存故障状态。

输出采用 VESA DMT 的 800×600 时序[5]，像素时钟为 40 MHz，光栅参数见表2-15，水平和垂直同步均为正极性。扫描频率为

$$f_{\text{scan}} = \frac{40 \times 10^6}{1056 \times 628} \approx 60.32\,\text{Hz}$$

扫描频率按重复光栅计数，新图更新率按不同采集帧计数，模型处理率按完成整图推理计数。

表 2-15 显示光栅时序参数

时序项目	水平方向（像素时钟）	垂直方向（行）
有效区	800	600
前肩	40	1
同步脉宽	128	4
后肩	88	23
总周期	1056	628
同步极性	正极性	正极性

固件初始选择 RAW 模式，KEY1 按 RAW、BOX、AVATAR 顺序循环请求模式。CPU 通过显示接口写入 640 B 记录，填充原图地址、行跨度、槽号、帧号及代次、结果标识、检测框、主框索引、素材身份及头像目标矩形，再写入显示模式、提交序号并发出提交请求。没有新帧时，CPU 可对活动帧进行仅模式更新；硬件逐字核对完整记录，保持原图与检测结果不变。显示接口以 RAM 保存完整影子记录及活动历史，内部准备寄存器保留 32 字头部和 16 组各 2 字矩形坐标，另用标志记录非法载荷，外部 160 字接口保持完整。硬件检查记录格式、地址范围和帧身份，带头像的非空记录还需等待素材就绪并匹配素材身份；跨域握手期间保持整组参数稳定，在匹配的有效图像起点整体应用。VSYNC 维持固定的扫描光栅，COMMIT 反馈标识新记录的实际像素生效；旧帧引用解除后另行返回 RELEASE 事件。显示接口分别保持提交和释放事件，供 CPU 读取并确认。

2.6 模型部署与应用软件

2.6.1 模型选型

系统采用 YuNet 人脸检测网络[6]，以 OpenCV Zoo 发布的`face_detection_yunet_2023mar.onnx` 为权重来源[7]。YuNet 采用无锚框检测结构，由特征提取主干、轻量特

征金字塔（TFPN）和检测头组成，原始网络结构如图2-15所示。主干利用逐点卷积与深度卷积提取图像特征，TFPN 通过上采样和逐元素相加融合不同尺度的特征，检测头据此预测不同尺寸的人脸。逐点卷积和深度卷积分别映射到 NPU 的通道归约与通道独立计算通路，固定输入尺寸便于离线确定各层形状及存储布局。

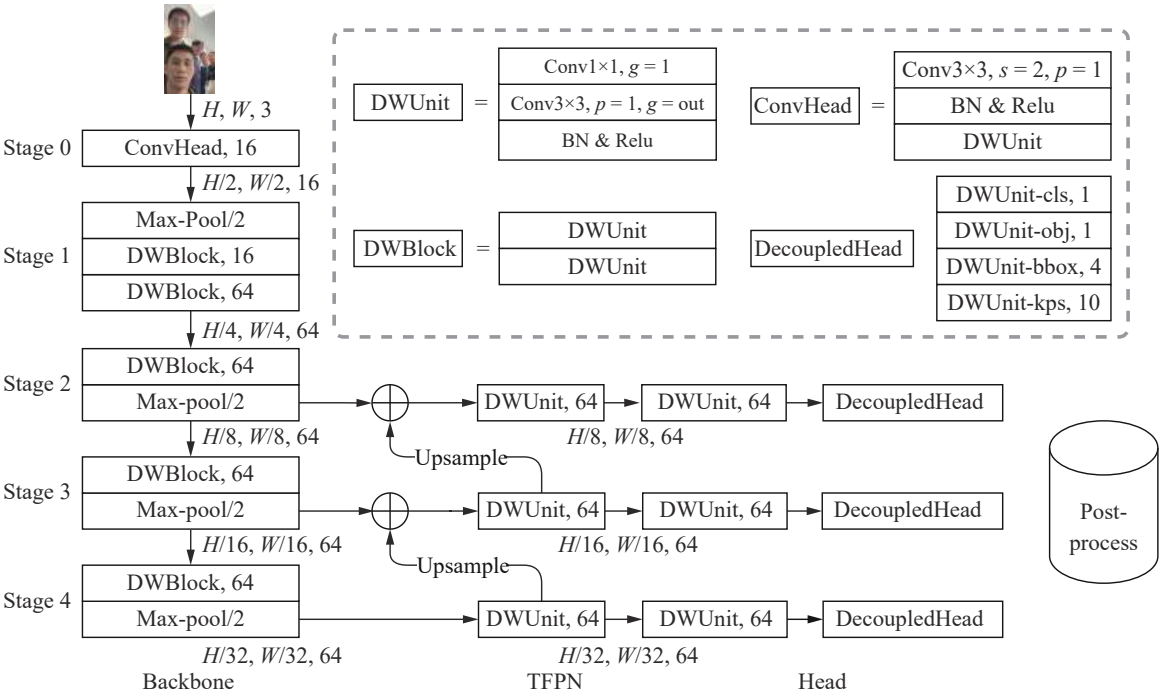


图 2-15 YuNet 原始网络结构（引自文献[6] 图 4，CC BY 4.0）

本项目部署图将输入固定为 $[1, 3, 320, 320]$，裁剪原图中的关键点预测（kps）及其依赖分支，保留分类概率 cls、目标概率 obj 和检测框 bbox 输出，保留分支的权重不变。步长为 8、16、32 的三个检测尺度分别对应 40×40、20×20 和 10×10 的网格，每个尺度输出 cls、obj 和 bbox，共九组输出。对于板级 800×600 输入，Video 将图像缩放为 320×240，上下各补 40 行零值，形成 320×320 模型输入。NPU 完成定点网络计算，CPU 对九组输出进行解码和非极大值抑制（NMS），选择主人脸并提交显示记录，Video 在同帧原图上绘制检测框或合成透明头像。

部署包记录权重来源、版本和哈希，以及固定形状计算图的算子、张量和输出顺序。输入尺寸、预处理、量化和后处理参数作为同一配置管理，浮点与整数参考采用相同的输入画面和候选筛选规则。

2.6.2 量化方案

外部输入采用 UINT8 BGR NCHW，保留 0 至 255 像素值，尺度为 1、零点为 0。首层直接以 U8 激活乘 S8 权重，不进行减 128 转换。网络内部按张量选择 U8 或 S8：融合 ReLU 的输出为 U8，非 ReLU 的 8 位输出为 S8。权重采用 $[-127, 127]$ 的对称 INT8 编码和逐输出通道尺度，激活采用逐张量尺度[8]，全部零点为 0。Bias 按输入尺度与

权重尺度的乘积量化为 INT32，完整卷积和加 Bias 后仅重定标一次；系数乘法保留 64 位精度，最近偶数舍入后按输出范围饱和。BN 融合、重定标系数及移位在线下生成，Add 以共同移位分母对两路输入尺度对齐后相加，只进行一次舍入和饱和。

表2-16列出卷积主干与检测框、概率输出的类型。表中 S8、S16 使用对称范围 $[-127, 127]$、$[-32767, 32767]$；U8 和 U16 分别覆盖 $[0, 255]$、$[0, 65535]$，激活的符号属性随算子记录切换，零点列按激活/权重顺序列出。

表 2-16 NPU 各阶段整数类型

阶段	激活	权重	零点	累加类型	输出
视频输入与补边	U8	—	0/—	—	U8
首层 3×3 卷积	U8	S8	0/0	S24 本地和，扩展至 INT32	U8（ReLU）
后续普通/逐点卷积	U8 或 S8	S8	0/0	S24 本地和，扩展至 INT32	S8
深度卷积	S8	S8	0/0	INT32	U8 或 S16
概率 Sigmoid	S8	—	0/—	索引重定标、查表	U16

首层 U8 输出经后续逐点卷积重定标为 S8，再进入 S8×S8 深度卷积。视频上下补边与卷积窗口越界补边均写入整数 0，表示实数 0；没有输入偏移引入的 Bias 补偿项。

离线工具以代表性图像进行训练后量化（PTQ）校准[8]，记录逐层浮点与整数误差，硬件按整数参考中的舍入及饱和规则执行。板级 800×600 输入采用固定 5:2 双线性缩放，按 BGR 顺序组织，并在上下各补 40 行零值。WIDER 校准与验证按各图像的原始长宽比，将长边缩放至 320 像素，短边尺寸按就近取整、半数取偶规则确定，再居中补零至 320×320；奇数余量置于右侧或下侧。FP32 与软件整数参考共用该数据集预处理流程。输入不附加除以 255 的归一化，也不补齐至八个通道。

PTQ 从 WIDER FACE 训练集中选取 512 张与验证集无内容重复的图像进行校准，在全部 3226 张验证图像上比较 FP32 与 bbox16 软件整数参考。bbox16 保留 INT16 框回归输出，概率采用 U16 Q0.16，其余卷积激活按 U8/S8 编码。AP 评价采用候选分数阈值 0.001、NMS IoU 阈值 0.3 及 5000 的候选上限，按官方难度子集以 IoU 不小于 0.5 匹配，计算 1000 个阈值的 VOC AP；0.6 用于单独的工作点分析。结果如下表所示。

表 2-17 YuNet 软件整数参考的检测精度

难度	FP32 AP（%）	bbox16 AP（%）	AP 下降（百分点）
Easy	78.3929	78.2039	0.1889
Medium	67.2333	66.9742	0.2591
Hard	31.5613	31.3584	0.2029

AP 下降按未舍入原值计算，各项分别保留四位小数。Easy 差值 0.188946 个百分点舍入后为 0.1889。在本项目固定 320 单尺度预处理及相同后处理、评价规则下，bbox16 相对 FP32 的 AP 最大下降为 0.2591 个百分点，整数方案的增量损失较小。当前 FP32 基线的 Hard AP 为 31.5613%，固定 320 输入下的小脸检测仍受限制。OpenCV Zoo 参考评价按原图尺寸设置输入，候选筛选和结果处理也与本项目不同，因此本表不与官方公布 AP 直接比较[7]。本表使用主机端解码及 NMS，评价对象为软件整数参考；融合固件定点后处理和板级视频链路的端到端 AP 需另行评测。

2.6.3 模型导出

导出流程生成固定形状算子表、权重、Bias、量化参数、查找表及张量工作区布局。逐层记录输入输出形状、有效与存储通道、核尺寸、步长、补边、分组、搬运字节数和生命周期。模型输入使用 B、G、R 三平面 NCHW 布局，中间张量的物理布局、对齐和步长由算子表逐项指定。

部署程序采用 NPM3 格式，128 B 程序头后依次保存 79 条 128 B 算子记录，总长 10,240 B。记录给出算子类型、形状、张量引用和数值参数，头部保存模型身份、区域尺寸及校验信息。模型包包含版本、哈希、预处理配置、输出格式和逐层参考向量。硬件装载器完成各段 CRC 校验与写入确认后封存模型，CPU 提交绑定输入、输出及工作区的整帧任务。NPU 计算图执行器读取程序，解析算子和相对张量引用，并按本帧绑定关系生成层任务。算子表还记录每层的输入输出区域；跨层连接所需的原张量保留至最后一个使用该张量的算子执行完成。计算图推进、层内遍历与数据搬运均由 NPU 完成，模型程序及参数在运行期间保持不变。CPU 解码表独立存放于共享 DDR，由软件在检测后处理时按候选读取。

2.6.4 算子映射

普通与逐点卷积使用输入通道归约模式，深度卷积使用通道独立模式；池化、Add 和上采样由 NPU 辅助单元执行。NPU 辅助通路完成概率 Sigmoid 查表，CPU 处理输出候选、框解码和 NMS，Video 承担输入逐像素处理及显示合成。部署程序包含 30 条普通/逐点卷积、17 条深度卷积和 32 条辅助算子记录，共 79 条；普通/逐点卷积中 23 条为 1×1、7 条为 3×3。算子执行方式如下表所列，完整特征图运算均在 NPU 中完成。

表 2-18 算子执行方式表

算子或阶段	数量	执行单元	执行方式	数值约定
普通/逐点卷积	30	NPU	空间块、16 输出通道块，完整 K 归约	U8/S8 乘 S8，INT32 累加结果
深度卷积	17	NPU	16 通道并行、3×3 逐通道计算	各通道 INT32 部分和独立
Add	2	NPU	两源读取、共享量化、向量写回	尺度对齐后统一舍入和饱和

表 2-18 算子执行方式表（续）

算子或阶段	数量	执行单元	执行方式	数值约定
MaxPool	4	NPU	2×2、步长 2、无补边	尺寸向下取整，保持原编码
Resize	2	NPU	最近邻 2 倍上采样	非对称坐标映射、向下取整
Transpose	9	NPU	按固定维序读写	保持元素编码
Reshape	9	NPU	按地址和长度搬运	保持元素编码与序列
概率 Sigmoid	6	NPU	索引重定标、定点查表	U16，尺度 2^{-16}
Bias、重定标、激活	—	NPU	融合在卷积记录中	U8/S8/S16 输出，显式饱和
解码、NMS	—	CPU	候选筛选、定点查表与排序	输出尺度统一，检查中间位宽
缩放、输入格式化	—	Video	流式逐像素处理	UINT8 BGR NCHW、上下补零
图像叠加	—	Video	原图读取、边框绘制及头像透明合成	RGB888 背景、RGBA8888 纹理

2.6.5 检测后处理

YuNet 部署图按 stride 8、16、32 输出 cls、obj 和 bbox，320×320 输入对应 1600、400 和 100 个位置，共 2100 个预测位置。每位置包含 1 个分类概率、1 个目标概率和 4 个框参数，最终均以 16 位格式交接，有效数据量为 $2100 \times 6 \times 2 = 25,200\,\mathrm{B}$。各输出头按部署清单分配跨度，合计 26,368 B；下表偏移均相对于本帧输出槽基址，数据采用小端格式。

表 2-19 九个检测头的输出布局

输出头	编码	字节偏移	有效字节	分配字节
`cls_8`	U16 Q0.16	0	3200	3328
`cls_16`	U16 Q0.16	3328	800	1024
`cls_32`	U16 Q0.16	4352	200	256
`obj_8`	U16 Q0.16	4608	3200	3328
`obj_16`	U16 Q0.16	7936	800	1024
`obj_32`	U16 Q0.16	8960	200	256
`bbox_8`	S16	9216	12800	12800
`bbox_16`	S16	22016	3200	3328
`bbox_32`	S16	25344	800	1024
合计	—	—	25200	26368

CPU 对 NPU 输出的 U16 概率进行阈值筛选，再对保留候选解码；Sigmoid 在 NPU

中查表。CPU 解码表保存按尺度索引的偏移及尺寸变换值，软件将其 FP32 编码转换为含 16 位小数的定点数，使用整数运算恢复检测框。

参考分数为 $\sqrt{cls \cdot obj}$，筛选阈值为 0.6。概率头采用相同尺度，软件先比较概率乘积与阈值平方对应的整数界限，再为通过筛选的候选生成舍入后的分数。候选按分数降序、候选序号升序处理，使用 64 项缓冲分页排序；NMS 保留结果跨页累计，直到得到 16 个有效框或遍历完符合阈值的候选。贪心 NMS 在 IoU 大于 0.3 时抑制候选，利用 $10I > 3U$ 比较交集面积 I 和并集面积 U，以整数运算实现门限判定。检测坐标扣除补边后恢复到原始 800×600 画面：

$$x_{\text{raw}} = \frac{5}{2}x_{\text{net}}, \qquad y_{\text{raw}} = \frac{5}{2}(y_{\text{net}} - 40)$$

框采用左闭右开范围，坐标扣除补边并裁剪至有效图像区域后按 5:2 映射到原图，左上角向下取整、右下角向上取整；删除零面积框及补边区无效候选。

主人脸从保留的 NMS 结果中选择，优先取有效面积最大者，同面积时按分数高者、候选序号小者决定。CPU 将保留框和主人脸索引写入显示记录，并以主框尺寸及素材锚点计算头像覆盖区域。显示模式为 RAW、BOX 或 AVATAR，由按键请求经软件提交；头像定位、缩放与透明合成见第2.5.4节。显示记录中的原图坐标与帧号、槽号及会话身份共同提交，使检测结果与所显示的原图对应。

2.6.6 裸机运行时

配置位流初始化 Boot ROM 及 SRAM 中的固件内容，CPU 从 Boot ROM 跳转到 `0x0001 0000` 处的应用入口。应用代码、只读数据、变量和栈位于 32 KiB SRAM；启动代码设置栈及异常入口、清零 BSS，然后进入主循环。DDR 初始化和 HDMI 器件配置由板级硬件完成，模型及头像素材通过硬件 UART 装载器在启动阶段写入 DDR。

装载顺序为 NPM3 程序、参数镜像、CPU 解码表、A、B、C 三个输出槽的头数据初值及 RGBA 头像。前六段模型数据共 1,770,496 B，头像为 262,144 B，七段合计 2,032,640 B。UART 采用 115200 baud、8N1，装载器按 256 B 载荷接收数据包，核对段号、偏移和 CRC，并等待 DDR 写入确认；七段数据均通过校验后封存模型及素材。仅传输有效载荷的理论时间为 $2,032,640 \times 10/115200 \approx 176.444$ s。计入每包 16 B 包头及 12 B 确认包后，7,940 包的停等协议线速预算约为 195.743 s，尚未计入主机调度、确认周转及 DDR 等待。装载记录没有保存实际起止时刻，因此上述数值仅为启动传输预算，与逐帧推理耗时分别统计。CPU 轮询 NPU 就绪状态，重复读取 IDLE 与有效会话标识并确认一致，随后初始化三帧槽、选择 RAW 模式并开放采集和推理。

驱动封装整帧 NPU、采集、显示及帧池接口，公共配置生成硬件与软件共用的地址和寄存器定义。帧记录维护槽号、帧号、会话身份、地址及状态；CPU 候选缓冲和显示记录位于 SRAM，中间张量及九头输出位于共享 DDR。应用采用轮询主循环，依次处理显示事件、采集完成、NPU 完成、显示提交和下一帧采集。NPU 忙时轮询返回主循环；整帧结果就绪后，CPU 在本次收取过程中完成后处理并生成显示记录。运行

计数及错误信息保存在 SRAM。KEY1 经同步和消抖后产生模式请求，CPU 按 RAW、BOX、AVATAR 顺序循环选择，使用完整显示记录和提交序号更新模式。KEY0 执行协调复位并清除装载封存状态，重新运行需再次上传七段数据；断电后还需重新下载 FPGA 配置。

2.6.7 帧调度

软件通过帧池中的整图活跃标志和帧身份管理计算所有权。完整帧的 RAW 与模型输入两条采集支路写入结束后进入 READY；当没有活跃整图任务且 NPU 处于 IDLE 时，CPU 选择最早就绪的帧，锁定该帧和共享模型工作区。CPU 绑定程序、工作区及本帧输入输出地址，写入 32 字整帧任务并启动 NPU；计算图执行器依次完成模型各层。

NPU 发布整帧完成记录并进入 HELD 后，CPU 核对执行状态、命令号、帧号、会话身份、输出区域及发布标志，再读取同帧九头结果。CPU 完成定点解码、NMS 和主人脸选择，计算头像覆盖区域，构建显示记录，然后以匹配身份 ACK 整帧任务。确认 NPU 返回 IDLE 后，帧池释放整图工作区并将帧置为待显示状态，允许启动下一帧计算。

CPU 提交显示记录后将帧置为显示提交等待状态，显示命令接收后继续等待像素端 COMMIT 事件，确认实际生效后转为正在显示状态。旧显示槽的读取和流水引用排空后产生 RELEASE 事件，CPU 核对槽号、帧号及会话身份并成功 ACK 后，才将该槽归还用于采集。采集、检测完成和实际显示分别计数；没有空闲帧槽时暂停分配新的采集任务。采集下一帧、处理当前帧、显示上一完成帧通过同一套所有权协议重叠运行。

2.6.8 异常处理

启动阶段等待硬件就绪与模型封存，任务提交时核对接口版本及会话身份。运行中发生 I/O 错误、NPU 执行错误或帧事件不匹配时，驱动在 SRAM 中保存首个错误域和错误码，帧池进入错误保持状态，主程序记录运行状态后停驻。计算和显示引用保留原有归属，软件停止提交后续任务。

CPU 异常入口将 `mcause`、`mepc` 和 `mtval` 保存到 SRAM，并保持停驻以保留现场。有效无脸结果按正常完成流程发布空检测记录，使同帧画面清除检测框和头像；执行错误保留错误状态和帧引用。

2.7 FPGA 实现

FPGA 顶层集成 TriHaRiscv SoC、视频子系统、共享 DDR 互连及板级支撑逻辑，整体结构如图2-16所示。SoC 与 Video 作为两个独立模块，通过寄存器接口传递采集和显示控制信息，通过共享 DDR 交换帧数据、模型输入及计算结果。HDMI 收发芯片、

DDR3 芯片和 USB-UART 接口位于 FPGA 之外，由顶层完成信号连接与时序适配。

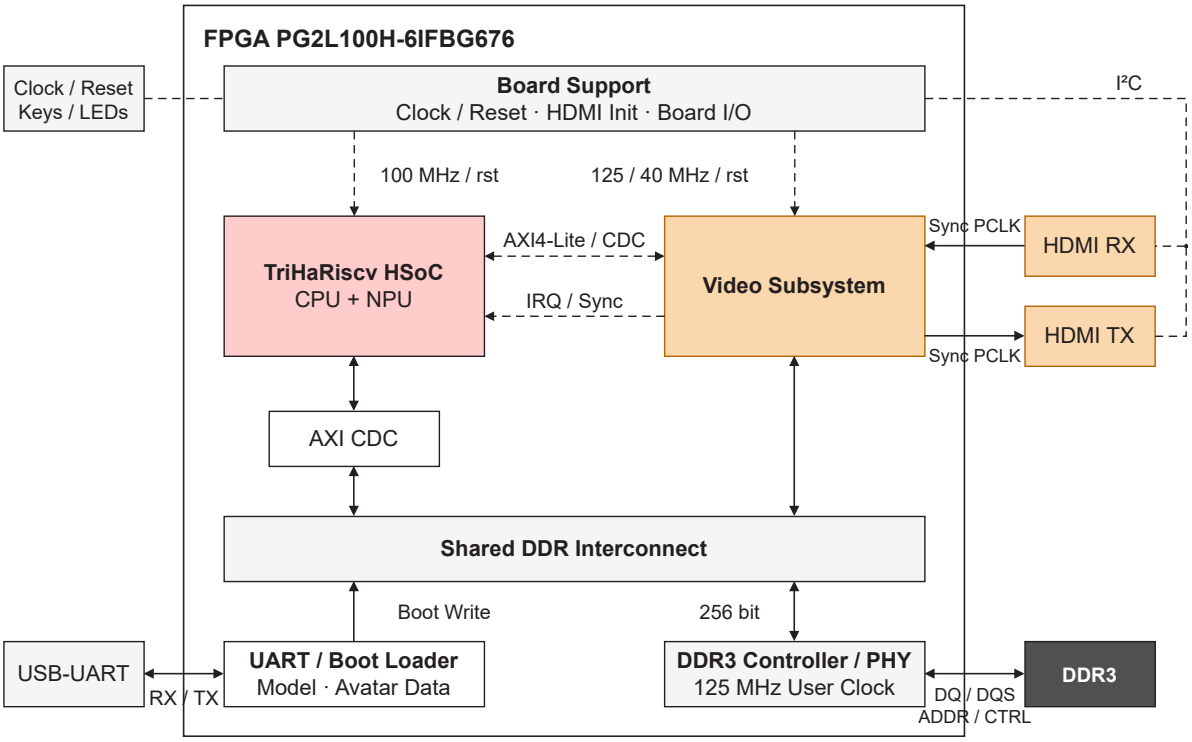


图 2-16 FPGA 顶层架构与板级接口

SoC 内部包含 CPU、NPU、片上存储和控制接口，人脸检测应用运行于 50 MHz 计算时钟域。CPU 与 NPU 的外存事务汇入 128 bit AXI4 通路，经跨时钟域桥进入 125 MHz 的共享 DDR 互连。Video 的采集写入和显示读取直接接入该互连；其控制寄存器通过 AXI4-Lite 跨域桥与 SoC 连接，中断信号经同步后送入 SoC。共享互连完成访问仲裁及 128 bit 至 256 bit 的位宽转换，通过厂商 IP 用户端口连接 DDR3 控制器与 PHY，再由 32 bit 物理接口访问板载 DDR3。

串口装载器在启动阶段接收模型程序、参数、CPU 解码表、三份输出头初值及 RGBA 头像，经共享 DDR 写通路写入约定地址。七段数据通过写入确认与 CRC 校验并封存后，启动写通路交还给视频采集逻辑，头像读取器从 DDR 校验并填充片上缓存。运行期间的采集、推理与显示共同使用同一 DDR 控制器。HDMI 接收芯片提供 RGB 像素、同步信号和输入像素时钟；视频子系统完成采集、预处理与显示合成后，将输出像素和同步信号送入 HDMI 发送芯片。

板级支撑逻辑集中处理时钟分配、复位同步、HDMI 芯片初始化及按键和 LED 接口。计算时钟和输出像素时钟由 PLL 生成，125 MHz 的 Video 核心与 DDR 用户时钟由 DDR IP 提供，各时钟域分别同步释放复位。HDMI 收发芯片通过 I^2C 完成初始化，输入像素跨域采用异步 FIFO。图中实线表示数据连接，虚线表示控制、时钟及复位关系。

2.7.1 目标平台

本设计采用盘古 100pro+ 开发板，主控为紫光同创 Logos-2 系列 PG2L100H-6IFBG676。CPU、NPU 和视频逻辑使用片上 LUT、FF、APM 及 DRM 实现，器件资源容量依据 Logos2 数据手册，平台主要配置见表2-20。板载 DDR3 由两颗 4 Gbit 芯片组成，总容量 1 GB，数据通路 32 bit；模型程序、参数、中间张量和三帧槽位于软件规划的 DDR 窗口中。

表 2-20 平台配置表

项目	配置	说明或用途
开发板	盘古 100pro+（MES2L676-100HP）	FPGA 及板级外设平台
主控器件	PG2L100H-6IFBG676	紫光同创 Logos-2 系列 FPGA
逻辑资源	66,600 个 LUT，133,200 个 FF	CPU、NPU 及系统控制逻辑
运算资源	240 个 APM	乘法与 MAC 运算
片上存储	155 个 36Kbit DRM	CPU 存储、Cache 及 NPU 缓冲区
外部存储	1GB DDR3，2 颗 4Gbit 芯片，32bit 数据位宽	模型参数、特征图及视频帧存储
视频接口	1 路 HDMI 输入、1 路 HDMI 输出	视频接入与显示
通信接口	USB-UART	板载通信与串口调试
辅助接口	按键、LED、SD 卡接口	控制输入、状态显示及扩展存储
下载与供电	JTAG、12V 电源接口	比特流下载、逻辑调试和板卡供电

开发板实物及连接如图2-17所示。图中可见 FPGA 核心板、板载 DDR3 及 HDMI 输入输出接口，外接 JTAG 下载器用于比特流下载与逻辑调试。

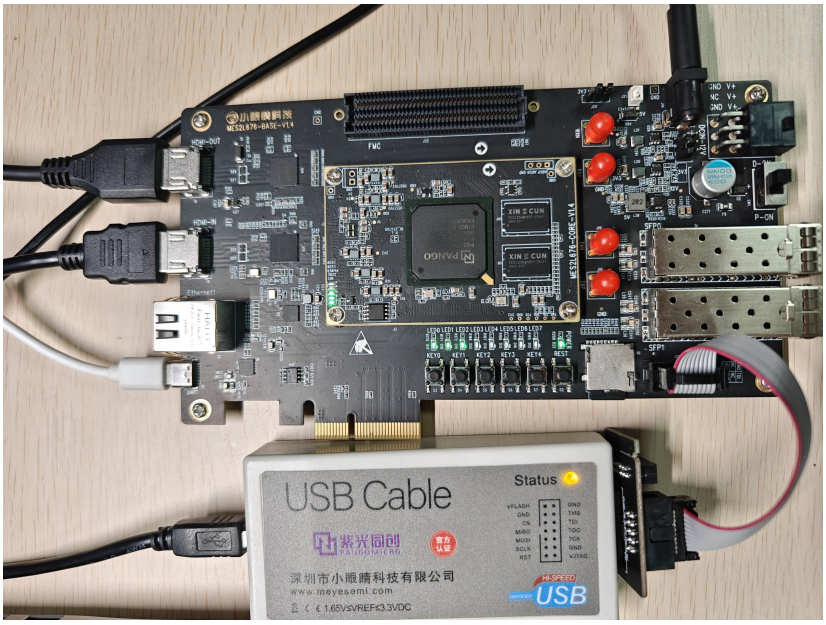


图 2-17 盘古 100pro+ 开发板实物

2.7.2 资源统计

表2-21列出人脸检测整机的综合资源，统计对象为同一综合网表中的顶层及主要模块层级。DRM 按 36 Kbit 等效块计，两个 18 Kbit 原语计为一个等效块；LUT 总数包含分布式 RAM。整机采用单发射控制 CPU、两空间行 Dense 阵列、单量化通路和 RAW、BOX、AVATAR 显示配置，包含固定照片的无损行区间缓存及合成流水。

表 2-21 人脸检测整机综合资源

统计对象	LUT	FF	DRM	APM
整机顶层	45716	41848	113.5	64
应用 CPU 及本地存储、路由	3416	2739	18.5	4
NPU	22968	17818	33.5	49
Video 应用引擎	9944	11920	43.5	10
DDR 厂商 IP	4701	5157	0	0
共享 DDR 仲裁与桥接	1350	689	4	0
计算数据跨域桥	71	153	7	0
视频控制跨域桥	61	149	2.5	0

顶层占用器件 68.64% 的 LUT、31.42% 的 FF、73.23% 的 DRM 和 26.67% 的 APM。模块行均包含在整机总数中；顶层还包括装载、初始化及状态控制逻辑，合计直接采用顶层报告。应用 CPU 一行包含 ROM、SRAM 和地址路由，与第三部分独立 CoreMark 测试中的 CPU 核统计范围分别记录。

NPU 的 A、B 和 O 缓冲声明容量各为 16 KiB，ROI 为 64 KiB，行暂存为 4 KiB；部分和保存在计算通路的累加器中。综合依据空间并行度和有效端口裁剪存储体，表中 33.5 个 DRM 等效块为该配置的实际映射结果。资源、后布线及配置校验记录保存于`docs/verification/soc/face_photo_report_20261104/`。

2.7.3 工程配置

工程以 SoC、CPU、NPU 和 Video 独立验证顶层共用一份 RTL，板级目录集中管理 DDR、HDMI、PLL、引脚和时序约束。CPU 发射宽度、Cache 与分支预测开关、NPU 并行度、地址映射及视频模式在综合前确定。人脸检测顶层采用单发射精简控制 CPU，应用代码和数据位于片上 SRAM；NPU 空间并行度为 2，量化通路数为 1，图控制器执行完整模型，CPU 完成九头输出的后处理。视频配置采集原图与模型输入，显示端按模式输出原图、全部有效检测框或主人脸位置的透明头像。CPU 独立验证顶层另通过 `DUAL_ISSUE` 和 `CACHE_ENABLE` 配置发射宽度与缓存。构建流程依次生成存储初始化文件、综合网表、布局布线结果和位流。

发布包记录源码提交、工具与 IP 版本、硬件配置、固件及模型哈希。软件装载地

址与链接布局相互核对，位流、固件和数据包按同一接口版本配套使用。

2.7.4 时钟复位

CPU、NPU 和 SoC 控制逻辑使用独立 PLL 产生的 50 MHz 计算时钟；Video 预处理、DMA 及本地存储控制使用 125 MHz 核心时钟。计算 PLL 以 125 MHz 参考时钟为输入，生成时钟约束采用 2/5 频率比，与实际 PLL 输出一致。HDMI 输入使用接收侧像素时钟，输出使用 40 MHz 像素时钟；DDR 用户时钟按厂商 IP 配置。相关异步输入分别同步，像素数据跨域使用异步 FIFO，多位地址、帧标识和显示记录使用保持数据束及握手[9]。

复位在各本地域同步释放。CPU 可先从 ROM/SRAM 启动，DDR 初始化成功后才开放外存 DMA，视频输出先产生本地背景，再接受完整有效帧。失锁和整机重初始化同时处理旧事务、FIFO 与帧所有权。

2.7.5 板级接口

HDMI 一入一出均采用 800×600 固定模式，接收和发送芯片通过板级控制接口完成初始化。DDR3 接口连接板载 32 bit 存储通路，USB-UART 承担模型、头像装载及确认响应，接口使用 115200 波特率、8 位数据、无校验和 1 位停止位。KEY0（C22）提供协调复位，KEY1（C23）经同步和 20 ms 消抖后请求循环切换显示模式，LED 指示运行状态，JTAG 用于 FPGA 配置和逻辑调试。

板级封装根据原理图定义引脚方向、电平和 I/O 标准，输入输出像素与同步信号保持对应时序。应用固件随片上存储初始化内容配置，模型程序、权重参数、解码表、输出头初值和头像通过串口写入约定地址，封存后由 FPGA 执行处理和显示。

2.7.6 实现约束

内部时序约束覆盖计算、DDR 用户、输入像素和输出像素时钟，并定义 PLL 生成关系及复位同步路径。跨域例外对应具体 FIFO 或握手结构，数据总线与控制信号按其同步方式设置约束。布局布线后分别分析各时钟域的建立保持裕量、关键路径和跨域结构。

后布线检查包含建立、保持、恢复、移除及最小脉宽，并对异步 FIFO 的 Gray 指针数据路径和源、目的时钟偏差进行核对。本配置的内部时序及 CDC 检查通过，双角结果在第三部分汇总；实际工作频率按相应时钟约束列出。最差建立路径位于 125 MHz DDR 用户时钟域内的模型输入采集 DMA 控制逻辑，其寄存器起止点、逻辑级数及延迟构成见表3-16。内部 STA 核对已约束的寄存器路径，CDC 核对异步 FIFO 与握手结构的数据及控制时序；DDR 初始化、模型装载和 HDMI 画面观察属于板级功能验收。外部 DDR PHY 与 HDMI I/O 的板级 AC 约束和裕量需单独验收，内部正裕量不能替代这项检查。

第三部分 完成情况及性能参数

3.1 整体介绍

TriHaRiscv 集成了支持单/双发射配置的六级 CPU、NPU、AXI 互连、片上与外部存储，以及 HDMI 采集和全屏显示链路。裸机程序提交 YuNet 整帧推理任务，NPU 完成计算图执行，CPU 完成定点后处理和主人脸选择，单路 HDMI 按键切换原图、同帧检测框和透明头像叠加。应用配置的 CPU 与 NPU 运行于 50 MHz，Video 与 DDR 用户接口运行于 125 MHz，输出像素时钟为 40 MHz。工程成果按硬件、软件、功能验证和应用演示组织，性能参数按 CPU、NPU、系统及 FPGA 实现分别统计。

3.2 工程成果

3.2.1 硬件成果

CPU 模块提供可配置单/双发射的六级流水线、两路组相联 I/D Cache、动态分支预测和机器异常中断；人脸检测应用采用单发射控制配置，以片上程序和数据存储承担帧任务调度。NPU 完成普通卷积、深度卷积、辅助算子和 DMA，并由硬件执行模型计算图；Video 完成原图采集、双线性缩放及补边、BGR NCHW 输入组织、原图行缓冲、检测框绘制及透明头像合成。SoC 通过 AXI 连接处理单元、视频 DMA 和 DDR，板级封装统一管理时钟、复位及接口。

3.2.2 软件成果

软件由启动程序、采集与显示驱动、NPU 整帧任务驱动、检测后处理和帧池管理组成。模型工具组织计算图程序、INT8 权重、量化参数、CPU 解码表及张量布局；硬件 UART 装载器接收六段模型数据和一段 RGBA 头像，完成写入确认与 CRC 校验后封存。CPU 轮询采集、计算和显示事件，在 NPU 完成后读取九个输出头，执行阈值筛选、框解码和非极大值抑制，再提交同帧显示记录。异常信息保存在片上 SRAM，应用停止提交后续任务并保留现场。

3.2.3 功能验证

CPU 与 NPU 组件验证、底层互连接口验证、Video 独立板测分别使用各自配置；CPU 独立 CoreMark 采用 100 MHz，Video 三模式演示使用独立顶层。头像应用的融合仿真采用 50/125/40 MHz 时钟和实际应用固件，DDR 端点为存储代理，验收摘要见表3-1。两组场景均核对九头输出、完整画面、同帧身份及旧槽释放复用，详细事件计数见附录F。数值参考与硬件采用相同的输入、量化和后处理规则。

表 3-1 头像应用融合仿真验收摘要

场景	通过内容	依据
自然人脸	两帧均输出 1 脸，RAW、BOX、AVATAR 逐像素核对，旧槽经确认后复用	附录 F 自然人脸回归
零 RGB 空场景	两帧均输出 0 脸，三模式均保持原图，旧槽经确认后复用	附录 F 零 RGB 回归

3.2.4 应用演示

人脸检测与头像叠加应用

应用接入 800×600、标称 60 Hz 的 HDMI 视频，经过原图采集、模型输入预处理、NPU 整图推理和 CPU 后处理，按选择的模式输出原图、检测框或透明头像。CPU 从保留的检测框中按面积、分数和候选序号选择主人脸，头像覆盖区域随主框位置和尺寸更新；无脸时显示原图。CPU/NPU 运行于 50 MHz，Video/DDR 用户接口运行于 125 MHz，输出像素时钟为 40 MHz。

作品实物及视频演示环境如图3-1所示。中央笔记本提供 HDMI 视频输入，右下方的开发板完成视频处理，左侧显示器展示叠加人脸检测框后的画面。



图 3-1 TriHaRiscv 作品全景

开发板的 JTAG Program/Verify 及七段模型和头像装载均通过。传输共 2,032,640 B、7,940 包，重传 0 次，最终确认游标为 (7, 0)，随后完成封存。现场通过 HDMI 输出确认了三模式切换、头像随人脸位置和尺寸更新、透明区域显示背景、无脸时清除及人脸返回后恢复，结果见表3-2。

表 3-2 人脸检测与头像叠加板测结果

检查项目	结果	验证方式与范围
配置下载与校验	Program / Verify 通过	PG2L100H 易失 JTAG 配置
模型及头像装载	七段传输与封存通过	7,940 包，0 重传，最终游标 (7, 0)
模式切换	RAW、BOX、AVATAR 正常	KEY1 逐次切换并观察 HDMI 输出
头像跟随	随人脸位置和尺寸更新	改变人脸位置及大小
透明合成	透明像素处可见背景	核对输入照片的透明区域
无脸与恢复	头像消失，人脸返回后恢复	现场移除和重新引入人脸

上述板测记录用于功能验收。配置、模型和头像采用易失装载，断电后重新执行 JTAG 配置和七段数据上传。115200 baud、8N1 下，仅有效载荷的理论传输时间约为 176.4 s；含包头和确认包的线速预算约为 195.7 s，实际装载历时没有记录。本次验收也未记录明确窗口内的完成结果数和新帧首次显示数，实际应用板级帧率尚未测得。

Video 独立三模式验证

Video 独立板级验证使用 `video_live_mode_board` 目标，接入动态 HDMI 源，核对三种模式的连续背景、按键切换及协调复位。BOX 使用人工生成的框和关键点，AVATAR 使用 256×256 RGBA 算术测试纹理。这些输入用于检查视频数据通路、模式控制与逐像素合成，板测数据见表3-3。

表 3-3 Video 独立板级验证结果

检查项目	结果	统计范围
图像与时钟	800×600；40/125 MHz	输出像素时钟/核心时钟
显示扫描频率	约 60.32 Hz	1056×628 光栅
新图更新率	30.1548 fps	复位后 84 s 生命周期采样窗口
连续健康观察	87.000 s	复位后的持续观察时长
模式与复位	三次模式切换、一次复位通过	KEY1 循环三模式，KEY0 恢复 RAW
欠载、溢出及错误	均为 0	已记录的连续三模式及复位观察
LUT / FF	16770 / 18150	整个独立 Video 板级目标
片上存储	73.5 个 DRM36K 等效块	含 64 个测试纹理缓存块
Slow setup / hold	+0.063 / +0.129 ns	实际后布线时序报告
Fast setup / hold	+2.911 / +0.097 ns	实际后布线时序报告
配置下载与校验	Program / Verify 通过	与时序报告对应的板级镜像

板测记录与机器证据保存于`docs/verification/video/live_modes.md` 及其关联证据文件。新图更新率按采集帧标识变化计数；扫描频率按输出光栅计数。表中 30.1548 fps 对应独立 Video 目标的采集与显示生命周期，不计入真实人脸检测应用的

推理性能。

3.3 特性成果

3.3.1 CPU 性能

CPU 性能采用 CoreMark 1.0[10]，按发射宽度和存储路径分别记录板测结果。双发射与单发射均列出本地 SRAM 和 L1 Cache ＋板载 DDR 两类配置。

双发射板测采用六级 RV32IM/Zicsr 核，分别连接 32 KiB 片上 SRAM 和 L1 Cache ＋板载 DDR。SRAM 配置将程序、数据及栈放在本地存储，关闭 Cache；DDR 配置将程序、数据及栈放在私有 DDR 区，经各 16 KiB 的 I/D Cache 访问。CPU 标称频率为 100 MHz，编译器为 GCC 15.2.0，使用`-march=rv32im_zicsr`、`-mabi=ilp32` 和`-O2`，不启用 LTO；CoreMark 执行 5000 次迭代。两组结果见表3-4，两者均通过标准 CRC 检查，`errors=0`、`short=0` 且输出 `CPU_COREMARK_BOARD_PASS`。

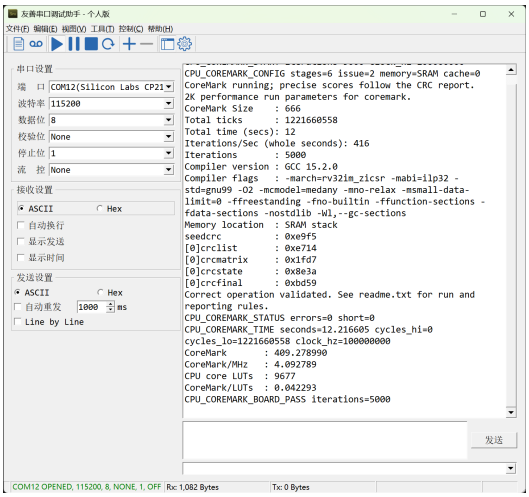
表 3-4 双发射 CPU 板级 CoreMark 结果

项目	TCM	L1 Cache ＋板载 DDR
发射宽度	2	2
CPU 标称频率	100 MHz	100 MHz
迭代次数	5000	5000
计时区间周期数	1,221,660,558	1,586,210,467
运行时长	12.21660558 s	15.86210467 s
CoreMark	409.278990	315.216681
CoreMark/MHz	4.092789	3.152166
CPU 核占用 LUT	9677	9707
CoreMark/LUT	0.042293	0.032473

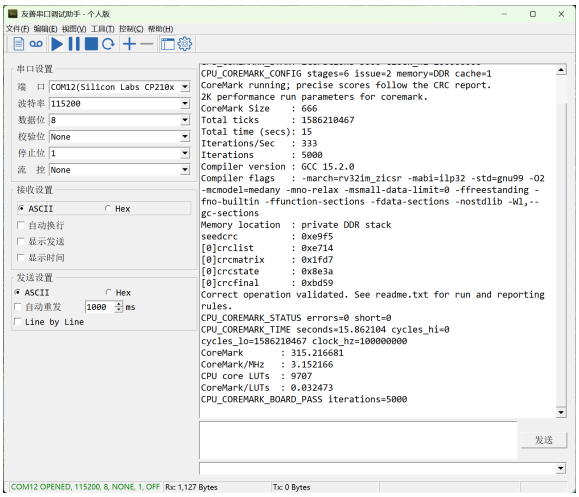
运行时长由完整周期数除以 100 MHz 计算，CoreMark 为 5000 次迭代除以运行时长，CoreMark/MHz 再除以 100，CoreMark/LUT 除以对应 CPU 核 LUT 数。表中分数保留串口输出的六位小数，其输出方式为截断；运行时长按完整周期数列出，不使用整数秒对应的 Iterations/Sec 字段。SRAM 的本地访问延迟较低，Cache ＋ DDR 配置还包含缓存查询、未命中填充及外部存储等待，两者分别描述对应存储路径下的板级性能。

图3-2并列展示两组完整串口原始截图，保留配置、编译参数、计时、评分和校验信息。原图及配置、资源、时序证据索引见`docs/verification/cpu/report_configurable_issue_2026-11-02/`。

单发射板测分别采用 TCM 本地存储和 L1 Cache 配合板载 DDR。TCM 由直接连接 CPU 本地访问路径的 32 KiB 片上 SRAM 实现，程序、工作数据和栈均驻留其中，不



(a) SRAM，Cache 关闭



(b) L1 Cache ＋板载 DDR

图 3-2 双发射 CPU 两种存储配置的 CoreMark 完整串口截图

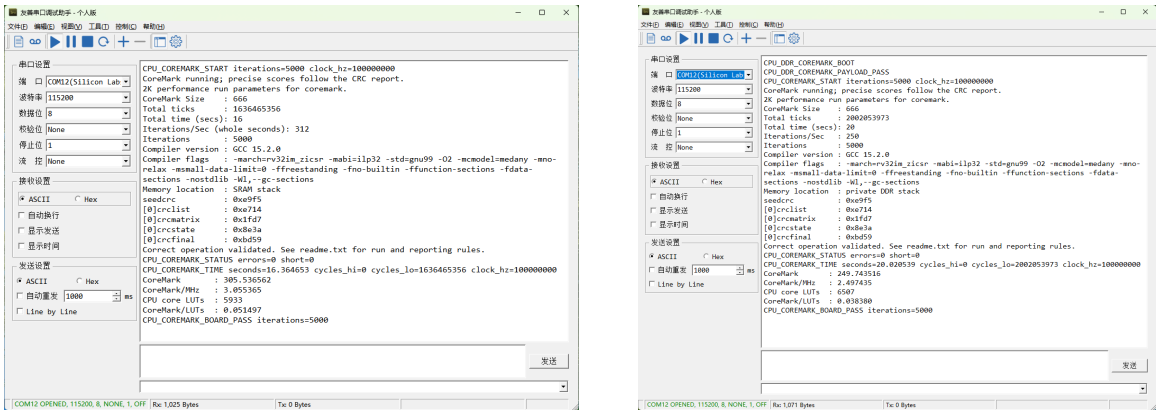
经过 I/D Cache 及 DDR 控制器。L1 Cache 配置将程序、数据和栈放在私有 DDR 区，通过各 16 KiB 的 I/D Cache 访问；两者均为两路组相联、32 B 缓存行，D-Cache 采用写回、写分配及成功加载返回重叠配置，未命中时访问板载 DDR。两种配置均在 PG2L100H 上运行六级单发射 RV32IM/CSR 核，BTB/BHT 各 512 项、RAS 为 4 项，GShare 关闭，CPU 标称频率为 100 MHz。软件均使用 GCC 15.2、-O2 优化、ILP32 ABI 和 5000 次迭代，不启用 LTO，板测结果见表3-5。

表 3-5 单发射 CPU 板级 CoreMark 结果

项目	TCM	L1 Cache ＋板载 DDR
CPU 标称频率	100 MHz	100 MHz
迭代次数	5000	5000
计时区间周期数	1,636,465,356	2,002,053,973
运行时长	16.36465356 s	20.02053973 s
CoreMark	305.536562792	249.743516780
CoreMark/MHz	3.055365628	2.497435168
CPU 核占用 LUT	6139	6507
CoreMark/LUT	0.049769761	0.038380746

两组完整运行均通过标准 CRC 校验：`seedcrc=e9f5`，链表、矩阵和状态机 CRC 依次为 `e714`、`1fd7` 和 `8e3a`，最终 CRC 为 `bd59`，错误计数和短测标志均为零。运行时长按 CPU 周期数除以标称 100 MHz 换算，均超过 10 s；CoreMark 按迭代次数除以运行时长计算。时钟采用板级配置值。后文以单发射 TCM 调试镜像说明片上调试方法，采样结果对应此处列出的单发射配置。L1 Cache 项在 I/D Cache 开启、D-Cache 采用写回策略的配置下完成板测，反映 CPU 经 L1 Cache 访问板载 DDR 时的整体执行性能。两种配置的存储延迟、访问路径和固件布局不同，以上结果描述各自工作点，不单独用于推导 Cache 开关的性能增益。

图3-3并列展示 TCM 与 L1 Cache ＋板载 DDR 的完整串口原始截图，包含编译参数、CRC、计时评分和完成标志。TCM 截图中的 5933 个 CPU 核 LUT 及 0.051497 CoreMark/LUT 为固件保留的原发布版打印值；表3-5按调试构建的 6139 个 CPU 核 LUT 计算资源效率。截图中的整数秒及上游 Iterations/Sec 字段不用于精确评分，表中分数按 `CPU_CORE_MARK_TIME` 的完整周期数复算。



(a) 片上 SRAM/TCM　　(b) L1 Cache ＋板载 DDR

图 3-3 单发射 CPU 两种存储配置的 CoreMark 完整串口截图

双发射 SRAM 与 Cache ＋ DDR 的 CoreMark/MHz 分别比表3-5中的单发射结果高

约 33.95% 和 26.22%。两组板级版本还包含前端、配对和数据通路优化，以上为各自完整配置的结果比较，不将差值全部归因于发射宽度开关。

表3-6按综合报告中的 CPU 核层级统计资源，片上 SRAM、调试核、DDR 及板级逻辑均不计入 CPU 核。单发射 CPU 核的 688 个 RAM-LUT 和双发射 CPU 核的 752 个 RAM-LUT 均已包含在各自 LUT 总数中。Cache 及传输控制另列，DRM 以 36 Kbit 块统计。

表 3-6 CPU 核及缓存资源统计

统计范围	LUT	FF	DRM	APM
单发射 TCM 原发布版 CPU 核	5933	3528	0	4
单发射 TCM 调试版 CPU 核	6139	3528	0	4
单发射 Cache ＋ DDR 版 CPU 核	6507	3562	0	4
单发射配置 Cache 及传输控制	668	807	11	0
双发射 SRAM 版 CPU 核	9677	4968	0	4
双发射 Cache ＋ DDR 版 CPU 核	9707	4995	0	4
双发射配置 Cache 及传输控制	675	807	11	0

TCM 调试版在新综合上下文中得到 6139 个 CPU 核 LUT，CPU 源码和固件与原发布版逐字节相同，板测周期也相同。原固件在计时结束后仍打印历史 5933-LUT 元数据；表3-5使用本次调试构建的 6139 个 LUT 重新计算效率。原无调试核发布包的 5933 个 CPU 核 LUT 和 0.051497819 CoreMark/LUT 保留为该发布包的结果。

单发射 TCM 调试版和 L1 Cache ＋ DDR 版分别满足 100 MHz 实际后布线时序，结果见表3-7。TCM 调试版沿用原 100 MHz、I/O 延迟及 hold 约束；四路 LED 均映射到器件输出寄存器，状态显示延后一拍，退出码组合译码不再直接驱动引脚。调试核各时钟与跨时钟检查、复位恢复和移除检查均通过，performance slack 与最差 setup 一致。

表 3-7 单发射 CPU 板级后布线时序裕量

检查项	TCM 调试版	L1 Cache ＋ DDR 版
Setup，Slow/Fast	+0.366/+3.662 ns	+0.309/+3.551 ns
Hold，Slow/Fast	+0.061/+0.034 ns	+0.151/+0.099 ns
最小脉宽，Slow/Fast	+3.655/+4.202 ns	+3.655/+4.202 ns

双发射两个板级配置也完成 100 MHz 后布线时序检查，CPU 时钟域结果见表3-8。资源与时序分别取自对应板级构建，Cache ＋ DDR 版的 9707 个 CPU 核 LUT 与串口分母一致，Cache 及传输控制的 675 个 LUT 单独计数。双发射提高了单位频率性能，同时增加寄存器读写端口、前递及配对控制的资源开销；按现有板级数据，其 CoreMark/LUT 低于对应单发射配置。

表 3-8 双发射 CPU 板级后布线时序裕量

检查项	SRAM 版	Cache ＋ DDR 版
Setup，Slow/Fast	+0.220/+3.768 ns	+0.223/+3.447 ns
Hold，Slow/Fast	+0.162/+0.104 ns	+0.158/+0.102 ns
最小脉宽，Slow/Fast	+3.655/+4.202 ns	+3.655/+4.202 ns
Recovery，Slow/Fast	+2.036/+4.167 ns	+4.062/+6.174 ns
Removal，Slow/Fast	+0.653/+0.441 ns	+0.188/+0.103 ns

100 MHz 为这些配置完成时序检查并用于板测的工作点；Fmax 需通过对应配置的频率约束扫描确定。板测 CoreMark 记录提供运行周期，整段退休 IPC 另需在同一计时窗口累计两路有效完成指令数。

表3-9按相同存储路径比较两组完整配置，片上路径的单发射分母采用 TCM 调试版。双发射在片上 SRAM 路径增加 3538 个 CPU 核 LUT，获得 33.95% 的 CoreMark/MHz 提升；在 Cache ＋ DDR 路径增加 3200 个 LUT，获得 26.22% 的提升。资源增长快于性能增长，两条路径的 CoreMark/LUT 分别下降约 15.02% 和 15.39%。

表 3-9 单发射至双发射完整配置的性能与资源变化

比较项	片上 SRAM/TCM	L1 Cache ＋ DDR
CPU 核 LUT	6139 → 9677	6507 → 9707
CPU 核 LUT 增幅	57.63%	49.18%
CoreMark/MHz 增幅	33.95%	26.22%
CoreMark/LUT 变化	−15.02%	−15.39%
Slow 角 Setup 裕量	0.366 → 0.220 ns	0.309 → 0.223 ns

从资源分配看，双发射适合需要提高 CPU 处理吞吐的配置；单发射在单位 LUT 性能上更有利，可为 NPU 和视频缓冲保留更多器件资源。双发射需要为两条指令提供操作数和独立写回，并扩展操作数前递及配对控制；两种配置均使用 4 个 APM，增加的资源主要体现在 LUT 和 FF。两组双发射实现均在 100 MHz 下保持正的建立、保持裕量，但 Slow 角建立裕量较对应单发射版本减小，继续提高频率需要重新评估关键路径。这些比较对应各自的前端、数据通路及物理实现，不能等同于同一版本仅切换 `DUAL_ISSUE` 所得的消融实验。

Cache ＋ DDR 路径的增益较片上路径低，与访存等待限制可并行执行的机会相符。共享 LSU/MUL、指令相关及前端供给共同影响双发射利用率。本节 IPC 与周期分类来自单发射数字仿真，测试配置与统计窗口见下文。

3.3.2 CPU 流水线效率

片上调试以单发射 TCM 版本为例，采用 PDS DebugCore/Fabric Debugger 关联 CoreMark 函数入口与流水线状态。调试镜像保留已验收的 CPU 及 5000 次-O2 固件，仅在板级封装中寄存四路 LED 输出并加入 38 位 DebugCore 探针；实际后布线时序及 Program/Verify 通过后再进行采样。该镜像的完整板测周期与无探针发布版一致。采样时钟为 CPU 时钟上升沿。三个触发端口分别为 30 位 WB PC[31:2]、1 位 WB 有效和 7 位流水控制状态，合计 38 位，采样深度为 1024 拍。按 32 bit 指令对齐规则在 PC 低位补零，结合 5000 次板测 ELF 定位链表归并排序、矩阵乘法和状态转换的真实函数入口。触发条件同时要求入口 PC 匹配、`valid_wb_q=1` 和 `pipe0_stall_raw_w=0`。

表3-10统计完整采样窗口。WB 事件按有效位与非暂停状态的交集计数，避免指令保持时重复累加。窗口可能进入被调函数或调用者；这些局部 WB 事件不构成整段 CoreMark 的架构退休 IPC，内存 WB 有效也不等同于 DDR 事务或 Cache 事件。

表 3-10 单发射 TCM CPU 三个热点的 1024 拍板上采样

触发算法	触发 PC	WB 事件	内存 WB 有效拍
链表归并排序	`0x00011D48`	951	119
矩阵乘法	`0x000124E8`	964	247
状态转换	`0x00012C1C`	916	182

图3-4、图3-5和图3-6为 Fabric Debugger 最大化窗口的原始采样波形截图，放大展示各入口附近的局部采样。插图仅裁剪窗口边缘及空白区，保留原始信号名、读值和采样刻度。金色游标 0 为指定函数入口，对应原始 CSV 索引 1；其前一拍为工具刻度 −1，对应 CSV 索引 0。三窗在触发拍均为 WB 有效且非暂停。`wb_pc_word` 为 PC[31:2]，游标读值 `0x4752`、`0x493A` 和 `0x4B07` 分别对应三个字节地址；PC 总线仅在 WB 事件有效时解释为完成指令地址。图中 `valid_e2_q` 表示 E2 在途状态，`squash_e1_e2_w` 表示取消条件，`div_pending_q` 和 `csr_pending_q` 表示对应串行操作未结束，`fetch0_valid_w` 与 `writeback_mem_valid_w` 分别表示前端及内存返回有效。三窗 `pipe0_stall_raw_w` 均为零，但 `valid_wb_q` 仍有空隙；发射空泡、相关性和控制流开销需结合更多前端信号分析，不能只凭该暂停位判断。

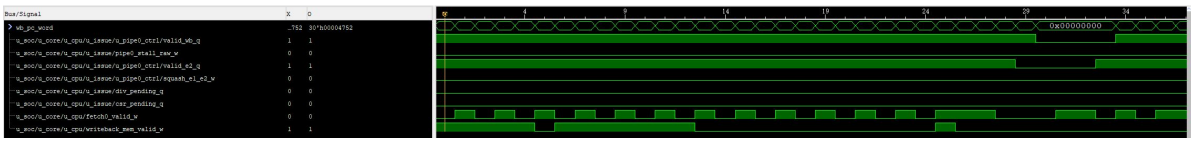


图 3-4 单发射 TCM CPU 链表热点的原始采样截图，触发 PC 为 0x00011D48

采样归档包含原生 DPRJ/WDF、逐拍 CSV、触发条件及位流和 ELF 哈希。state 窗口的 38 位通道和 PC 总线在全部 1024 拍上与原始 VCD 逐位一致。原始 VCD 导出的 100 ns 标注属于工具的样本刻度，本节横轴统一使用采样序号；硬件配置的 100 MHz

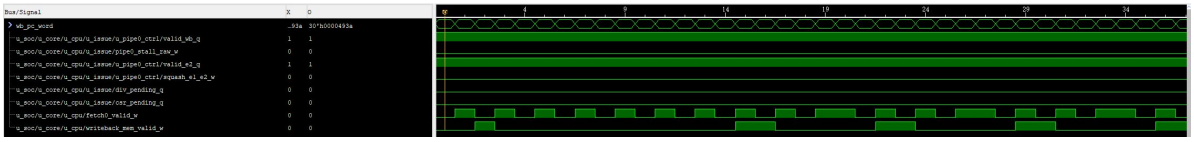


图 3-5 单发射 TCM CPU 矩阵热点的原始采样截图，触发 PC 为 0x000124E8

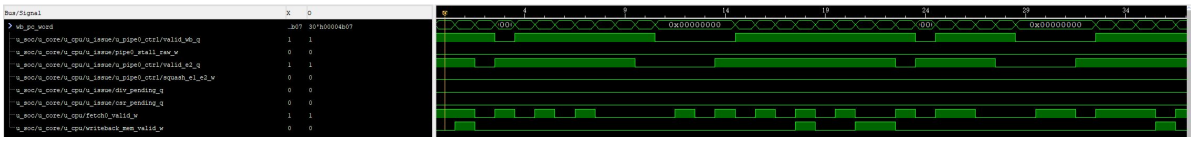


图 3-6 单发射 TCM CPU 状态转换热点的原始采样截图，触发 PC 为 0x00012C1C

采样时钟名义周期为 10 ns。上述片上采样属于单发射 TCM 配置，与对应的单发射完整板测配套；双发射及 L1 Cache ＋ DDR 配置分别记录。

单发射 Cache/DDR 版的整段流水线统计来自 ModelSim 数字 DDR 及 PLL 模型，采用与该版 5000 次板测 CPU/Cache RTL 逐字节一致的配置。软件仍为 GCC 15.2、-O2、无 LTO，仿真执行 5 次迭代，UART 除数仅在测试台中缩短，计时仍来自 CPU 周期 CSR。这项短测保留不足 10 s 提示，用于分析执行过程。2026 年 10 月 28 日的观测重放得到 2,018,576 个计时周期、1,448,453 条 WB 观察序列及 70,840 次 store，标准 CRC 及完整序列哈希均与原记录一致。观测模块不驱动 CPU、Cache 或软件状态。

统计窗口对应软件两次周期 CSR 读取之间的区间。WB 计数包括分支和存储指令，暂停时不重复累加，窗口内异常检查为零。对应的数字 WB 观测 IPC 和 CPI 为

$$IPC_{\mathrm{WB}} = \frac{1,448,453}{2,018,576} = 0.717562, \qquad CPI_{\mathrm{WB}} = 1.393608 \tag{3-1}$$

该数值为单发射 Cache/DDR 数字模型的 WB 观测 IPC，其计数范围与板上采样、双发射板测分别记录。

每周期按发射成功、取消、LSU 等待、后端流水等待、除法、CSR、前端无有效指令、执行资源、相关性及其他原因的优先顺序归类。发射成功优先，未发射时取首个阻塞原因；各项互斥且总和等于计时周期数，结果见表3-11。

表 3-11 单发射 Cache/DDR 数字 CoreMark 的互斥周期分类

周期类别	周期数	占比（%）
发射成功	1,448,453	71.756179
LSU 等待	358,437	17.756924
后端流水等待	12,993	0.643672
前端无有效指令	98,793	4.894193
相关性等待	99,891	4.948588
CSR 等待	9	0.000446
取消、除法、执行资源及其他类别合计	0	0.000000
总计	2,018,576	100.000000

该数字窗口中，D-Cache 接受 273,477 次读及 70,840 次写，读命中 273,461 次、读未命中 16 次，读命中率为 99.994149%。写命中 70,824 次、写分配未命中 16 次，共 32 次行填充；计时窗口内没有脏行逐出或显式清理，下级写次数为零。高命中率仍伴随 17.756924% 的 LSU 等待，说明请求接纳、同步 RAM 读取和返回节拍也限制吞吐；成功加载返回与下一请求的重叠发生 75,408 次。Cache 与总线各状态的周期统计可能包含同一周期，因此不能与上述互斥类别直接相加。

3.3.3 NPU 性能

人脸检测应用的 NPU 采用两空间行 Dense 阵列和单量化通路，计算时钟为 50 MHz。CPU 提交整帧任务后，图控制器读取 NPM3 程序，依次执行普通卷积、深度卷积及辅助算子，完成九个检测头的写回。CPU 随后读取这些输出，执行定点解码、NMS 和主人脸选择。

框检测部署图的普通及逐点卷积每帧执行 68,837,800 次 MAC，深度卷积执行 15,685,200 次 MAC，总计 84,523,000 次。Dense 阵列每个有效拍完成 $16M$ 次 MAC，DW16 每个有效拍完成 16 次 MAC，两条通路按算子分时运行。在计算频率 f 下，持续有效供数时的纯卷积算术时间下界为

$$T_{\mathrm{conv,min}}(M,f)=\frac{68\,837\,800}{16Mf}+\frac{15\,685\,200}{16f}$$

代入 $M=2$、$f=50\times10^{6}$ Hz，普通及逐点卷积的理想时间为 43.023625 ms，深度卷积为 19.606500 ms，合计 62.630125 ms。该下界按持续有效供数和满通路计算，仅计乘加，不包含尾通道及空间块利用率损失、量化、数据搬运、辅助算子、图控制和 CPU 后处理；组件的数值及时序验证见附录C.1。对应的理想纯卷积吞吐上限约为 15.97 帧/s。24 帧/s 要求每帧不超过 41.667 ms，已小于普通及逐点卷积单项的理想时间，因此 24 帧/s 属于后续优化目标，当前 50 MHz、$M=2$ 配置即使忽略其他开销也无法达到。

完整应用仿真采用与上板版本一致的计算、Video 和像素时钟，运行实际 CPU 固

件、NPU、Video 及跨域逻辑，外存端点采用存储代理模型。输入经过视频采集与预处理后进入 NPU，计算结果由 CPU 后处理，并形成同帧显示记录。统计以 CPU 后处理结果完成事件为边界，数值列于表3-12。

3.3.4 系统性能

系统分别记录结果完成、显示记录生效和实际像素输出。表3-12给出 50 MHz 应用仿真的两组结果间隔。自然人脸与零 RGB 空场景各运行至两个结果完成，每组取相邻完成事件之间的一个间隔样本。

表 3-12 整帧应用仿真的结果完成间隔

场景	间隔（计算周期）	间隔（ms）	结果速率（fps）
自然人脸	18079060	361.58120	2.76563
零 RGB 空场景	18060318	361.20636	2.76850

自然人脸的结果完成标记为第 22,023,646 和 40,102,706 个计算周期，零 RGB 空场景为第 21,998,338 和 40,058,656 个计算周期。间隔与结果完成率按下式计算，其中 C_1、C_2 为两个事件的计算周期计数：

$$\Delta t = \frac{C_2 - C_1}{50 \times 10^6}, \qquad r_{\text{result}} = \frac{50 \times 10^6}{C_2 - C_1}$$

日志标记 `FACE_SYSTEM_NPU_CPU_RESULT` 由软件 `results` 计数增加触发，位于 CPU 后处理、显示记录构建及 NPU 完成确认之后。相邻事件间隔覆盖逐帧预检、图执行、数据访问、CPU 后处理及调度等待；一次性模型及头像装载和启动位于首个事件之前，不计入该间隔。两组结果反映给定存储代理环境中的结果生成率，每场景只有一个间隔样本，不能作为板级持续帧率。

两个间隔分别为纯卷积算术下界的 5.7733 倍和 5.7673 倍，相差 298.9511 ms 和 298.5762 ms。差额混合了实际计算利用率损失及公式未计入的系统工作，不能全部归为访存或调度开销，也不能据此反推 PE 实测利用率。现有日志每十个算子记录一次完成检查点。第二帧的 `op0` 至 `op10` 完成间隔分别为 163.47200 ms 和 163.48584 ms，表明该区间是后续剖析的重点；其中包含 `op1` 至 `op10` 的混合算子、访存及层间等待。`op70` 至结果完成之间还包括末尾八个算子、图退出和 CPU 处理，不能直接作为 CPU 后处理耗时。表3-13列出分项分析可用的证据与缺失观察量。

表 3-13 整机耗时分析的证据与缺失观察量

分项	已有证据	分解实测耗时所需记录
Dense	算子形状及 32 MAC/周期理想峰值	逐算子起止、有效计算与停顿周期

表 3-13 整机耗时分析的证据与缺失观察量（续）

分项	已有证据	分解实测耗时所需记录
DW	16 路通路及量化握手	逐算子起止、有效输入输出与阻塞周期
量化	单量化通路配置	请求、完成和反压周期，与计算的重叠范围
辅助算子	32 个非卷积算子及稀疏完成点	各算子类型的完整执行区间
DMA 及回写	累计读写事务数	按帧及请求方记录字节数、首末响应与等待
图控制	逐帧预检和阶段 / 算子状态	预检、执行、层间等待的起止及重叠关系
CPU 后处理	结果完成事件位于后处理之后	输出头收取、解码 / NMS、记录构建及 ACK 的独立起止

上述通路可以重叠工作，各自忙周期不能直接相加为总时间。现有正式日志没有完整的分项边界，因此不列各模块耗时比例。

有效输出帧率以统计窗口内首次实际显示的不同采集帧数计数。同一帧的重复扫描不增加新帧数；采集、结果完成与显示扫描使用各自事件标记，不能互相替代。板内可见延迟的起点为选中输入帧的有效画面开始，终点为全屏输出首次显示该帧结果的有效像素。表3-14列出各阶段的事件边界。其中 `RESULT_READY` 表示上述应用软件结果完成边界，不指代单独的 NPU 完成脉冲。

表 3-14 系统处理与显示延迟的统计边界

阶段	起止事件	计入内容
输入形成	输入首个有效像素至 `INPUT_READY`	采集、预处理、DDR 写入及响应排空
推理与后处理	`INPUT_READY` 至 `RESULT_READY`	调度等待、NPU 图执行、计算与搬运、CPU 后处理
显示等待	`RESULT_READY` 至匹配显示记录的 SOF	提交握手与帧边界等待
像素生效	记录生效 SOF 至首个对应有效像素实际输出	原图读取、框或头像合成及输出流水
有效输出吞吐	相邻新帧首次实际显示事件	不同采集帧的更新间隔

40 MHz 像素时钟和 1056 × 628 扫描总时序对应约 60.31654 Hz，扫描周期为 16.5792 ms。当显示记录和原图数据就绪、下一帧边界可以接纳时，等待 SOF 的时间处于一个扫描周期内；SOF 后的像素输出还经过读取和合成流水。该扫描参数用于解释显示时序，相邻检测结果的完成间隔按上表的独立事件统计。板测记录确认了三模式切换、头像跟随、透明合成和无脸清除与恢复，但没有保存同一采样窗口内的结果完成数、新采集帧首次显示数及扫描数，实际板级结果完成率和新图更新率尚未测得。后

续测量需在同一会话内记录起止时间及对应计数，同时核对帧身份、仅模式切换事件、拒绝、欠载和错误；软件无空闲槽的尝试次数不等于输入丢帧数。本节仿真、板级功能验收及原始日志统一归档于`docs/verification/soc/face_photo_report_20261104/`。

3.3.5 FPGA 实现指标

人脸检测整机在 PG2L100H-6IFBG676 上完成综合和布局布线，综合总数为 45,716 个 LUT、41,848 个 FF、113.5 个 DRM36K 等效块和 64 个 APM。按器件容量折算分别为 68.64%、31.42%、73.23% 和 26.67%；同一构建的模块层级资源见表2-21。CPU 与 NPU 工作于 50 MHz，Video 与 DDR 用户接口工作于 125 MHz，输出像素时钟为 40 MHz。

表3-15列出最终布线数据库的内部时序检查结果，所有列示检查的失败端点数均为 0。

表 3-15 人脸检测整机内部后布线时序裕量

检查项	Slow（ns）	Fast（ns）
建立时间	+0.100	+2.891
保持时间	+0.187	+0.131
恢复时间	+2.066	+4.089
移除时间	+0.314	+0.206
最小脉宽	+0.035	+0.035

Slow 角的计算时钟域建立裕量为 +2.287 ns，DDR 用户时钟域为 +0.100 ns，输出像素时钟域为 +9.406 ns。CDC 检查覆盖 23 组、116 个 Gray 指针位及 928 次源、目的时钟观测，最大保守到达偏差界为 3.479 ns，各组均满足各自约束。这些结果对应冻结布线数据库中的内部路径与跨域约束，工作频率按该构建的实际时钟列示。

表3-16列出一条最差建立路径，位于模型输入采集 DMA 的基址及描述符校验逻辑中，经过 10 级组合逻辑。报告起点为 `dma_base_reg[14]` 的 CLK 端，对应首个数据弧为 `dma_base_reg[12]` 的 Q 端，终点为 `row_base[22]` 的时钟使能端；这些名称按综合后报告保留。该路径属于 125 MHz DDR 用户时钟域，至行基址寄存器第 24、26、28 位的路径具有相同的最差裕量。

表 3-16 整机 Slow 角最差建立路径

项目	报告值
发起 / 接收时钟	`ddrphy_sysclk`，125 MHz，周期 8.000 ns
组合逻辑级数	10
逻辑 / 布线延迟	1.734 / 5.335 ns，逻辑项含时钟到 Q 延迟
总数据路径延迟	7.069 ns
有效时钟偏差	−0.255 ns，已计时钟悲观量移除

表 3-16 整机 Slow 角最差建立路径（续）

项目	报告值
不确定度 / 建立时间	0.350 / 0.226 ns，采用报告实际计入值
数据到达 / 要求时间	12.640 / 12.740 ns
建立裕量	$12.740 - 12.640 = +0.100$ ns

布线占该路径数据延迟的 75.47%，说明此路径的时序主要受布局与连线影响；该比例只描述此条路径，不是整机运行时间占比。外部 DDR PHY 和 HDMI I/O 的板级 AC 验收与上述内部 STA 结果分别管理。

第四部分 总 结

4.1 可扩展之处

系统以主人脸矩形定位并缩放固定透明头像，可进一步接入关键点或表情模型，扩展对齐方式和素材选择。扩展时在现有帧调度中增加裁脸、缩放和分类阶段，将结果写入公共显示记录，并配置相应的像素合成通路。多素材应用可沿用头像编号与代次管理，分别生成素材布局及校验信息；素材切换与同帧原图保持一致。新增模型、素材访问和后处理开销应计入整机帧率、延迟及片上存储预算。

CPU 可进一步研究前端供数、预测策略和流水线并行度，NPU 可扩展算子及并行规模。扩展工作沿用公共地址、寄存器和版本管理方式，分别检查指令语义、整数结果、共享存储顺序及 FPGA 时序资源。

4.2 心得体会

本次参赛，我们以自主设计 RISC-V 处理器为核心，希望不仅完成 CPU 的功能验证和性能测试，还能将其部署到国产 FPGA 上，通过实际视觉应用检验处理器与硬件加速器的协同工作能力。在研发过程中，我们逐渐认识到，单个模块的性能固然重要，但如何在有限的硬件资源下完成系统集成，并保证各模块稳定运行，同样是 FPGA 设计中不可忽视的挑战。

CPU 设计是本次作品投入精力最多的部分。我们从已有的五级流水线出发，逐步完成六级流水线、动态分支预测、Cache 和可配置双发射结构的设计。双发射的实现尤其让我们体会到处理器设计的复杂性：增加第二条执行通路只是其中一步，还必须解决指令配对、数据相关、结果前递、共享运算资源和异常恢复等问题。为了保证执行正确性，我们针对分支恢复、访存等待和错误路径副作用设计了定向测试。在一次零比较分支配对测试中，我们还发现测试平台自身的输入条件与现有设计不一致。这让我们认识到，验证不仅是检查设计能否通过测试，还需要保证测试条件与硬件实际行为相符。

NPU 与视频系统的集成使我们对软硬件协同设计有了更加具体的理解。NPU 的设计需要从 YuNet 的实际算子和数据规模出发，在计算并行度、片上缓冲容量和数据搬运开销之间作出取舍。视频链路则不仅需要完成采集和图像叠加，还必须确保检测结果与原始画面对应。为此，我们设计了三帧槽管理和显示释放确认机制，并通过流式合成实现检测框及透明头像叠加。在头像功能中，利用无损行区间缓存减少片上纹理存储占用，也使我们更加直观地认识到，针对具体应用特点优化存储组织，往往比单纯增加硬件资源更有效。

在 FPGA 上板过程中，我们进一步体会到，仿真正确并不意味着系统集成就已经

完成。时钟域交互、DDR 访问、存储器响应以及布局布线后的时序约束，都可能成为影响系统运行的关键因素。因此，我们采用模块验证、系统联合仿真与板级测试相结合的方式，逐步排查问题，并通过统一接口和配置管理减少不同模块集成时的差异。

目前，作品已经实现完整的人脸检测与头像叠加板级演示，但整网推理速度仍有较大的优化空间。这也让我们认识到，硬件设计不能只关注理论计算峰值，还应当结合实际执行周期，分析数据搬运、控制调度及计算单元的利用情况。

通过本次比赛，我们不仅加深了对 RISC-V 处理器微架构和 FPGA 硬件设计的理解，也积累了从 RTL 开发、功能验证到系统集成和实板调试的工程经验。相比单独完成某个模块，能够让自主设计的 CPU、NPU 和视频系统共同运行，并通过真实应用检验设计，是本次参赛最有价值的收获。

第五部分 参考文献

[1] RISC-V International. The RISC-V instruction set manual, volume I: unprivileged architecture: Version 20260120[S/OL]. 2026[2026-11-03]. https://docs.riscv.org/reference/isa/v20260120/unpriv/unpriv-index.html.

[2] RISC-V International. The RISC-V instruction set manual, volume II: privileged architecture: Version 20260120[S/OL]. 2026[2026-11-03]. https://docs.riscv.org/reference/isa/v20260120/priv/priv-index.html.

[3] SZE V, CHEN Y H, YANG T J, et al. Efficient processing of deep neural networks: a tutorial and survey[J/OL]. Proceedings of the IEEE, 2017, 105(12): 2295-2329. DOI: 10.1109/JPROC.2017.2761740.

[4] ARM. AMBA AXI and ACE protocol specification: ARM IHI 0022E[S/OL]. 2013[2026-11-03]. https://developer.arm.com/documentation/ihi0022/e.

[5] Video Electronics Standards Association. VESA and industry standards and guidelines for computer display monitor timing (DMT)[S]. Version 1.0, revision 12 ed. Milpitas: VESA, 2008.

[6] WU W, PENG H, YU S. YuNet: a tiny millisecond-level face detector[J/OL]. Machine Intelligence Research, 2023, 20(5): 656-665. DOI: 10.1007/s11633-023-1423-y.

[7] OpenCV. OpenCV Zoo: YuNet face detection model[EB/OL]. (2023-06-06)[2026-11-03]. https://github.com/opencv/opencv_zoo/tree/f12e12798e8314f7c074a6656816c048dcc95b7a/models/face_detection_yunet. Model version: 2023mar.

[8] KRISHNAMOORTHI R. Quantizing deep convolutional networks for efficient inference: a whitepaper: arXiv:1806.08342[R/OL]. 2018[2026-11-03]. https://arxiv.org/abs/1806.08342.

[9] CUMMINGS C E. Clock domain crossing (CDC) design & verification techniques using SystemVerilog[C/OL]//SNUG Boston. Boston: Synopsys, 2008[2026-11-03]. https://www.sunburst-design.com/papers/CummingsSNUG2008Boston_CDC.pdf.

[10] EEMBC. CoreMark: run and reporting rules[EB/OL]. [2026-11-03]. https://github.com/eembc/coremark/blob/main/README.md. CoreMark 1.0.

[11] RISC-V Software. riscv-tests: unit tests for RISC-V processors[EB/OL]. [2026-11-03]. https://github.com/riscv-software-src/riscv-tests.

[12] RISC-V International. RISC-V architectural certification tests and the ACT4 framework[EB/OL]. (2026-07-10)[2026-11-03]. https://github.com/riscv/riscv-arch-test/tree/375292312d3d02b7468272eb550d81cb74be53d8.

附 录

A 仿真环境与验证方法

仿真验证按运算单元、功能模块和系统接口逐层展开，覆盖 CPU、NPU、系统互连及图像采集与显示链路。测试平台通过接口激励、参考结果比对和时序检查，核对计算结果、请求与响应关系以及异常处理过程。本节说明共用工具与测试方法，各模块的具体用例和结果分别列于后续附录。

A.1 工具配置

仿真环境部署在 Windows 平台，使用 PowerShell 和 Python 组织源码编译、固件生成、仿真运行及结果整理。ModelSim 承担 RTL 仿真和波形调试，Icarus Verilog 用于模块的功能回归。主要工具配置见表A-1。

表 A-1 仿真工具配置

工具	版本或配置	用途
ModelSim	SE-64 2020.4	Verilog/SystemVerilog 编译、四态仿真和 WLF 波形查看
Icarus Verilog	12.0 devel	数字逻辑、接口及地址译码的功能回归
RISC-V GCC	15.2.0，ILP32 ABI	编译裸机测试固件，生成 ELF、存储镜像和反汇编文件
Pango PDS 仿真库	2022.2-SP6.4 配套库	提供存储器、乘加单元等器件原语模型
Python	3.13.13	生成测试数据和参考结果，调度仿真并汇总日志

表中版本对应 CPU 及组件验证环境。附录 F 的人脸检测整机联合仿真使用 ModelSim 10.5、RISC-V GCC 15.2.0-1 与 Python 3.13.5，配套 PDS 版本为 2022.2-SP6.4。各项验证按其原始运行记录保留工具版本和输入校验值。

各测试顶层使用明确的源码清单、包含目录和参数配置，按依赖关系编译待测模块、测试台及所需原语模型。CPU 用例记录发射宽度、Cache 和分支预测配置；NPU 用例记录算子参数、并行度及模型数据；Video 用例记录图像尺寸、时钟关系、存储布局和显示模式。测试用例中的内存容量、地址范围和响应延迟随配置保存，用于确定该次运行的适用范围。

定向裸机固件按所需指令集和 ILP32 ABI 编译，载入测试台的程序存储模型。指令集回归使用对应测试集的 ELF 及参考签名；模型计算使用固定的输入张量、权重和量化参数。运行记录保存源码与输入文件的 SHA-256、工具版本、编译命令和仿真参

数，使结果能够对应到具体配置。

仿真输出包括编译与运行日志、检查计数、结果文件及 WLF/VCD 波形。波形查看日志与批处理运行日志分别保存，附录中的仿真截图来自已保存的波形文件。周期统计按测试台时钟和指定事件窗口计算，仿真器在计算机上的运行耗时单独记录。

A.2 测试平台

测试平台由激励产生、待测模块、外部接口模型和监测比对逻辑组成。激励端产生时钟、复位、指令或数据请求；接口模型模拟存储和外设的响应；监测逻辑在有效握手及结果有效时检查数据和状态。跨时钟域测试分别产生核心时钟与像素时钟，并设置不同相位、暂停及复位场景，检查异步 FIFO 和状态握手过程。

CPU 单元测试直接驱动 ALU、乘除法器、寄存器堆和 CSR 接口，将运算结果与独立计算值或预期寄存器状态比较。模块测试连接取指前端、发射通路、LSU 及 Cache 的请求与响应端口，插入响应延迟和背压，检查保持、旁路、冲刷及错误处理。整核测试由 CPU 执行固件，监测指令 PC、退休事件、寄存器写回、访存和异常入口；指令出队、发射与退休分别计数，避免将暂停期间保持的有效信号重复计入。对写回式 Cache，测试台分别检查 CPU 可见数据和下级存储数据，并在相应写回完成后核对下级内容。

NPU 计算测试使用与模型包一致的整数参考，逐项核对卷积、辅助算子、重定标和输出张量。参考计算明确输入符号、累加精度、补边、舍入与饱和规则；存储模型检查访问范围、字节使能、数据排列及重复或遗漏写入。任务控制测试另外检查启动参数锁存、输出写入完成、完成记录保持和标识匹配的 ACK，区分算子结束、结果就绪与完成记录释放。

Video 测试由可重复的像素序列产生原图及预期输出，分别核对缩放、通道排列、补边、DMA 写入和显示合成。参考像素根据源坐标与规定的整数运算生成，并随输出流水延迟核对 RGB、同步信号、坐标和帧标识。帧管理测试同时跟踪槽号、帧号及代次，检查显示记录接收、首个有效像素实际输出、旧帧引用解除和释放确认的先后关系。连续帧场景在采集、计算控制和显示读取重叠时核对数据，检测提前复用、重复写入和显示欠载。

系统接口测试通过 CPU 固件或总线激励访问 MMIO，结合字节存储模型检查 CPU、NPU 与 Video 之间的数据传输。AXI 写地址、写数据及写响应通道分别施加背压，读通道独立设置返回延迟；测试包含跨边界访问、错误响应和完成事件延后等场景。已接纳事务的地址、数据和响应对应关系持续跟踪到事务结束，错误注入用例用于检查故障保持、恢复和错误路径隔离。

每个用例设置有限运行周期和预期结束事件，执行脚本同时读取自检标志、错误信息及退出状态，并保存对应日志。测试记录注明采用的外部存储模型和时钟配置；RTL 功能仿真、布局布线时序分析及板级测量分别记录。

B CPU 仿真验证

B.1 指令集验证

本节指令集回归采用单发射 `rv32im_zicsr` 配置；双发射的参数组合与关键接口检查列于系统级仿真验证。回归用例分别取自 `riscv-tests`[11] 和基于 ACT4 框架的 RISC-V 架构测试集[12]。`riscv-tests` 功能测试结果为 **50/50 PASS**，ACT4 架构一致性测试结果为 **53/53 PASS**。支撑材料保存于`docs/verification/cpu/isa_support/`。

B.1.1 riscv-tests 逐项结果

表B-1列出 RV32I 的 40 项、RV32M 的 8 项及机器 CSR 的 2 项测试。

表 B-1 riscv-tests 逐项用例清单（50/50 PASS）

序号	配置	指令集	用例	状态
1	rv32im_zicsr	RV32I	rv32ui-p-simple	PASS
2	rv32im_zicsr	RV32I	rv32ui-p-add	PASS
3	rv32im_zicsr	RV32I	rv32ui-p-addi	PASS
4	rv32im_zicsr	RV32I	rv32ui-p-and	PASS
5	rv32im_zicsr	RV32I	rv32ui-p-andi	PASS
6	rv32im_zicsr	RV32I	rv32ui-p-auipc	PASS
7	rv32im_zicsr	RV32I	rv32ui-p-beq	PASS
8	rv32im_zicsr	RV32I	rv32ui-p-bge	PASS
9	rv32im_zicsr	RV32I	rv32ui-p-bgeu	PASS
10	rv32im_zicsr	RV32I	rv32ui-p-blt	PASS
11	rv32im_zicsr	RV32I	rv32ui-p-bltu	PASS
12	rv32im_zicsr	RV32I	rv32ui-p-bne	PASS
13	rv32im_zicsr	RV32I	rv32ui-p-jal	PASS
14	rv32im_zicsr	RV32I	rv32ui-p-jalr	PASS
15	rv32im_zicsr	RV32I	rv32ui-p-lb	PASS
16	rv32im_zicsr	RV32I	rv32ui-p-lbu	PASS
17	rv32im_zicsr	RV32I	rv32ui-p-ld_st	PASS
18	rv32im_zicsr	RV32I	rv32ui-p-lh	PASS
19	rv32im_zicsr	RV32I	rv32ui-p-lhu	PASS
20	rv32im_zicsr	RV32I	rv32ui-p-lui	PASS
21	rv32im_zicsr	RV32I	rv32ui-p-lw	PASS
22	rv32im_zicsr	RV32I	rv32ui-p-or	PASS
23	rv32im_zicsr	RV32I	rv32ui-p-ori	PASS
24	rv32im_zicsr	RV32I	rv32ui-p-sb	PASS
25	rv32im_zicsr	RV32I	rv32ui-p-sh	PASS

表 B-1 riscv-tests 逐项用例清单（50/50 PASS）（续）

序号	配置	指令集	用例	状态
26	rv32im_zicsr	RV32I	rv32ui-p-sll	PASS
27	rv32im_zicsr	RV32I	rv32ui-p-slli	PASS
28	rv32im_zicsr	RV32I	rv32ui-p-slt	PASS
29	rv32im_zicsr	RV32I	rv32ui-p-slti	PASS
30	rv32im_zicsr	RV32I	rv32ui-p-sltiu	PASS
31	rv32im_zicsr	RV32I	rv32ui-p-sltu	PASS
32	rv32im_zicsr	RV32I	rv32ui-p-sra	PASS
33	rv32im_zicsr	RV32I	rv32ui-p-srai	PASS
34	rv32im_zicsr	RV32I	rv32ui-p-srl	PASS
35	rv32im_zicsr	RV32I	rv32ui-p-srli	PASS
36	rv32im_zicsr	RV32I	rv32ui-p-st_ld	PASS
37	rv32im_zicsr	RV32I	rv32ui-p-sub	PASS
38	rv32im_zicsr	RV32I	rv32ui-p-sw	PASS
39	rv32im_zicsr	RV32I	rv32ui-p-xor	PASS
40	rv32im_zicsr	RV32I	rv32ui-p-xori	PASS
41	rv32im_zicsr	RV32M	rv32um-p-div	PASS
42	rv32im_zicsr	RV32M	rv32um-p-divu	PASS
43	rv32im_zicsr	RV32M	rv32um-p-mul	PASS
44	rv32im_zicsr	RV32M	rv32um-p-mulh	PASS
45	rv32im_zicsr	RV32M	rv32um-p-mulhsu	PASS
46	rv32im_zicsr	RV32M	rv32um-p-mulhu	PASS
47	rv32im_zicsr	RV32M	rv32um-p-rem	PASS
48	rv32im_zicsr	RV32M	rv32um-p-remu	PASS
49	rv32im_zicsr	CSR	rv32mi-p-csr	PASS
50	rv32im_zicsr	CSR	rv32mi-p-mcsr	PASS

B.1.2 ACT4 逐项结果

表B-2列出 I 类 39 项、M 类 8 项及 Zicsr 类 6 项测试。

表 B-2 ACT4 逐项用例清单（53/53 PASS）

序号	配置	指令集	用例	状态
1	rv32im_zicsr	I	I-add-00	PASS
2	rv32im_zicsr	I	I-addi-00	PASS
3	rv32im_zicsr	I	I-and-00	PASS

表 B-2 ACT4 逐项用例清单（53/53 PASS）（续）

序号	配置	指令集	用例	状态
4	rv32im_zicsr	I	I-andi-00	PASS
5	rv32im_zicsr	I	I-auipc-00	PASS
6	rv32im_zicsr	I	I-beq-00	PASS
7	rv32im_zicsr	I	I-bge-00	PASS
8	rv32im_zicsr	I	I-bgeu-00	PASS
9	rv32im_zicsr	I	I-blt-00	PASS
10	rv32im_zicsr	I	I-bltu-00	PASS
11	rv32im_zicsr	I	I-bne-00	PASS
12	rv32im_zicsr	I	I-fence-00	PASS
13	rv32im_zicsr	I	I-jal-00	PASS
14	rv32im_zicsr	I	I-jalr-00	PASS
15	rv32im_zicsr	I	I-lb-00	PASS
16	rv32im_zicsr	I	I-lbu-00	PASS
17	rv32im_zicsr	I	I-lh-00	PASS
18	rv32im_zicsr	I	I-lhu-00	PASS
19	rv32im_zicsr	I	I-lui-00	PASS
20	rv32im_zicsr	I	I-lw-00	PASS
21	rv32im_zicsr	I	I-nop-00	PASS
22	rv32im_zicsr	I	I-or-00	PASS
23	rv32im_zicsr	I	I-ori-00	PASS
24	rv32im_zicsr	I	I-sb-00	PASS
25	rv32im_zicsr	I	I-sh-00	PASS
26	rv32im_zicsr	I	I-sll-00	PASS
27	rv32im_zicsr	I	I-slli-00	PASS
28	rv32im_zicsr	I	I-slt-00	PASS
29	rv32im_zicsr	I	I-slti-00	PASS
30	rv32im_zicsr	I	I-sltiu-00	PASS
31	rv32im_zicsr	I	I-sltu-00	PASS
32	rv32im_zicsr	I	I-sra-00	PASS
33	rv32im_zicsr	I	I-srai-00	PASS
34	rv32im_zicsr	I	I-srl-00	PASS
35	rv32im_zicsr	I	I-srli-00	PASS
36	rv32im_zicsr	I	I-sub-00	PASS
37	rv32im_zicsr	I	I-sw-00	PASS
38	rv32im_zicsr	I	I-xor-00	PASS
39	rv32im_zicsr	I	I-xori-00	PASS

表 B-2 ACT4 逐项用例清单（53/53 PASS）（续）

序号	配置	指令集	用例	状态
40	rv32im_zicsr	M	M-div-00	PASS
41	rv32im_zicsr	M	M-divu-00	PASS
42	rv32im_zicsr	M	M-mul-00	PASS
43	rv32im_zicsr	M	M-mulh-00	PASS
44	rv32im_zicsr	M	M-mulhsu-00	PASS
45	rv32im_zicsr	M	M-mulhu-00	PASS
46	rv32im_zicsr	M	M-rem-00	PASS
47	rv32im_zicsr	M	M-remu-00	PASS
48	rv32im_zicsr	Zicsr	Zicsr-csrrc-00	PASS
49	rv32im_zicsr	Zicsr	Zicsr-csrrci-00	PASS
50	rv32im_zicsr	Zicsr	Zicsr-csrrs-00	PASS
51	rv32im_zicsr	Zicsr	Zicsr-csrrsi-00	PASS
52	rv32im_zicsr	Zicsr	Zicsr-csrrw-00	PASS
53	rv32im_zicsr	Zicsr	Zicsr-csrrwi-00	PASS

B.2 单元级仿真验证

测试台分别实例化 ALU、乘法器、除法器、CSR 和寄存器堆，在 ModelSim 中按组合求值或时钟边沿检查运算结果与控制行为。表B-3汇总自检结果，图B-1至图B-5给出关键场景的原始仿真波形。分别向五类测试的被测输出注入错误后，自检均报告数值不符并拒绝通过。测试台、运行日志、WLF/VCD 和截图登记保存于`docs/verification/cpu/unit_support/`。

表 B-3 CPU 单元级功能仿真结果

单元	测试场景	检查统计	结果
乘法器	四种乘法、符号与边界输入、无效气泡、暂停保持、低位部分积及早期有效条件	2730 次有效结果检查，904 个暂停周期	PASS
除法器	四种除法/求余、除零、有符号溢出、重复操作数、忙时重启及复位	308 次算术操作，312 次完成检查，16 次快速完成	PASS
CSR	六种读改写形式、只读访问、EPC 对齐、非法指令与 ECALL 异常、MRET 返回	21 次 CSR 访问，91 项检查，2 次异常，1 次返回	PASS
ALU	加减、逻辑、无符号/有符号比较、逻辑/算术移位、边界值及伪随机组合	17804 次结果检查，384 组移位扫描	PASS

寄存器堆	同步写入、异步双读、x0 保持零、复位屏蔽和单发射次读口约束	2476 次读值检查，1006 次写入，6 次扫描 PASS

B.2.1 乘法器

测试台以 66 bit 有符号整数乘法计算参考值，按 `MUL`、`MULH`、`MULHSU` 和 `MULHU` 的符号规则选择低 32 位或高 32 位结果。输入包括零、`0xFFFFFFFF`、`0x80000000`、`0x7FFFFFFF` 及伪随机数，并交错插入无效指令和 `hold`。图B-1中，`0x80000000` × `0xFFFFFFFF` 的低位结果为 `0x80000000`；`hold` 拉高时，结果与低位部分积保持，解除暂停后继续推进。`valid_out` 为测试台按相同暂停条件生成的参考有效标志，用于限定结果比较时刻。自检完成 2730 次有效结果检查，并检查 904 个暂停周期内的保持行为。

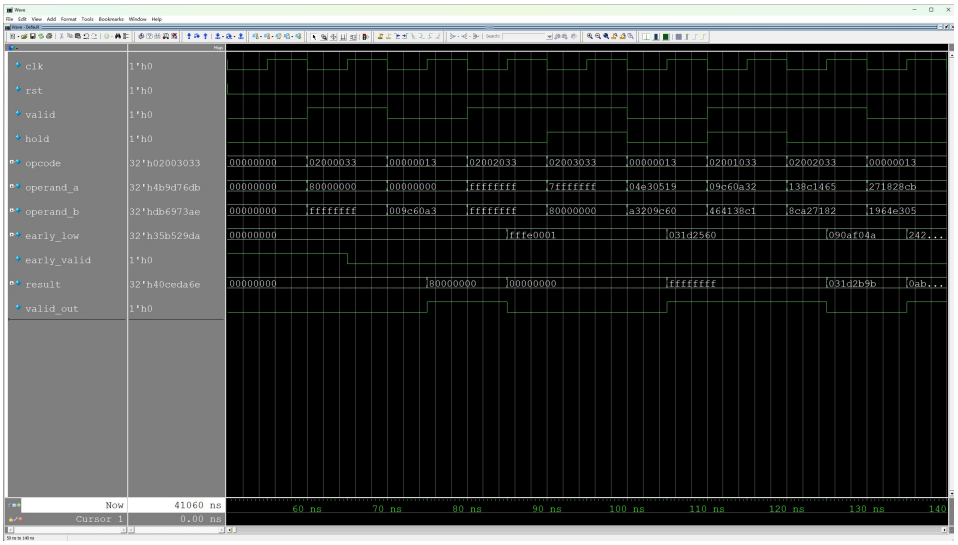


图 B-1 乘法器的运算结果、无效气泡与暂停保持波形

B.2.2 除法器

测试台按 RISC-V 除法规则独立计算商和余数，覆盖 `DIV`、`DIVU`、`REM` 和 `REMU`。除零时商为 `0xFFFFFFFF`、余数为被除数；有符号最小值除以 −1 时，商为 `0x80000000`、余数为零。完成检查同时比较参考除法单元的结果，并核对当前单元相对参考单元延后一周期的完成时序。

图B-2给出 −21 ÷ 4：输入为 `0xFFFFFFEB` 和 `0x00000004`，输出商为 `0xFFFFFFFB`（−5），符合向零截断规则。首次操作经过迭代完成，随后再次提交相同操作数时利用保存的结果快速完成；两次 `candidate_valid` 均比 `golden_valid` 晚一个时钟周期。算术测试完成 308 次操作；随后 1000 周期输入流继续覆盖忙时重启、连续有效输入和复位，累计检查 312 次完成事件。

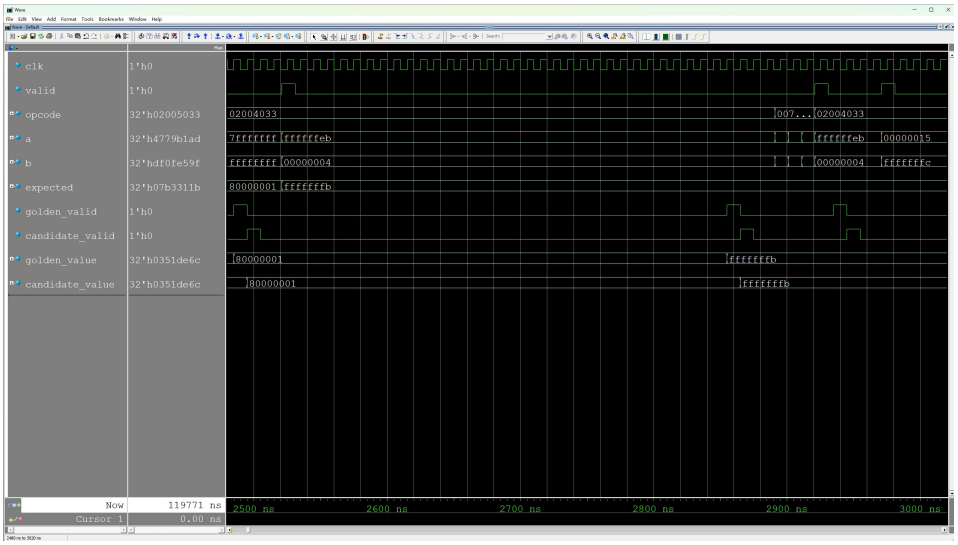


图 B-2 有符号除法结果、完成时序及重复操作数复用波形

B.2.3 CSR

测试台分别驱动指令输入与后续提交接口，检查 CSR 旧值、读改写结果和提交后的寄存器状态。图B-3以 `mscratch`（地址 `0x340`）展示六种 CSR 指令：先写入 `0x12345678`，再用 `0x00F0000F` 置位得到 `0x12F4567F`、清位得到 `0x12045670`；立即数交换、置位和清位依次得到 `0x1F`、`0x1F` 和 `0x1C`。`read_valid` 对应指令读值，`wb_write` 对应后续提交写入；`CSRRS` 以 `x0` 为源寄存器时只读取旧值。

异常测试检查 `mepc` 低两位清零、非法指令的 `mcause=2` 与 `mtval`、ECALL 的 `mcause=11`，以及异常入口和 `MRET` 的重定向地址与状态恢复。自检完成 21 次 CSR 访问和 91 项检查，覆盖两次异常与一次返回。

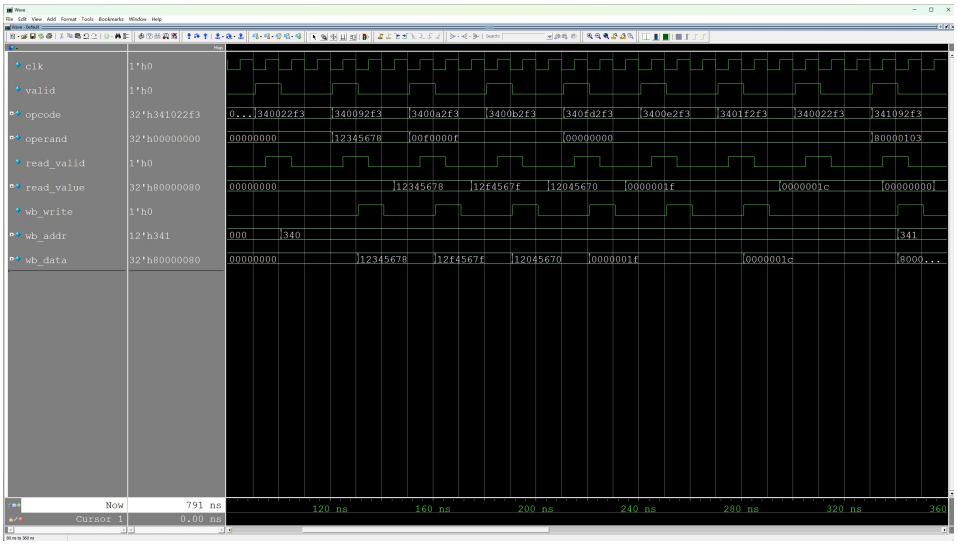


图 B-3 CSR 六种读改写形式的读值与提交写入波形

CSR 地址与中断优先级另由公开端口定向测试检查。主工程与 CPU 维护工作树

分别运行 6 种参数组合，覆盖机器模式与监督模式参数、生产编译与两种显式仿真控制配置，共 12 个实例、7916 个检查点通过。测试遍历 MEI/MSI/MTI 待决位的 8 种组合，检查机器中断优先级、外部中断输入及内部计时比较到期。未实现 CSR、只读 CSR 写入、零源只读访问、固定 MISA 和标准标识寄存器分别核对；`0x7C1` 至 `0x7C3` 检查命令编码、读回值、背压、成功和错误响应，旧 PMP 地址不再触发维护。

原 ISA 用例在维护版 CSR 实现上复跑，riscv-tests 为 50/50、ACT4 为 53/53 通过，ELF 和参考签名保持不变。首轮 102 项通过、1 项失败定位于标准标识 CSR 缺失；显式实现只读零后，该用例通过。原失败、后续结果和输入哈希集中登记于`docs/verification/cpu/document_review_csr_20261102/`。上述计数属于功能仿真，原板级资源、时序及 CoreMark 结果仍对应各自发布配置。

B.2.4 ALU

ALU 结构见图2-9。测试台对每个输入组合用独立的 RV32 参考函数计算结果，覆盖加法和减法的 32 位回绕、按位逻辑、无符号与有符号小于比较，以及逻辑左移、逻辑右移和算术右移。边界集合包含零、全一、最高位、最高位减一、交替位和 `0x80000001`；移位测试逐项检查 0 至 31 的移位量，并令移位操作数高位非零以检查只使用低 5 位。图B-4中，`op` 依次选择加法、移位、比较和逻辑运算，`expected` 与 `result` 在每个检查窗口相同，`check_ok` 标记该组合已经完成比较。测试还对 1024 组伪随机操作数执行全部 16 种控制编码，并覆盖默认操作的直通结果。

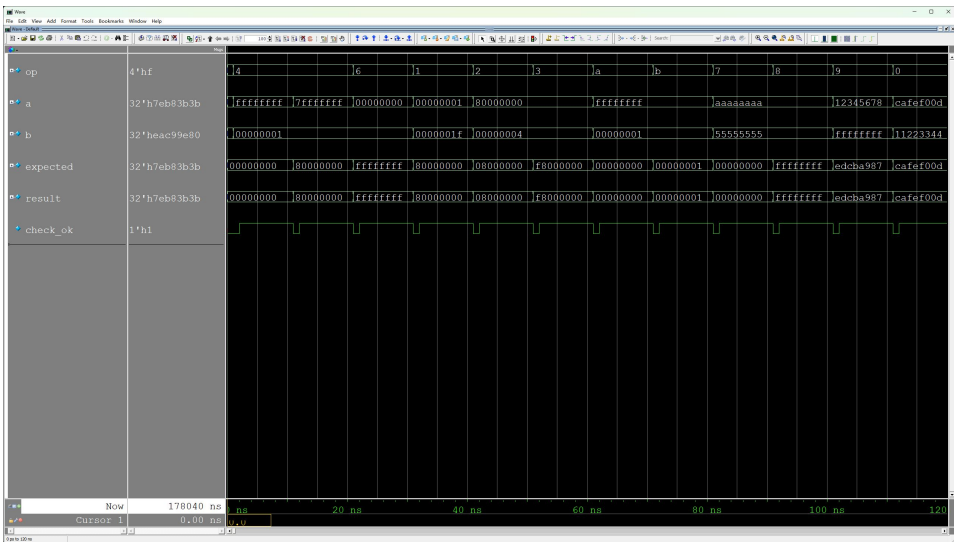


图 B-4 ALU 算术、移位、比较和逻辑运算结果波形

B.2.5 寄存器堆

寄存器堆测试台以数组模型作为参考，按时钟上升沿更新写入，并在地址变化后立即比较两个异步读端口。测试首先验证复位后的全部地址读为零，再逐字尝试写入

32 个寄存器；随后以伪随机地址和数据执行 1000 轮读写，并检查 x0 写入被丢弃。单发射配置的次写口置零，测试台同时检查次读口输出为零。图B-5显示 `rd`=5、6 的写入在时钟边沿后分别出现在 `value_a` 和 `value_b`，地址改变无需额外时钟即可读出；对 x0 的写入没有改变零值。测试同时确认复位在时钟边沿后使全部地址读零，未在该边沿前提前改变读出数据；重写一个寄存器后，其余地址仍保持零值。

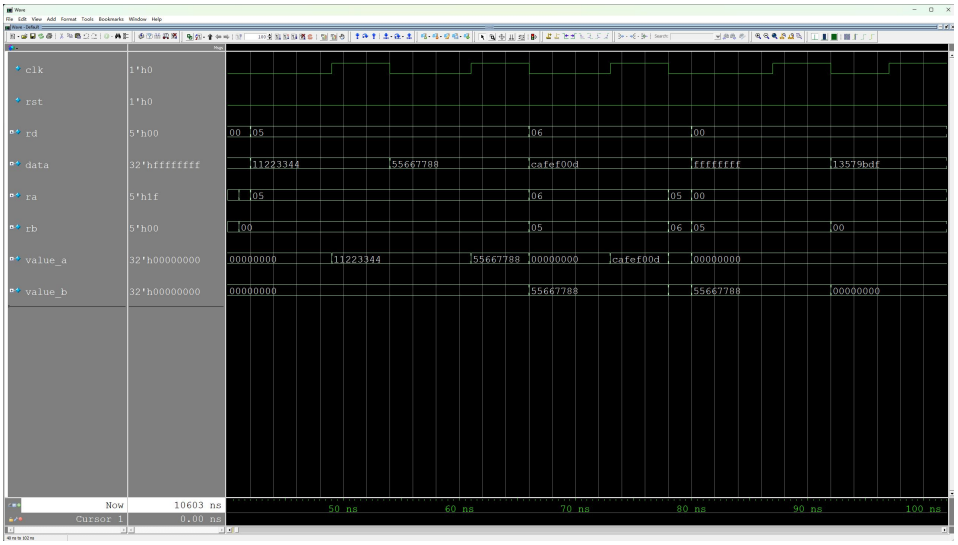


图 B-5 寄存器堆同步写入、异步读出及 x0 保持波形

B.3 模块级仿真验证

模块级测试覆盖前端重定向、发射与旁路、LSU 和 I/D Cache，自检结果见表B-4。前端测试在取指请求接纳、响应返回和指令输出握手处注入延迟，发射测试通过完整核的取指和数据接口运行固定指令序列；LSU 与 Cache 测试分别连接各自的请求和响应端口。结果、自检日志、WLF/VCD 和截图集中保存于`docs/verification/cpu/module_support/`。

表 B-4 CPU 模块级功能仿真结果

模块	场景	检查统计	结果
前端重定向	重定向请求寄存、迟到取指响应、接收端背压和连续重定向	4 次重定向，12 项有效指令输出，4 项迟到响应丢弃	PASS
发射/旁路	连续 ALU、加载、低位乘法、高位乘法和同目的寄存器写回	16 次发射，16 次退休，26 项操作数检查，38 周期	PASS
LSU	加载扩展、写掩码、请求保持、非对齐和总线错误	18 次访问，220/220 项检查	PASS
I/D Cache	命中、填充、两路替换、错误填充、维护及并发	1297/1297 项检查	PASS
数据 Cache 写回	脏行清理、失败重试、替换、刷新与共享区旁路	1355/1355 项检查	PASS

表 B-4 CPU 模块级功能仿真结果（续）

模块	场景	检查统计	结果
数据 Cache 响应重叠	连续命中、响应标签、后续请求及错误隔离	142/142 项检查，12 次连续读取	PASS

B.3.1 前端重定向

测试台关闭动态分支预测，使用固定 32 bit 指令返回，分别设置取指返回延迟、接收端暂停和分支目标。重定向请求在时钟边沿登记，旧取指响应在返回时丢弃；恢复后，接收端只接纳从目标地址开始的低字和高字。图B-6中，`branch` 在目标 `0x80001000` 和 `0x80002000` 处触发，`pending` 覆盖等待窗口，`response` 晚于请求返回，`dropped` 累计迟到响应；`pc0` 保持目标序列，未输出旧地址指令。测试完成 4 次重定向、12 项正确指令输出和 4 项迟到响应丢弃。

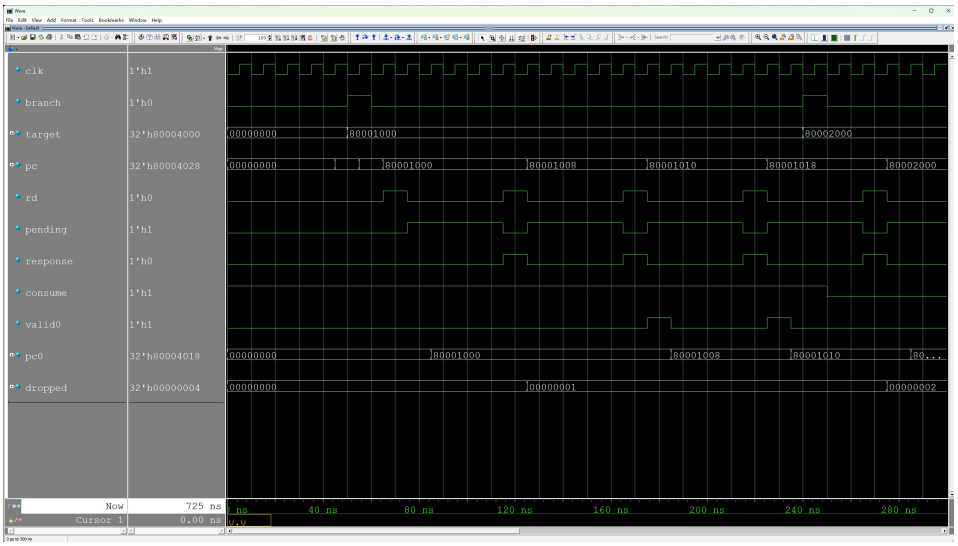


图 B-6 前端重定向请求寄存、迟到响应丢弃和目标指令输出波形

B.3.2 发射与旁路

测试台通过真实指令取指返回运行 16 条顺序指令，包含连续加法、一次加载、低位乘法、高位乘法和两次写入同一目的寄存器。数据端口在接纳请求前插入等待，并推迟加载返回；测试检查相关指令在数据可用后发射，高位乘法保持互锁，满足条件的低位乘法使用 E1 早期旁路。连续两条指令写入同一目的寄存器时，下一条相关指令必须读取较新结果。图B-7中，`d_rd`、`d_accept` 和 `d_ack` 形成一次完整加载事务；`load_e1/load_e2` 和 `pipe_stall` 标出等待区间，`mul_e1/early_mul` 标出低位乘法旁路，`wb_valid/wb_pc` 保持顺序退休。自检在 38 个周期内完成 16 次发射和 16 次退休，检

查 26 个操作数，记录 10 个加载相关等待周期和 1 个高位乘法等待周期；ALU E1 旁路、低位乘法早旁路和最新结果优先级各命中一次。

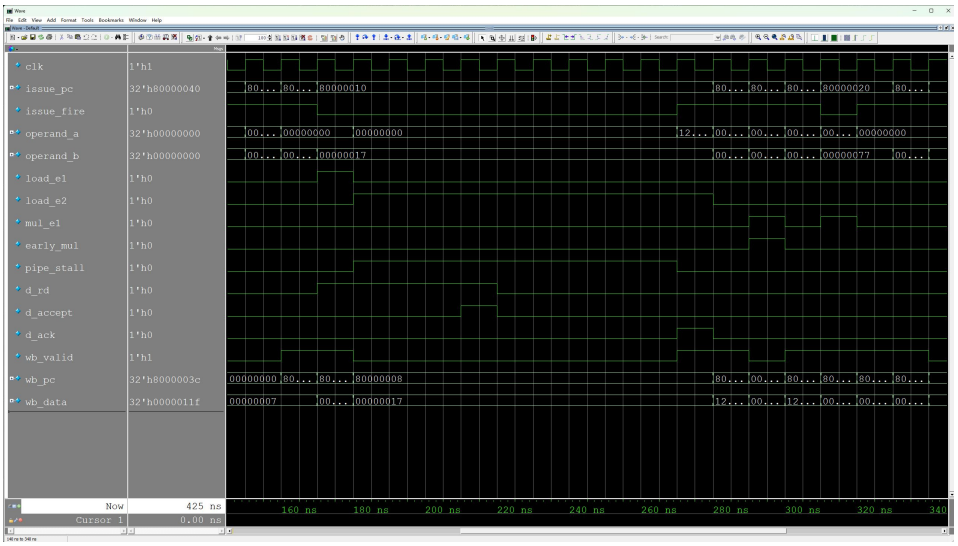


图 B-7 真实取指序列中的加载等待、乘法旁路和顺序写回波形

B.3.3 LSU 端口与异常响应

LSU 测试台连接公开的指令、操作数、请求和写回端口，按 RISC-V 访存规则给出预期数据、写掩码和异常编码。测试覆盖字节、半字和字访问，覆盖未接纳时保持地址和写掩码、加载符号扩展、存储数据排列，以及非对齐和总线错误的精确响应。18 次访问中，11 次加载和 4 次存储形成总线事务，其中 2 次返回总线错误；另有 3 次非对齐访问在发出总线请求前产生异常。测试累计检查 45 个未接纳保持周期，完成 220/220 项检查。图B-8依次给出字节、半字和字存储：写掩码为 `8`、`C` 和 `F`，对应数据为 `DD000000`、`CCDD0000` 和 `AABBCCDD`。`accept` 之前请求字段保持不变；后续非对齐访问仅产生 `wb_valid`，没有读写请求。

B.3.4 I/D Cache 命中、填充与维护

Cache 测试台使用独立的字节寻址内存模型，分别检查指令 Cache 填充和命中、数据 Cache 读写、行替换、写回、失效、刷新和错误重试。组合测试完成 1297/1297 项检查，产生 25 次指令下层读、27 次数据下层读和 25 次下层写，记录 2 次指令命中、2 次数据命中及 35 个 I/D 同时在途周期。图B-9中，`arvalid` 发起八拍填充，`rvalid/rlast` 返回整行数据，第二次相同地址直接由 `i_valid` 完成。

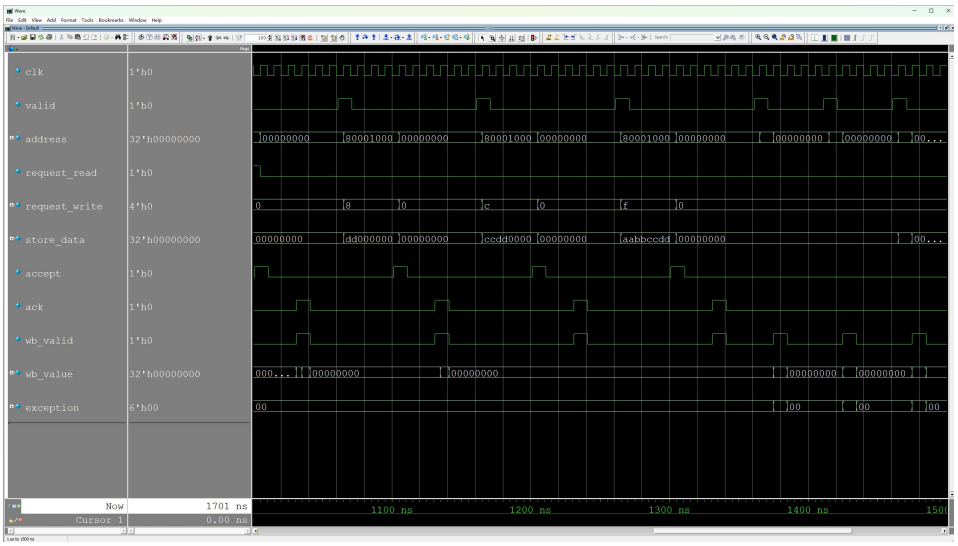


图 B-8 LSU 请求保持、字节写掩码和写回完成波形

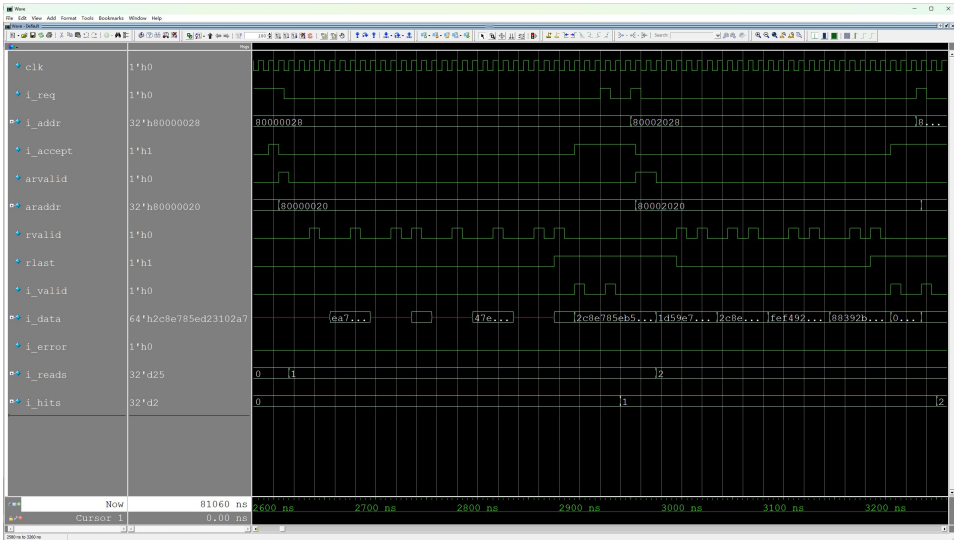


图 B-9 指令 Cache 填充、返回和命中波形

B.3.5 数据 Cache 写回与维护

写回测试完成 1355/1355 项检查、35 次下层读和 184 次下层写，覆盖全部 15 种非零字节掩码、脏行清理、替换前写回、整 Cache 刷新和共享区旁路。测试逐一在八个写回字上注入下层错误，检查失败后仍能读取脏数据，重新清理时完整写回八个字；填充错误也逐拍注入，失败行不得变为有效行。图B-10中，`clean` 启动行清理，`writes` 由 0 递增至 8，`ack` 在最后一次下层写响应后给出；随后再次读取和清理没有新增下层读写。

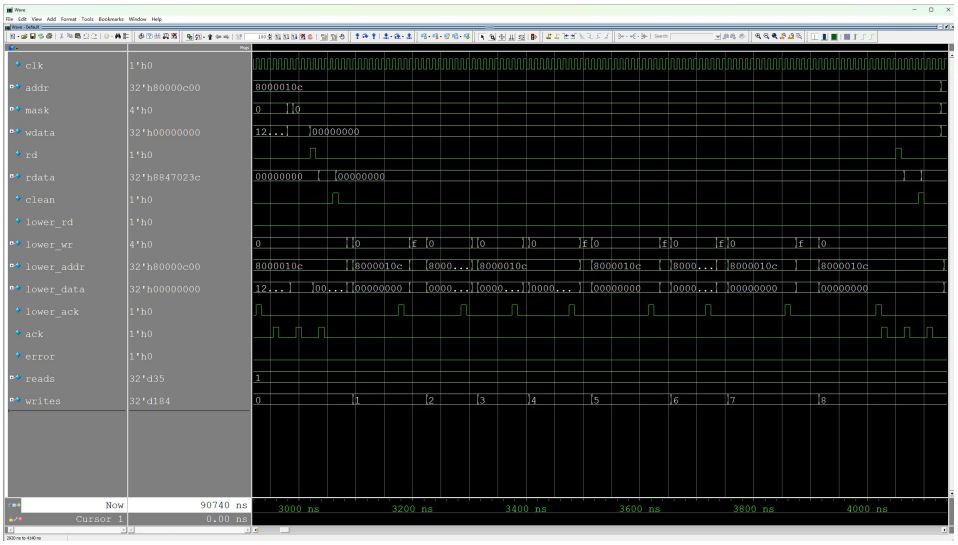


图 B-10 数据 Cache 脏行写回和清理完成波形

B.3.6 数据 Cache 响应重叠

重叠测试完成 142/142 项检查、11 次下层读和 25 次下层写，覆盖命中读取响应与下一请求重叠、标签顺序、跨组跨标记访问和错误响应隔离。图B-11在 `score_active` 期间连续发出 12 次命中读取，`tail` 统计接纳数、`head` 统计返回数，最终均为 12。命中读取每两个时钟周期接纳一次，`accept` 与旧请求 `ack` 在完成边沿重叠，`resp_tag` 按顺序返回，期间 `lower_rd` 保持为零。测试还分别接入存储、Cache 维护、共享区读取及下层错误，检查返回错误响应时不接纳后续请求、存储更新保持串行，脏行替换失败后数据仍可读取。

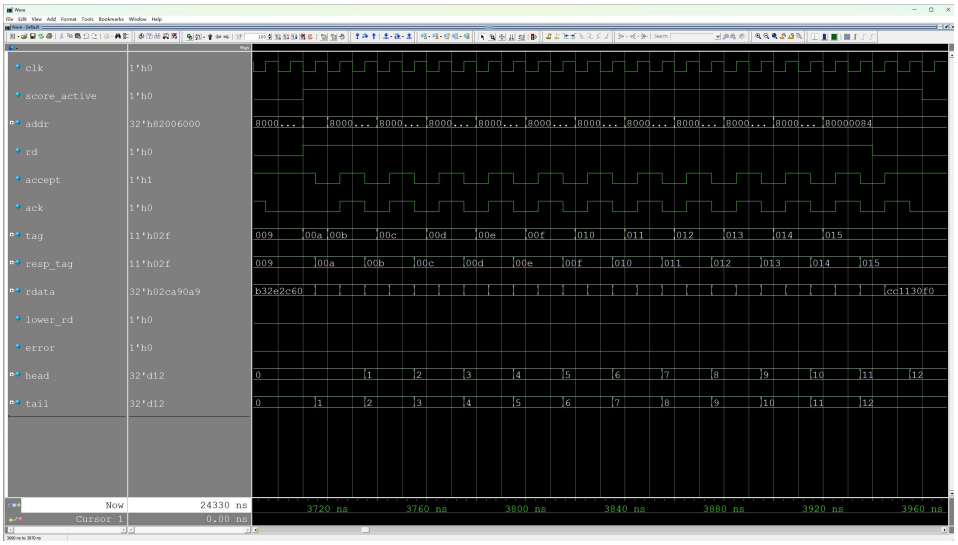


图 B-11 数据 Cache 响应重叠、标签顺序和连续命中波形

B.4 系统级仿真验证

B.4.1 存储访问与故障处理

系统级测试使用最小固件连接 Boot ROM、片上 SRAM、UART、私有 Cache、共享区旁路和模型 AXI128 下层。缓存配置下完成启动、私有区读写、共享区读写、`FENCE.I` 后的代码执行、DDR 就绪等待、下层读错误和平台故障后的本地访问；固件通过 `mtvec/mepc/mcause/mtval` 检查三次受控异常返回。正例自检完成 1277 项检查，统计 26356 个时钟周期、14 次读事务、35 次写事务和 3 次陷阱，返回码为零。图B-12选取第三次异常：平台故障使 `ddr_ready` 撤销、`fault` 锁存；随后访存错误使 `trap_count` 从 2 变为 3，`mcause=5`、`mtval=0x80000000`。异常处理程序将 `mepc` 从 `0x000101FC` 加 4 至 `0x00010200` 后执行 `MRET`；程序继续访问片上存储，最终 `done=1` 且 `exit_code=0`。

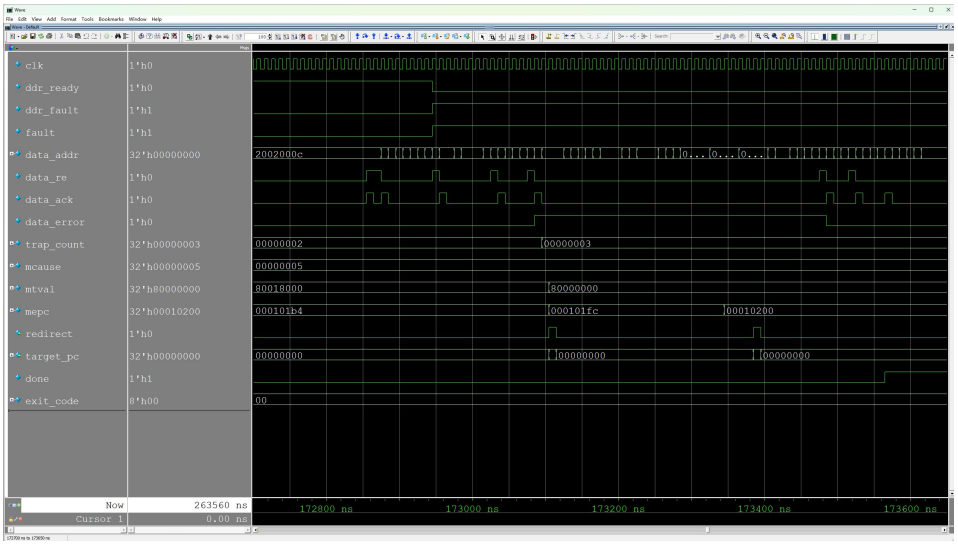


图 B-12 整核固件的平台故障、访存异常与 MRET 返回波形

独立路由测试完成 476 项检查、12 次读事务和 9 次写事务，覆盖片上地址不出 DDR、DDR 未就绪时请求保持、I/D 命中与共享区旁路、部分写掩码、响应所有权、错误锁存及复位恢复。同一固件关闭 Cache 后通过 809 项检查，作为存储属性对照。下层读数据异或一位的错误注入在固件退出检查处触发失败，负例退出码为 1。原始日志、固件、WLF/VCD 与截图保存在`docs/verification/cpu/system_support/`。上述结果来自 ModelSim 中使用行为级 AXI 内存模型的数字仿真。

B.4.2 外部中断进入与返回

中断测试采用单发射核、512 项直接映射 BTB、512 项 BHT 和 4 项 RAS，关闭 GShare，连接 128 KiB 行为级指令与数据存储器。固件先检查 8 类同步异常，再设置 `mie.MEIE` 和 `mstatus.MIE`；测试台在触发地址的存储请求被接纳后驱动外部 `irq`。中

断处理程序检查机器外部中断原因，清除 mie、将处理计数加一，再执行 MRET；返回后固件检查计数恰好为一并正常退出。

图B-13中，中断进入时 mcause=0x8000000B，mepc=0x80000150，mtval=0。mstatus 由 0x00000088 变为 0x00001880，对应 MIE 清零、MPIE 置位和机器态保存；MRET 后恢复为 0x00000088，退休记录从保存的 mepc 继续执行。固件完成 3582 个周期、864 次退休和 92 次存储，8 类同步异常与 1 次外部中断的覆盖掩码为 0x1FF。

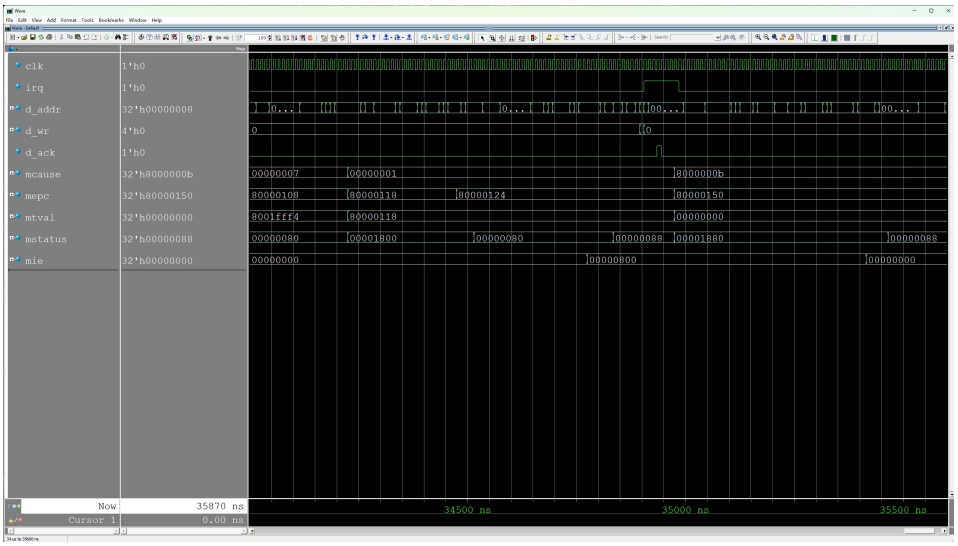


图 B-13 单发射整核的机器外部中断进入、CSR 保存和 MRET 返回波形

B.4.3 Cache 访存与中断交互

整核测试开启 16 KiB 指令 Cache 和 16 KiB 数据 Cache，两者均为两路组相联、32 B 缓存行，数据 Cache 采用写回与写分配。启动程序从片上 SRAM 复制测试代码到私有可缓存区，执行 FENCE.I 后转入该区；测试程序与中断处理程序均通过指令 Cache 取指。测试台在数据 Cache 命中返回、缺失填充等待和缺失加载返回三个时刻分别驱动外部中断，结果见表B-5。

表 B-5 Cache 开启时的外部中断场景

场景	中断触发条件	保存的 mepc	结果
命中返回	驻留数据加载返回，不产生新的下层填充	0x80004060	PASS
填充等待	缺失读事务已接纳，下层尚未返回数据	0x80004094	PASS
缺失返回	新缓存行填充完成，目标加载返回	0x800040C8	PASS

三次中断入口均检查 mcause=0x8000000B、mtval=0、mstatus=0x1880 和 mie=0x800。处理程序清除 mie，保持原 mepc 并执行 MRET；返回后 mstatus 恢复

为 0x88，首条退休指令的 PC 等于原 mepc。三个目标加载分别得到 0x13579BDF、0x2468ACE0 和 0x31415926，各加载及其后继指令均只退休一次，软件计数依次为 1、2、3。

图B-14展示填充等待场景：irq 在 114590 ns 置位，随后下层返回缓存行并完成加载，115395 ns 进入中断，115855 ns 执行 MRET。外部中断有效且读事务待完成、下层无有效返回的状态累计持续 66 周期。整次自检完成 11758 周期、1110 次退休和 12655 项检查，包含三次中断进入、三次返回及三次恢复执行。将处理程序中的 mepc 故意加 4 后，返回地址检查检出错误，测试入口返回 1。

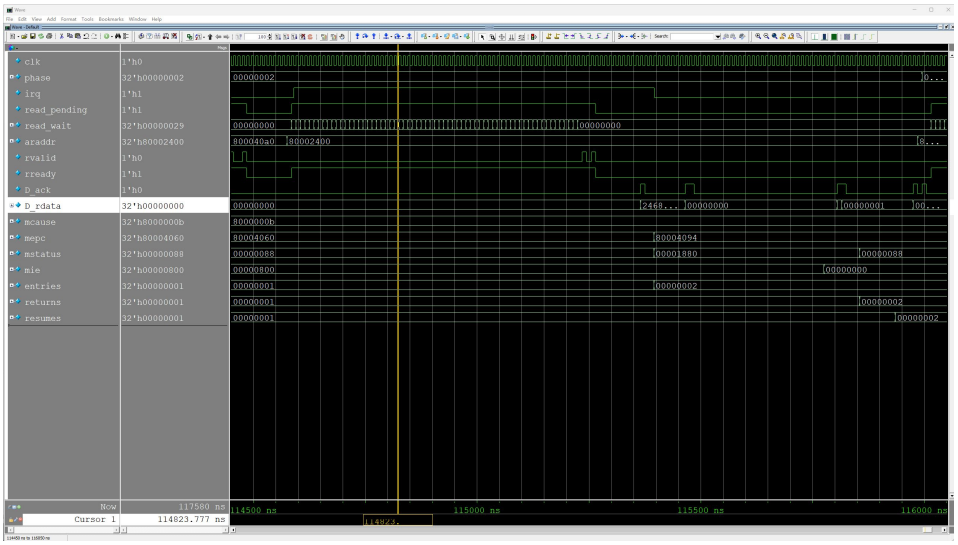


图 B-14 Cache 缺失填充等待期间的中断请求、加载完成与 MRET 返回波形

B.4.4 流水冲刷后的副作用隔离

整核测试通过公开取指和数据端口运行定向固件，分别检查分支误预测重定向与加载访问异常后的后续指令。三处分支的错误路径分别放置 SRAM 存储、MMIO 写入和 mscratch 写入；四处长延迟错误加载后分别放置这三类写入及一条写同一目的寄存器的加载。测试台在请求接纳前插入背压，将加载返回延迟设为 32 或 64 周期，并以 0xDEAFBEEF 作为错误响应和无效响应周期的数据。表B-6列出七个检查场景。

表 B-6 流水冲刷后的副作用隔离测试

触发条件	后续指令	检查结果	状态
分支误预测重定向	SRAM 存储	受保护字保持 0x13579BDF	PASS
分支误预测重定向	MMIO 写入	仅有一次合法的 0x42 写入	PASS
分支误预测重定向	CSR 写入	mscratch 保持 0x12345678	PASS
加载访问异常	SRAM 存储	无额外存储，受保护字不变	PASS
加载访问异常	MMIO 写入	无额外外设写入	PASS

表 B-6 流水冲刷后的副作用隔离测试（续）

触发条件	后续指令	检查结果	状态
加载访问异常	CSR 写入	无额外 CSR 写入	PASS
加载访问异常	同目的寄存器加载	后续请求未接纳；返回后重新加载正确值	PASS

七条禁止执行的指令均被取回，退休流中均未出现；其中四条异常后的后续指令进入译码。四次异常均保存故障加载的 PC，`mcause=5`、`mtval=0x8001F080`，故障加载不改变目的寄存器。处理程序跳过故障片段后执行 `MRET`，随后四次加载复用同一目的寄存器 `x20`，均返回 `0x11223344`。两组返回延迟分别完成 750 周期、2199 项检查和 1102 周期、3031 项检查，各退休 162 条指令，完整退休序列一致。另设四组错误输入：分别反转三处分支条件及将加载错误响应改报成功，均触发副作用检查失败。

图B-15展示第一次加载访问异常：2285 ns 返回错误响应，随后产生 `squash` 并转入异常入口；`mepc` 保存 `0x800000C0`、`mcause` 置为 5，`mscratch` 始终保持原值。异常返回后，恢复加载使 `recovered` 由 0 变为 1。该场景验证阻塞式 LSU 等待期间禁止接纳后续请求，以及异常后的恢复执行。本组测试不实例化 Cache，MMIO 由测试台状态模型提供；冻结输入、原始波形、退休流和运行索引见`docs/verification/cpu/system_support/SQUASH_README.md`。

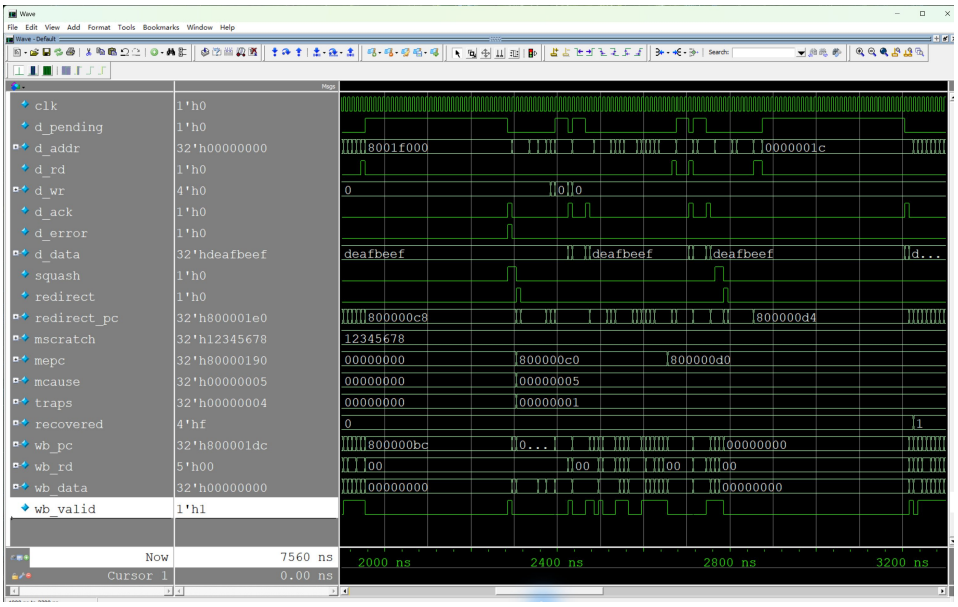


图 B-15 长延迟加载错误、流水冲刷和异常返回后的恢复加载波形

B.4.5 发射宽度与 Cache 配置组合

DUAL_ISSUE 与 CACHE_ENABLE 分别选择单/双发射和 I/D Cache 开关。ModelSim 在 CPU 与存储子系统测试、SoC 运行时测试中各覆盖四种组合，八组测试均通过。表B-7列出各次运行的双指令配对周期及 B 通路 WB 事件，两个数值均为完整测试运行的观测计数。单发射组合的配对周期和 B 通路 WB 事件均为零，双发射组合均观测到第二通路实际工作。

表 B-7 发射宽度与 Cache 配置组合验证

发射配置	Cache	CPU 测试：配对/WB1	SoC 测试：配对/WB1
单发射	开启	0 / 0	0 / 0
单发射	关闭	0 / 0	0 / 0
双发射	开启	39 / 34	80 / 72
双发射	关闭	25 / 22	80 / 72

PDS 综合与器件映射另检查单发射＋ Cache 开启、双发射＋ Cache 关闭及双发射＋ Cache 开启三种组合，实际展开参数与目标配置一致。该映射检查用于核对参数传递及生成结构，板级时序采用第三部分列出的独立后布线结果。记录归档于docs/verification/cpu/report_configurable_issue_2026-11-02/ 中的参数矩阵文件；指令集回归和定向接口仿真按各自配置统计。

B.4.6 双发射关键接口定向验证

本组仿真针对双发射配置的队列出队、整数结果寄存、分支恢复、乘法早期旁路及 LSU 元数据接口，使用 ModelSim 执行定向测试。输入清单、源码校验值、逐周期观测和原始日志登记于docs/verification/cpu/diagram_review2_2026-11-02/。该双发射配置的定向检查结果见表B-8。

表 B-8 双发射关键接口定向检查

检查对象	覆盖与观测结果	状态
FIFO 组件	20821 步，包含写指针 2008 次回绕与读指针 1926 次回绕；组件允许的操作与 CPU 出队约束分别检查	PASS
整核分支与双槽	156 周期、9 个保持周期、4 次恢复、3 次陷入，包含错误路径 Store 抑制、B 槽 ALU 取消、前序加载保持及恢复交叠	PASS
CPU 出队及 ALU 寄存	在上述整核程序中逐周期检查 B 出队必须伴随 A 出队；非保持沿后两路结果等于沿前组合 ALU 结果	PASS
乘法早期元数据	41027 个样本，对照完整写回值与早期值及其可用条件	PASS

表 B-8 双发射关键接口定向检查（续）

检查对象	覆盖与观测结果	状态
取指重定向	4 次重定向，输出 12 包、丢弃 4 包；检查错误路径返回不进入恢复指令流	PASS
LSU 元数据 FIFO	深度 2 与深度 4 各 10001 周期，检查组件空、满及同时出入队行为	PASS

原强制输入零比较配对测试出现期望 B 发射而实际未发射，失败定位于夹具未同步提供简单 ALU 资格、寄存器角色、操作数零标志及预测下一 PC，且配对预期沿用了旧策略。完整驱动现行接口的后继夹具通过 6168 个检查点；原测试、断言和 FAIL 记录保留，未修改发射 RTL。

真实取指、预译码、FIFO、发射及 LSU 路径的 72 次定向程序结果见表B-9。零比较用例按 BEQ/BNE、`x0` 位于 rs1/rs2、B 为 ALU/Load/Store 组合；每次程序还检查 A 生成数据、B 等待后存储的 RAW 序列。确认不跳转时允许后继访存，未确认或跳转时禁止后续 LSU 请求；推测 ALU 在恢复或异常时取消，并按 PC 检查写回值及外部访存次数。

表 B-9 双发射零比较分支与恢复的整核定向验证

场景	主要观测	结果
操作数就绪且确认不跳转	12 次目标配对；ALU、Load 和 Store 正常完成	12/12 PASS
实际跳转、冷启动预测未命中	12 次恢复，4 个推测 ALU 取消，8 个 LSU 后继无外部请求或退休	12/12 PASS
前序 Load 至 A 分支 RAW	累计等待 108 拍，12 次取得加载前递后配对，B 不越过 A	12/12 PASS
分支目标不对齐异常	12 次陷入与 MRET；4 个推测 ALU 取消，LSU 后继不执行	12/12 PASS
前序 Load 至 B 的 RAW	12 次 B 阻塞，A 先发射，B 取得结果后独立执行	12/12 PASS
普通双源分支对照	6 种组合各重复两次；4 个简单 ALU 配对，8 个 LSU 后继独立执行	12/12 PASS

当前 FIFO 队首出队约束下的前递检查另通过 694800 个检查点，覆盖两路数据与零标志、六级前递优先级、RF 写入、复位及 X/Z。保留失败包括 3 次旧夹具运行和 9 次诊断/故障注入；错误开放 LSU 配对、禁止 B 取消及破坏零标志均触发相应断言。源码版本标识、逐拍记录、覆盖矩阵和完整通过/失败记录见`docs/verification/cpu/document_review_zero_pair_20261102/`。本组结果对应上述配置下的零比较分支配对与前递定向仿真。

C NPU 仿真验证

本节整理 NPU 计算、缓冲、DMA 和控制接口的模块及子系统仿真结果。波形以 ModelSim Wave 窗口截图展示，数值结果采用对应测试台与整数参考模型的比较记录。各用例分别注明阵列规模、量化通路和存储模型，按其实际组件配置统计结果。50 MHz、$M = 2$、单量化通路的头像应用整机验证另见附录F。截图来源、测试记录和文件校验值保存于`docs/verification/npu/report_waveforms_3523d03/`。

表C-1汇总代表性数值用例与向量算子回归结果。前三项是空间层回归中的单个用例；后两项分别统计两种量化配置下的向量算子集合。

表 C-1 NPU 代表性数值验证结果

测试对象	测试配置与比较范围	匹配字节数
DW16	17 通道，4×19 输出位置，S16，Q4 时分量化	2584
MaxPool	64 通道，7×41 输出位置，7 行写回	18368
Resize	3 通道，40×38 输出位置，40 行写回	4560
向量算子回归	共享八通道量化器、片上存储原语及本地存储模型，14 例	150092
向量算子回归	Q4 时分量化、片上存储原语及本地存储模型，9 例	149212

C.1 计算正确性

普通卷积阵列采用 $M = 16$ 的组件配置，观察 K 向量接收、乘加提交和本地累加器更新，波形如图C-1所示。`in_k` 与 `retire_k` 分别标记输入和提交的归约索引，`accepted` 与 `retired` 记录对应事件数，`pending` 记录在途向量数。第 0 路累加器 `acc_flat[31:0]` 随有效提交更新，测试台同时检查其余通路及异常输入。阵列回归包含 9 个正常用例和 12 个错误用例，统计 223 次接收、219 次提交和 4 次清空，结束时在途计数为 0。

DW16 用例采用 17 个输入、输出通道，覆盖一个完整 16 通道组和一个尾通道，输出尺寸为 4×19。图C-2记录层控制状态、运算计数和完成信号。两个通道组分别遍历 76 个空间位置和 9 个核采样点，共产生 1368 个 MAC 向量；按该用例的 Q4 时分量化配置，统计 304 个量化组和 4 行写回。测试台对 $17 \times 4 \times 19 \times 2 = 2584$ B 的 S16 输出逐字节比较，结果与整数参考一致。

定点后处理单元的数值验证采用八条物理量化通路，检查 Bias 相加、整数系数乘法、右移、最近偶数舍入及目标类型限幅。输入接收与输出交付同时受共同推进条件和 `run_ce` 控制；暂停或输出背压期间，待交付数据和标签保持稳定。图C-3上窗展示混合样本的输入与输出，下窗展示第 0 路有符号重定标和 U8 限幅：舍入结果为 -64 时，输出值为 0。该单元回归核对 105321 个有效通道算术样本，并覆盖 256 种通道掩码、输出保持及清空恢复。图中的周期和流水延迟按八通道单元配置记录，正文中的

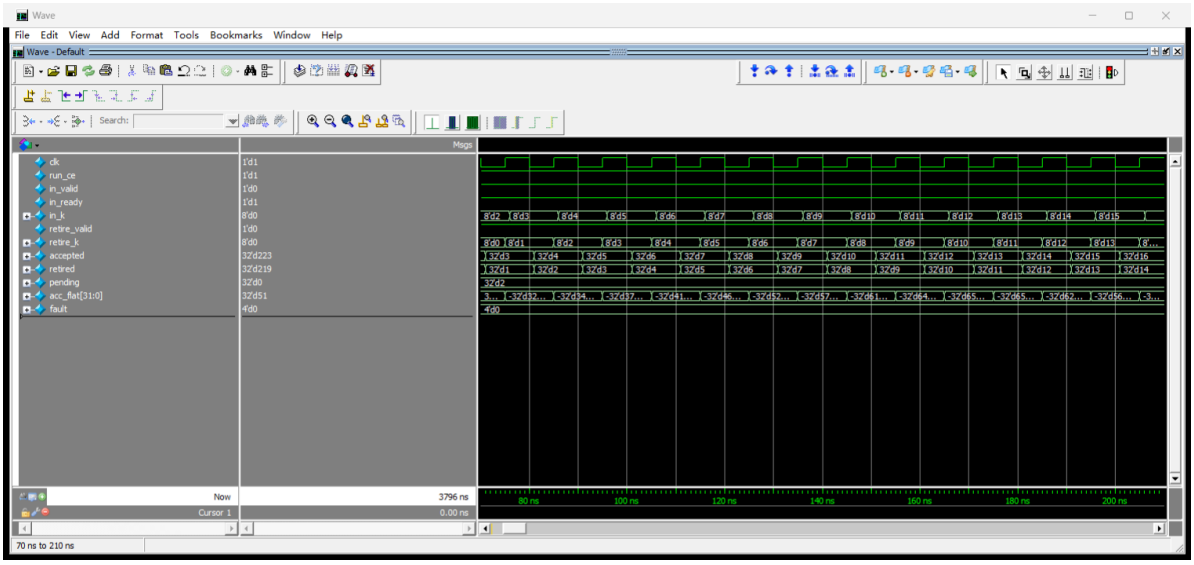


图 C-1 普通卷积阵列的输入、提交与累加波形（$M = 16$ 组件配置）

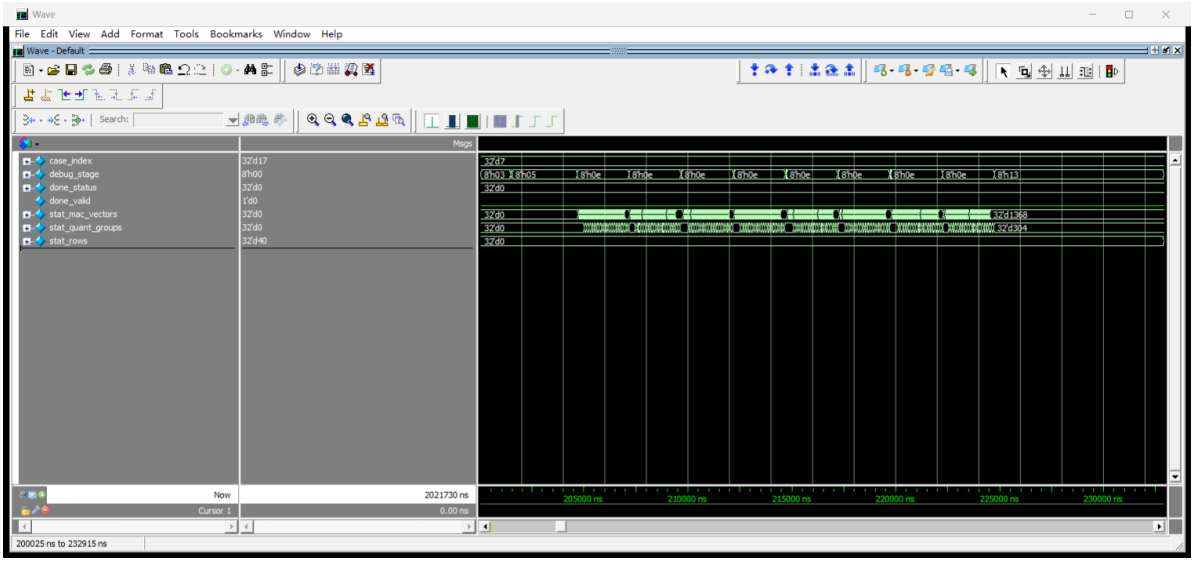


图 C-2 DW16 尾通道用例的层级计数与完成波形

单、双通路配置采用相同整数规则。

C.2 分块计算

输入缓冲验证分别检查驻留行复用和卷积窗口供数，波形如图C-4所示。上窗采用 $C_{\text{in}} = 64$ 的驻留命中用例，输入行已保存在片上；行装载状态更新用时两拍，期间外部读请求和读返回保持无效。下窗采用独立的窗口生成用例，`roi_req_a` 给出 ROI 读取请求，`a_write_valid` 与 `a_write_index` 标记依次写入的 A 向量。两个窗口分别沿用各自用例的时间轴。测试台依据输入张量、窗口坐标及补边规则检查返回数据与写入索引。

输出尾块用例采用 16 空间位置、16 输出通道的缓存组合配置，其中 7 个空间位

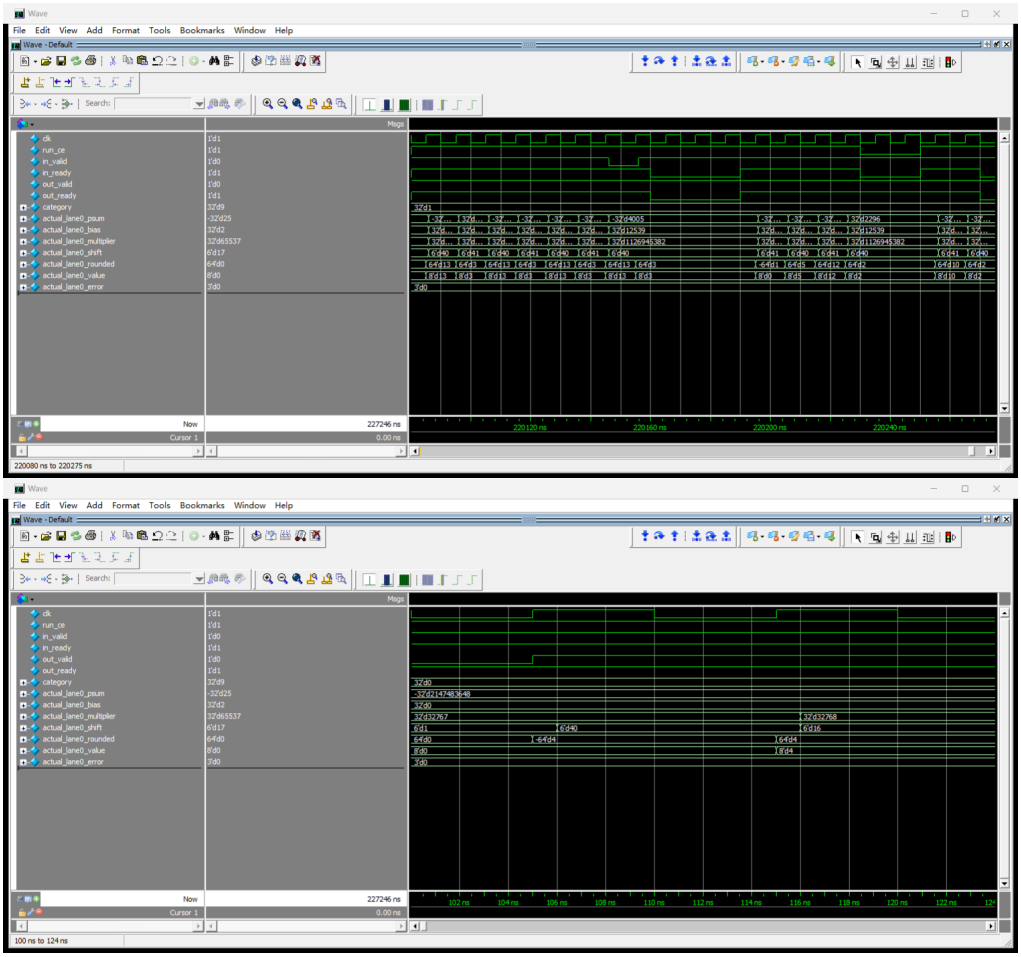


图 C-3 定点后处理的重定标与 U8 限幅波形（八通道单元配置）

置和 9 个输出通道有效。每个位置的量化结果按 8 个通道和 1 个尾通道写入，字节使能限定实际更新范围。图C-5上窗显示 512 位 `o_valid` 有效字节位图及量化组计数，下窗显示本地读请求和读响应。位图中每一位对应一个字节的有效状态；测试台按有效位置比较输出，并以预置哨兵检查相邻无效区域。同一缓存组合回归共核对 3840 个 O 数据字和 720 个 A/B/P/O 保护区数据字，读回结果均一致。

C.3 DMA 传输

块传输验证采用独立读写桥，本地域运行于 100 MHz，DDR 用户接口域运行于 125 MHz，连接 HMIC 控制器和两颗 x16 DDR3 器件模型。测试台发出写入与读回命令，在读数据和完成接口施加背压，并逐字节检查返回值。图C-6截取用例 17 至 20，展示地址接收、写数据末拍、读返回末拍和本地完成握手。`done_valid` 保持至 `done_ready` 允许接收，完成状态随记录一并交付。

该器件模型用例执行 12 次写入和 12 次读取，原生写拍与读返回拍均为 101 拍，3232 次字节比较全部匹配。覆盖的突发长度为 1、2、8、15、16 拍，同时检查全字节写、交替字节掩码和 4 KiB 边界两侧访问。测试台完成 259 次本地读数据保持检查、240 次

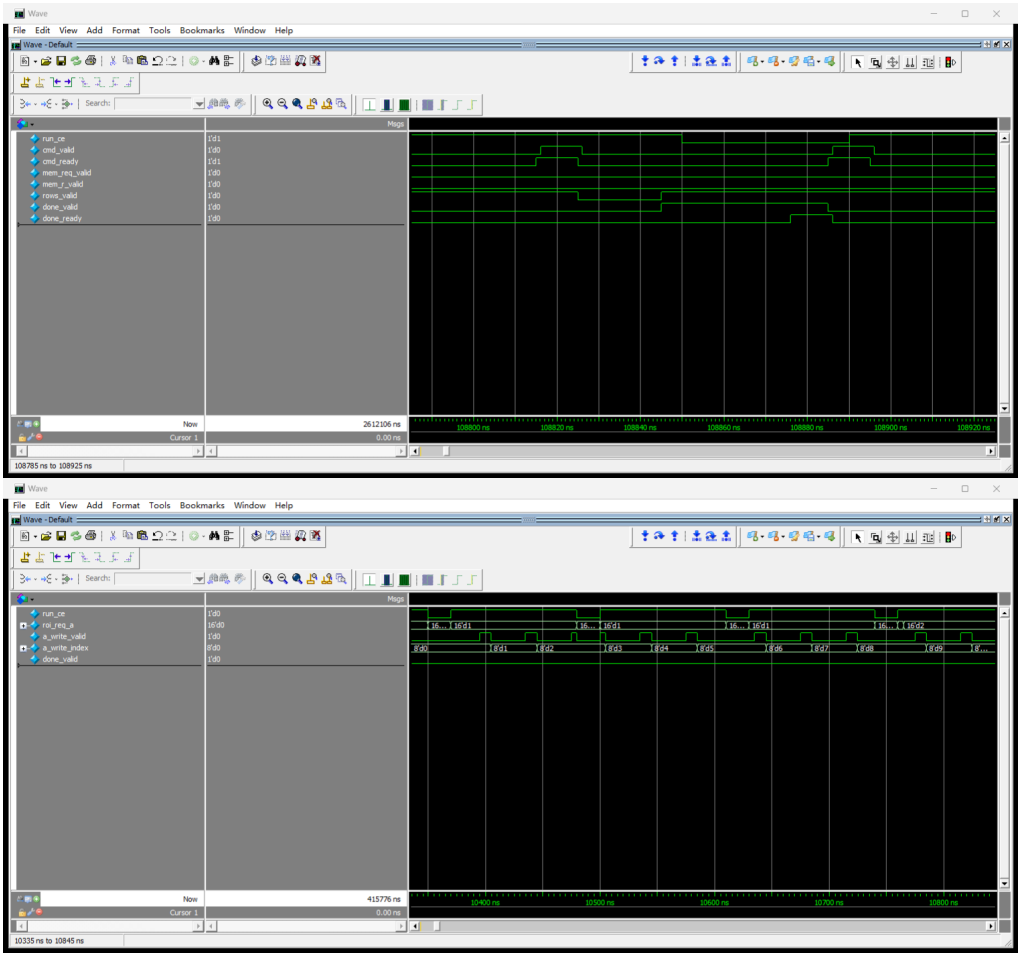


图 C-4 输入行驻留复用与卷积窗口供数波形

完成记录保持检查和 1746 次事务所有权保持检查。上述结果的统计范围为独立桥同一端口的功能读写与完成顺序。器件模型记录保留 267 条连接或端口警告，以及 84 条初始化和写校准运行时警告；功能数据比较与器件时序检查分别记录。

C.4 任务控制

单算子 MMIO 验证检查参数锁存、启动接收、完成等待、任务身份核对和故障响应。图C-7上窗展示层核提出完成时外部事务尚在处理的情况：`done_valid` 有效后，`retired` 为 0，`done_ready` 保持为 0；外部事务结束后，接口接收完成记录。下窗展示故障状态的置位及复位清除。配套用例分别注入错误任务标签和会话标识变化，检查接口对任务身份的判定。该接口回归共 464 项检查通过，覆盖 AXI4-Lite 写地址与写数据独立到达、背压、身份匹配 ACK 及完成发布条件。

输入 FIFO 验证以参考实现与载荷有效位适配实现逐拍比较公开端口，覆盖清空、复位、暂停和重新装载。图C-8中，清空后占用计数与 `payload_live` 均归零，公开输出 `od_dut` 与参考输出 `od_ref` 一致。私有存储体保留的字节由有效位隔离；重新装载后，两种实现恢复相同的计数和输出序列。完整回归记录 3202888 项检查通过，检查

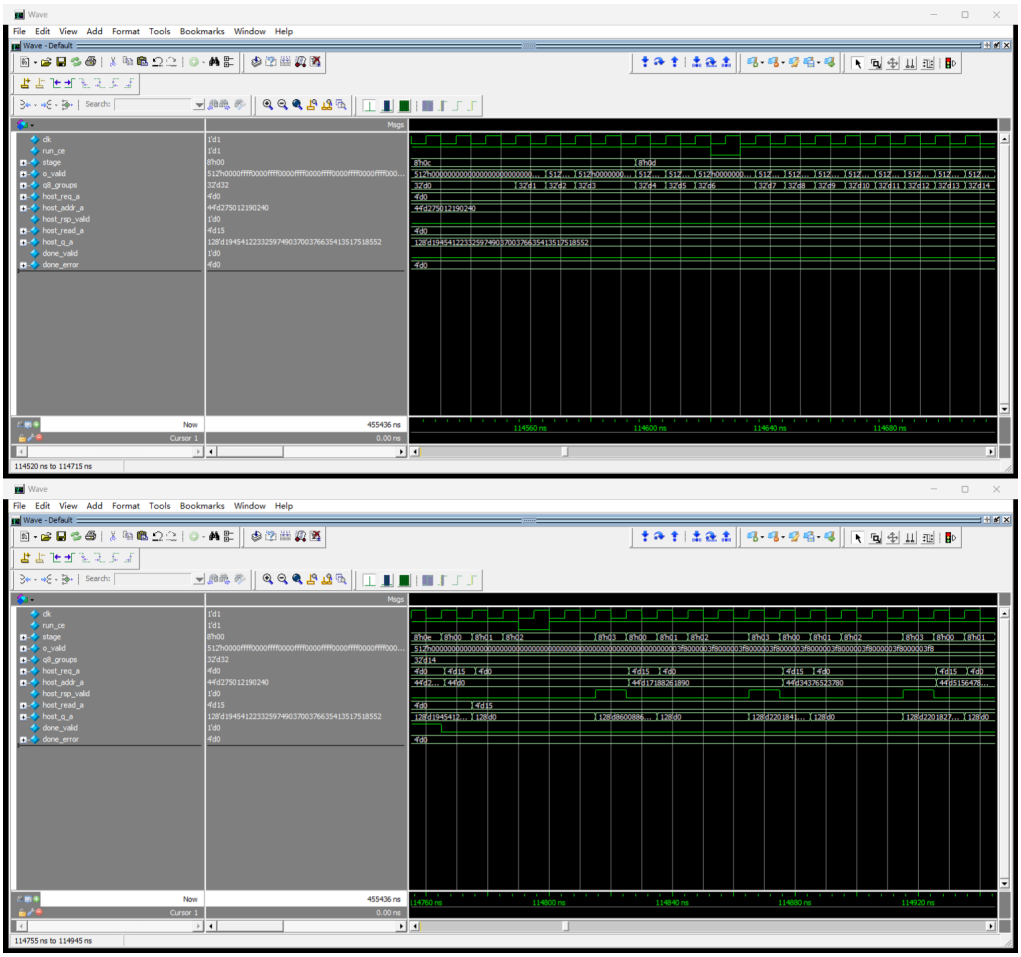


图 C-5 输出尾块的有效字节位图与本地读回波形

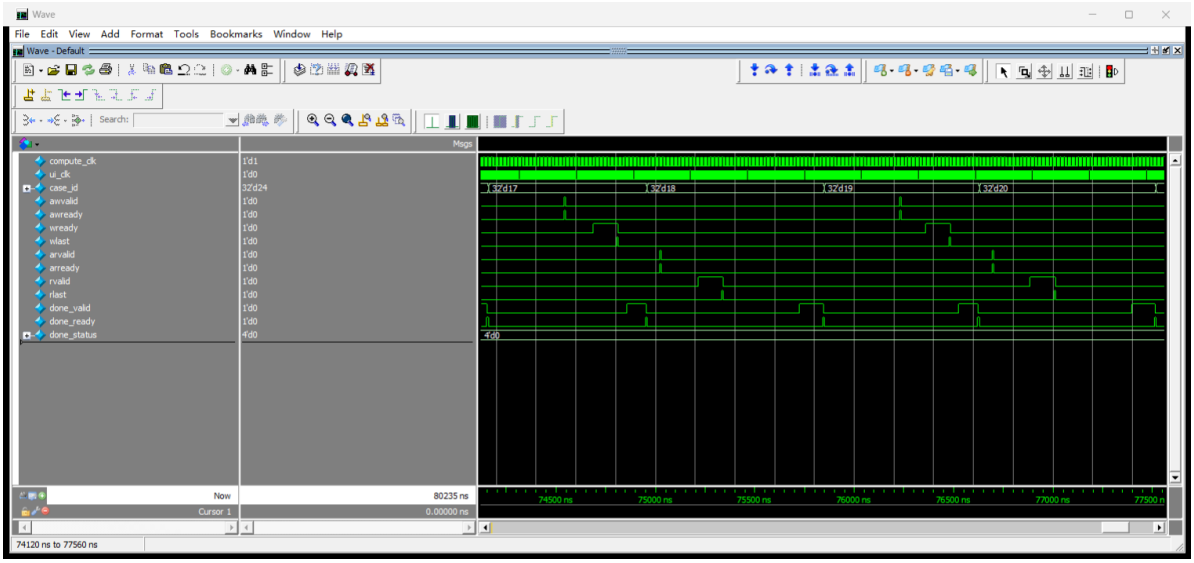


图 C-6 独立读写桥的跨时钟域传输与完成握手波形

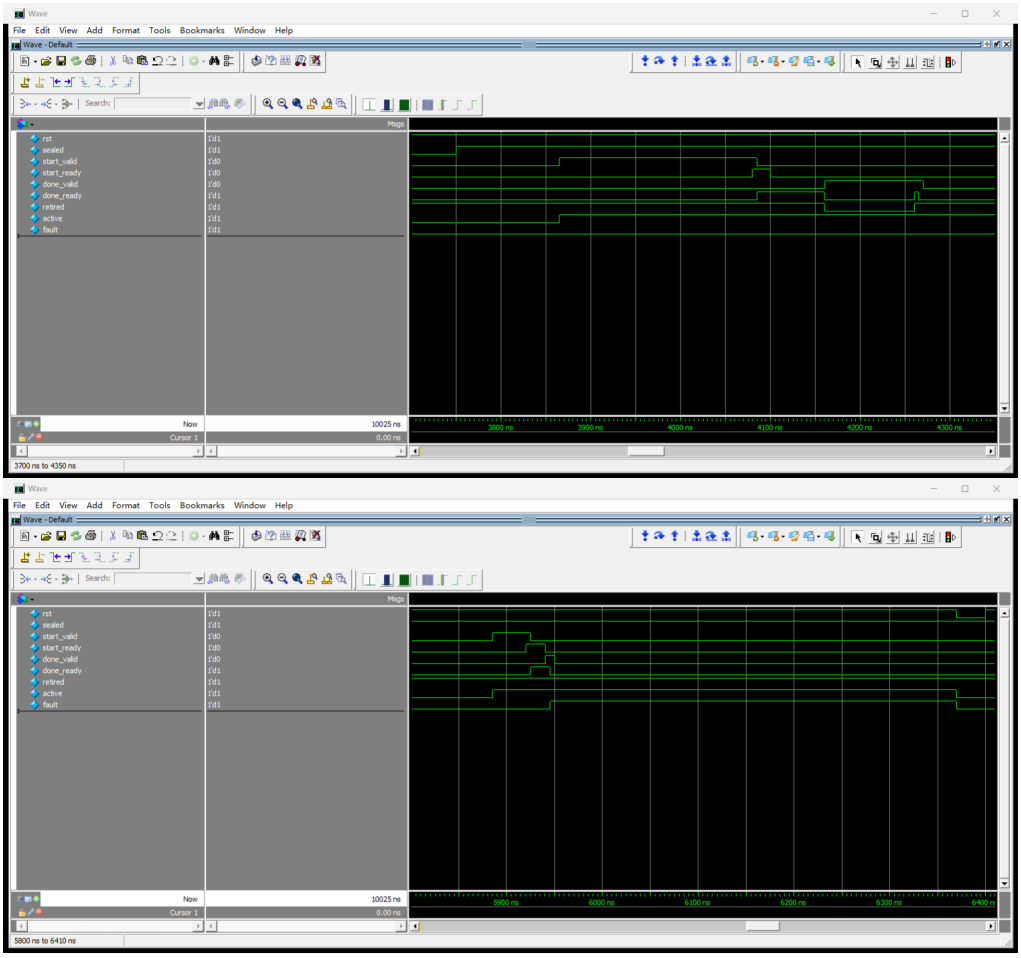


图 C-7 单算子接口的完成等待与故障响应波形

范围包含公开数据、计数、时钟使能、未知载荷及清空复位行为。

C.5 辅助算子

Add、Sigmoid、Transpose 和 Reshape 共用向量存储端口，测试台通过本地存储模型提供输入并核对输出。八通道量化配置的回归覆盖 14 个用例、150092 B；Q4 时分量化配置的回归覆盖 9 个用例、149212 B，分别与独立整数参考逐字节一致。图C-9取自 Q4 配置，显示 Add 和 Sigmoid 第一组量化处理的状态边界。状态值 `0x0C`、`0x0E`、`0x0F` 和 `0x10` 依次对应量化准备、请求、等待结果和概率表读取请求。Add 在量化等待后返回准备状态，处理下一组元素；Sigmoid 在量化等待后发出概率表读取请求。状态的含义依据该用例的 RTL 定义解释，数据比较采用测试台的独立记录。

Transpose 用例按目标维序搬运 $4\times1\times17$ 个 I16 元素，136 B 输出全部匹配。Reshape 用例保持 6400 个 I16 元素的顺序，12800 B 输出全部匹配。相关回归同时检查读写背压、元素边界和目的缓冲区相邻区域，确保地址变换与有效字节写入一致。

MaxPool 与 Resize 采用空间层和块 DMA 联合仿真，桥端使用字节存储模型，计算及片上缓冲使用相应 RTL 和器件原语。图C-10分别截取两个用例的末行写回与完成

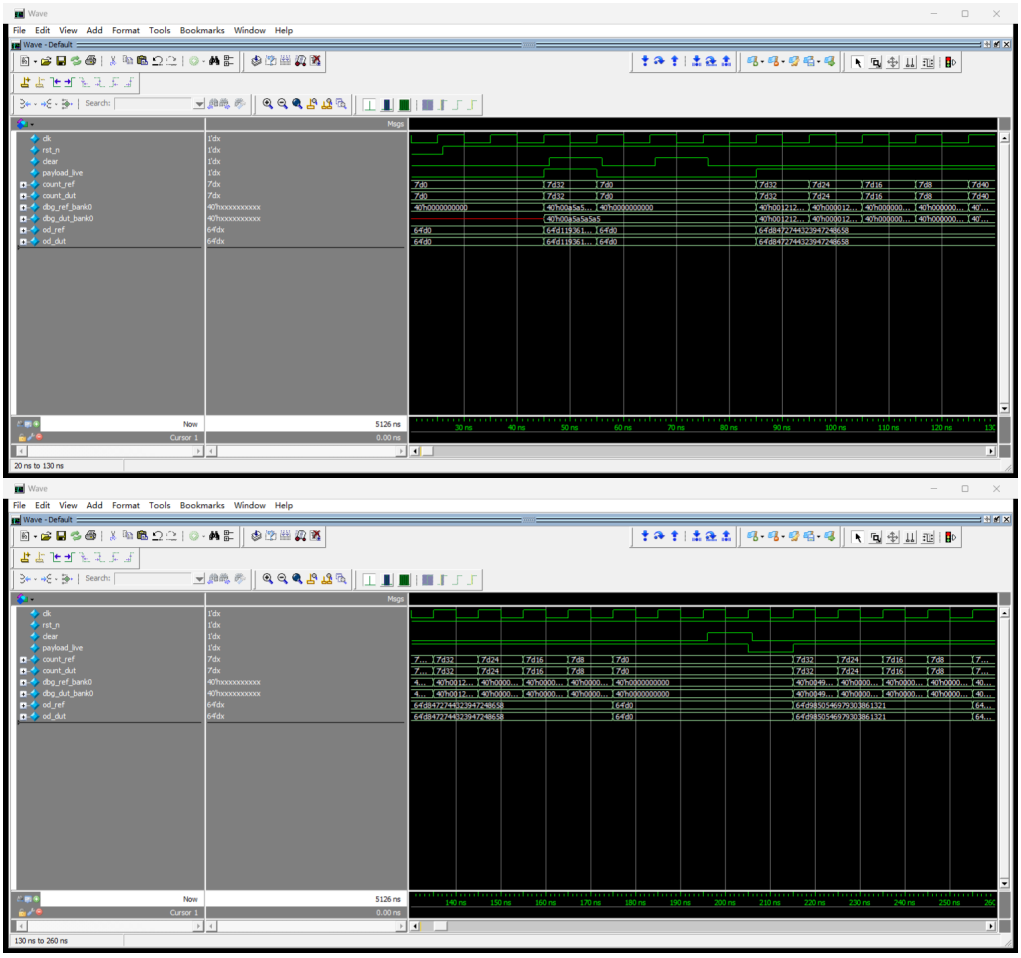


图 C-8 输入 FIFO 清空、载荷隔离与重新装载波形

过程。MaxPool 产生 7 行、18368 B 输出，Resize 产生 40 行、4560 B 输出，均与独立整数参考一致。两个用例的 MAC 向量计数和量化组计数保持为 0，`stat_rows` 在末行写回后分别达到 7 和 40，随后 `done_valid` 置位。

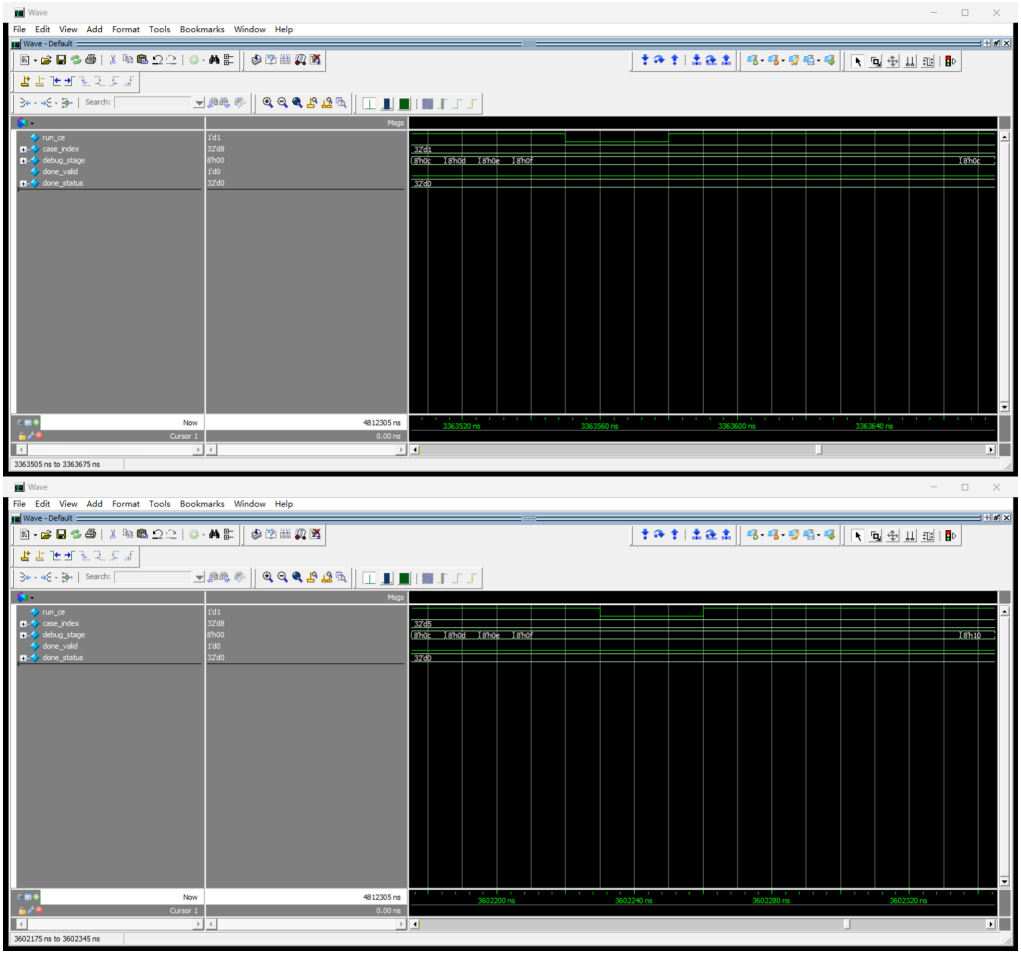


图 C-9 Add 与 Sigmoid 的量化处理状态波形（Q4 时分配置）

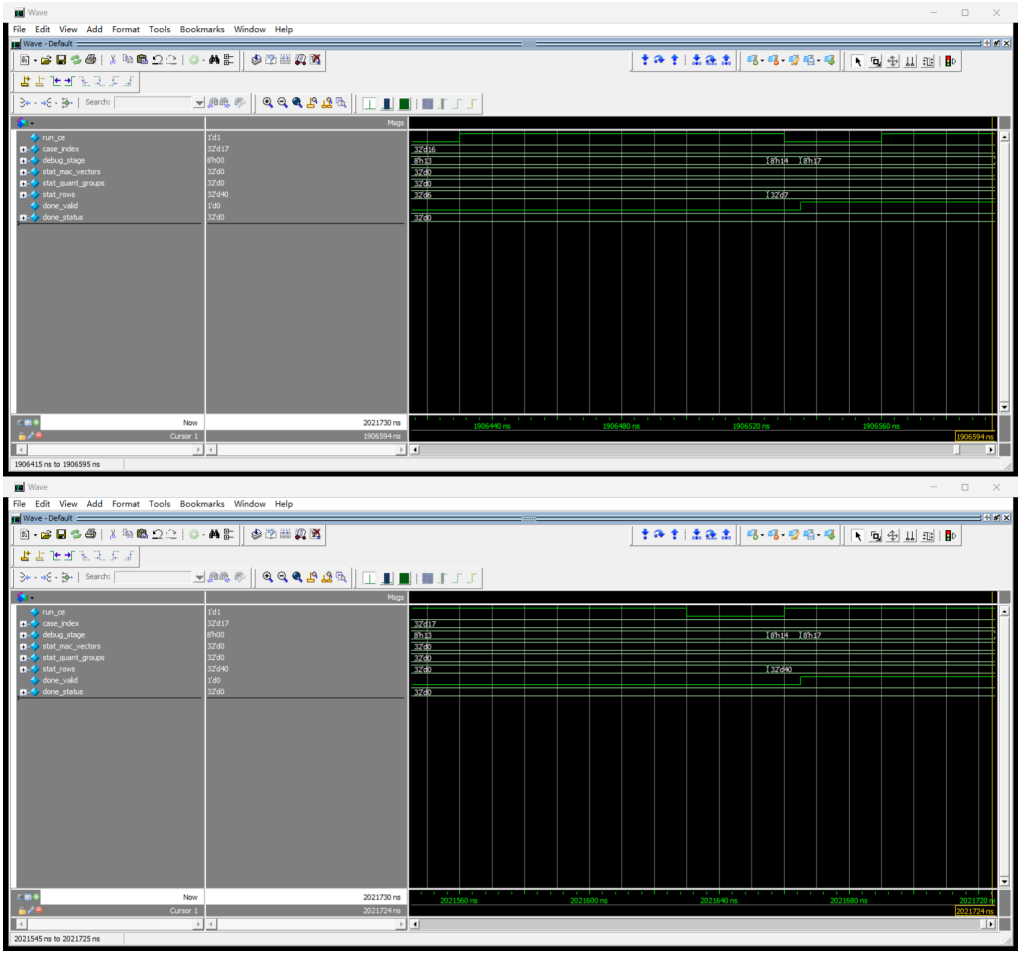


图 C-10 MaxPool 与 Resize 的末行写回及完成波形

D 系统互连与存储仿真验证

本节记录底层控制和互连的组件配置验证，其中 CPU 到 NPU 采用单算子 MMIO，CPU 到 Video 采用三路采集配置。本次头像应用整机的整帧图执行、两路采集及应用循环另见附录F。测试台均在 Windows 原生 ModelSim 中运行，图中波形为已保存 WLF 的原始 Wave 窗口截图。结果索引和源文件哈希见`docs/verification/soc/system_interconnect_storage_20261030.json`。

D.1 CPU 主端控制与外设访问

CPU 到 Video 的固件测试覆盖 6 个正例和 2 个负例。正例检查控制请求握手、三路 AXI 写通道、独立写响应、响应排空、完成事件和中断；负例分别注入错误响应和提前完成条件，均被测试台拒绝。图D-1给出一组原始仿真波形：请求握手后，AW/W/B 通道交错推进，`port_drained` 保持为零直到写响应全部排空，随后才产生 `completion_valid` 和中断。

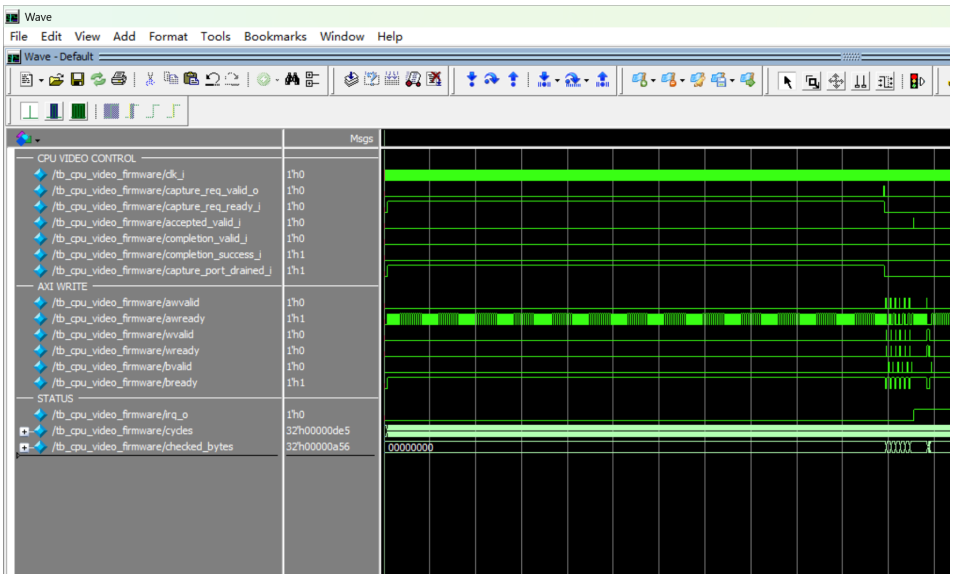


图 D-1 CPU 到 Video 控制请求、AXI 写入、响应排空和完成中断波形

CPU 到 NPU 的算子测试覆盖 Dense U8 和 DW S8 两类算子，分别检查轮询和中断完成路径，共 4 个正例通过。图D-2展示 AXI-Lite 参数写入与 AXI128 读写启动阶段，图D-3展示尾部写响应、读写握手背压以及事务计数收敛；`DONE` 只在最后一个写响应完成后产生。

D.2 共享存储与地址译码

Video 共享 DDR 测试覆盖读写竞争、配额、释放、错误保持和背压传播；地址译码回归完成 118 项正常检查。对“错误目标选择”“过早接受 W 通道”和“请求与读响应对应错误”分别进行变异，三项均以非零诊断退出，说明地址译码及请求与响应

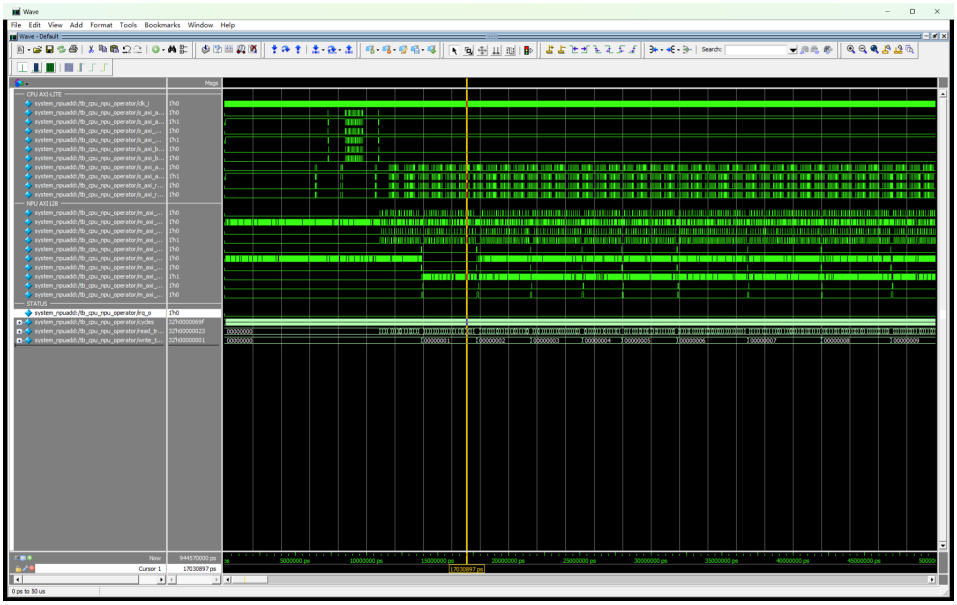


图 D-2 CPU 到 NPU 的 AXI-Lite 配置与 AXI128 初始传输波形

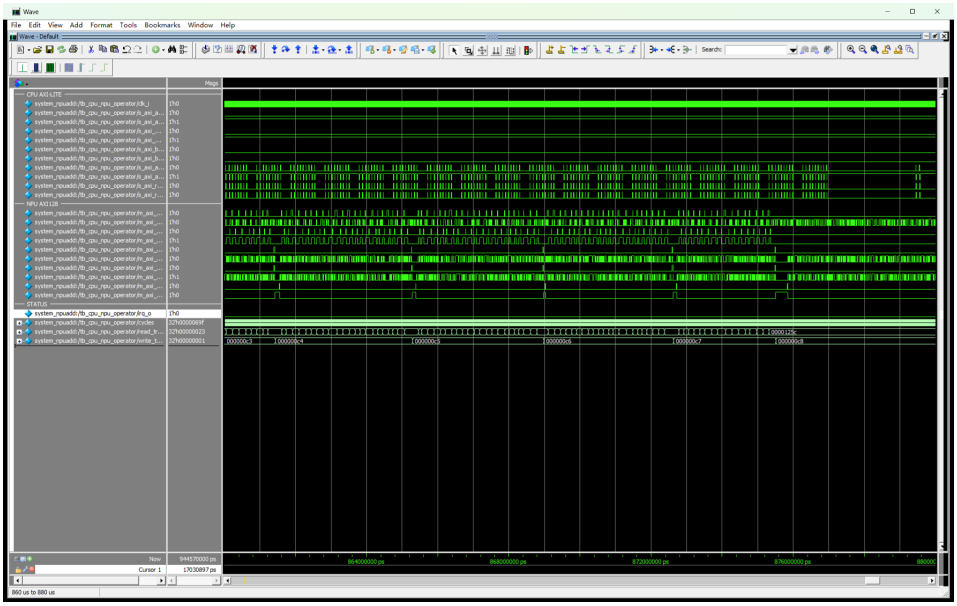


图 D-3 CPU 到 NPU 尾部写响应、背压和事务计数收敛波形

对应关系的检查有效。

表 D-1 系统互联与存储数字仿真结果

场景	覆盖内容	统计	结果
CPU 到 Video	控制握手、三路写入、响应排空、完成和中断	6 个正例、2 个负例	PASS
CPU 到 NPU	AXI-Lite 配置、AXI128 读写、轮询和中断完成	4 个正例	PASS
Video 共享 DDR	读写竞争、配额、释放、错误保持和背压	共享 DDR 回归	PASS
Video 地址译码	目标选择、边界访问和请求与响应对应关系	118 项正常、3 项变异	PASS

上述结果来自使用本地存储模型的数字仿真，覆盖 CPU 外设控制、地址译码、读写握手、访问仲裁和错误响应。

E 图像采集与显示链路仿真验证

本节包括采集重新使能、显示槽所有权、共享 DDR 背压和采显联合四类系统场景，以及紧凑显示组件回归。前四类场景采用独立 Video 组件配置，采集通路包含原图、预览图和模型输入三路。测试台在 Windows 原生 ModelSim 中运行，接口响应由可背压数字模型给出，所有比较均由测试台自检完成。前四类场景的回归日志、WLF/VCD、输入哈希和原始仿真截图由`docs/verification/video/evidence/video_high_value_waveforms_20261030.json`统一登记。

E.1 四类系统级场景

表E-1给出短场景波形对应的通过统计。短场景用于保留关键事件的原始仿真波形，完整通过结果仍由同一测试入口的回归日志核对。

表 E-1 Video 链路高价值系统级仿真结果

场景	测试顶层	关键检查	结果
采集重新使能	`capture_rearm`	7 次采集成功、7 次重新使能，三路写入共 18522 字节；完成响应排空后才发布完成事件	PASS
显示槽所有权	`slot_owner`	128 位状态检查 155 项；提交、实际生效、代次、RETIRING、释放确认和空槽位图顺序一致	PASS
共享 DDR 背压	`memory_scheduler`	12 次读、12 次写；读写最大等待均为 0，调度配额为 4，故障保持为 0	PASS

表 E-1 Video 链路高价值系统级仿真结果（续）

场景	测试顶层	关键检查	结果
采显联合	`shared_memory`	7 个任务、7800 个像素、13 次光栅；读写竞争重叠 11690 拍，读授权 262 次、写授权 441 次，欠载 0	PASS

E.2 采集与槽所有权

采集波形中，`req_valid/req_ready` 的握手先锁存帧号和槽号。三路写入计数 `raw_written_bytes`、`preview_written_bytes` 和 `input_written_bytes` 逐步增加；`port_drained` 置位后才出现 `completion_valid`。成功场景中 `error` 保持为零，重新使能次数与成功次数相等。

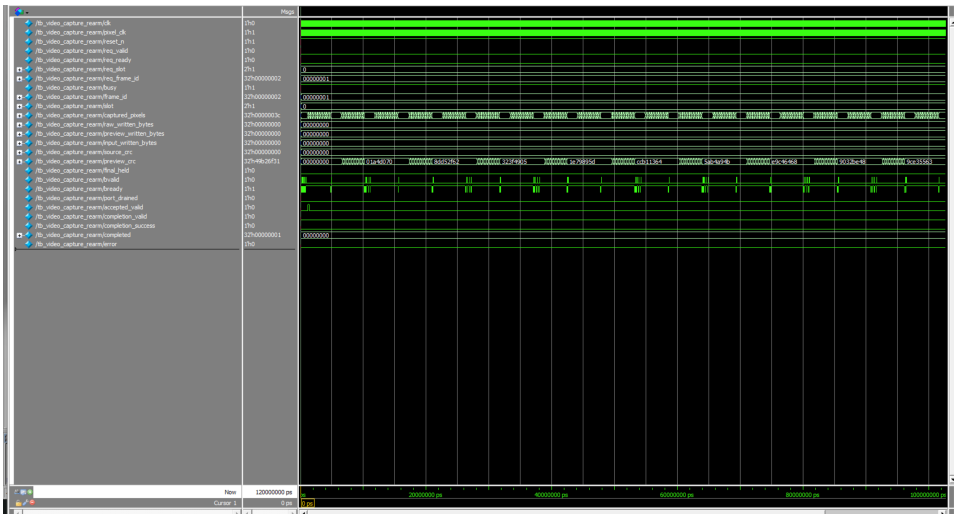


图 E-1 采集请求、三路写入计数、响应排空和完成事件波形

槽所有权场景将命令接收、显示生效和释放确认分开。`active_slot_o` 及 `active_frame_o` 在匹配的显示反馈握手后更新；旧槽依次进入 RETIRING 和等待释放确认状态，直至 `release_valid_o` 对应的释放握手完成才恢复为空闲。`protocol_error_o` 全程为零。

E.3 共享 DDR 背压与采显联合

背压场景同时驱动写地址/写响应和读地址/读数据通道。波形显示读写通道分别被暂停时，仲裁器仍保持请求与发起端的对应关系，`busy` 和 `port_drained` 在事务排空前不会提前结束；读写计数与测试台的 12 次读写请求一致。

联合场景把采集写、显示读、槽号及对应帧标识和像素输出放在同一时间轴上。`commit_frame_id`、`release_frame_id` 和 `display_frame` 分别标记提交、释放和实际显示的帧标识，用于核对同一帧的事件顺序。`underflows` 保持为零，`seen_commits` 与

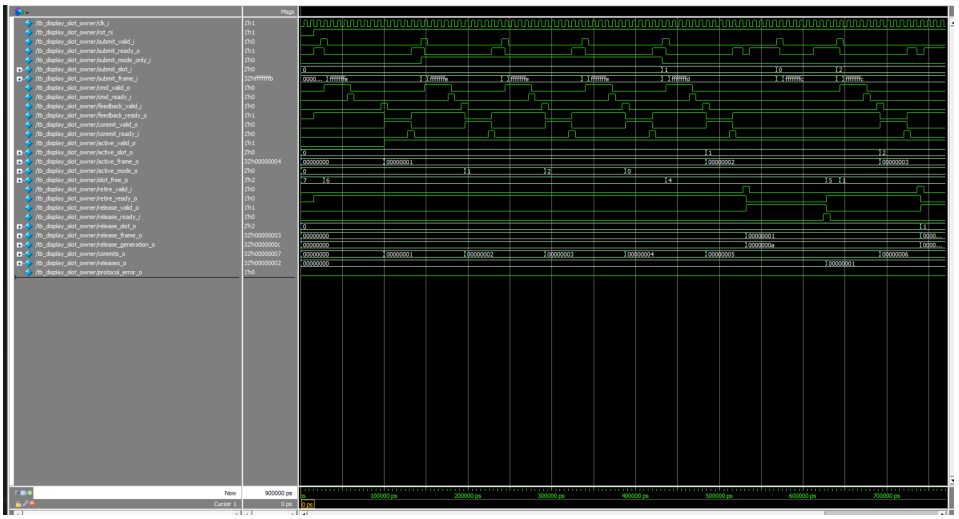


图 E-2 显示槽提交、生效、代次和释放确认波形

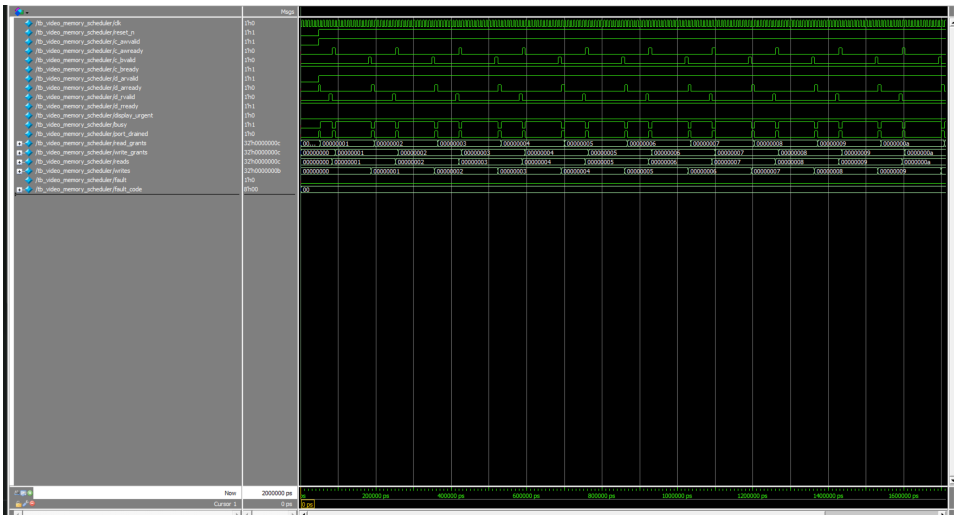


图 E-3 共享 DDR 读写握手、背压、仲裁计数和排空波形

`seen_releases` 按提交和释放事件推进，`checked_pixels` 持续增加。2 毫秒短窗口已经覆盖四个连续帧的提交/释放和采显重叠，完整 13 次光栅的数值结果见表E-1。

上述四类场景核对数字模型中的握手、所有权和背压传播，统计范围对应各自的组件测试；真实人脸检测应用的端到端功能由系统联合仿真及实板观测分别验证。

E.4 紧凑显示组件回归

紧凑显示接口在接收 AXI-Lite 读地址后先寄存请求，再执行译码。记录复制占用存储端口时，接口接收一项读请求并等待端口可用；响应背压期间保持返回数据和响应码，复位时清除尚未完成的请求。MMIO 回归检查接收地址保持、单项排队、响应稳定性及复位后的请求清除。回归还检查显示命令接收后事件保持为零、实际像素反馈产生 COMMIT，以及释放事件经 ACK 后更新空闲槽位图。

表E-2列出紧凑显示接口的组件回归结果，原始配置与日志保存于`docs/verificat`

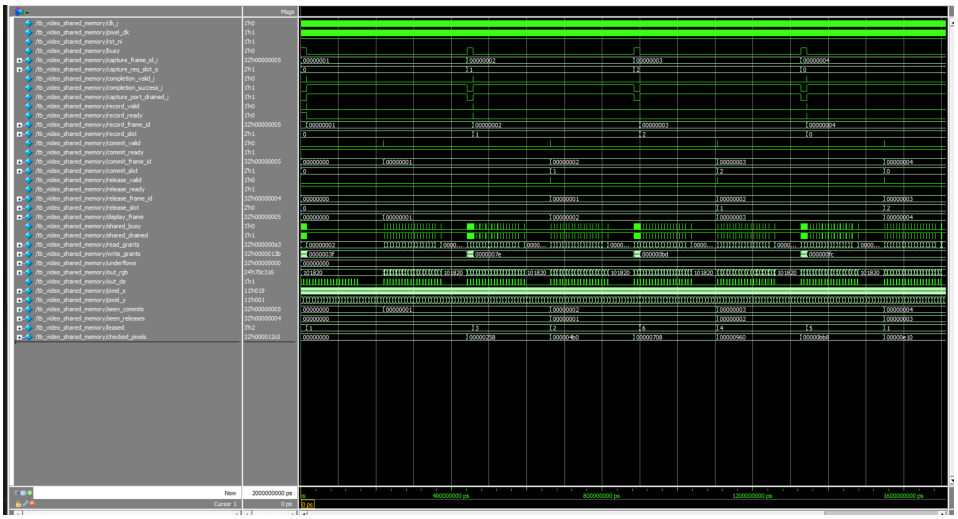


图 E-4 采集、共享 DDR、槽释放和像素输出联合波形

ion/soc/face_box_report_20261103/。头像应用的完整采集、推理与三模式输出验证见附录F。27 项为组件用例数，246 项和 1653 项分别为记录检查与排队读用例内部的检查数。

表 E-2 紧凑显示组件回归结果

测试范围	统计量	主要检查	结果
显示组件回归	27 项用例	显示接口、跨域交互与像素组合行为	PASS
记录合法性检查	246 项检查	显示记录格式、身份及边界条件	PASS
MMIO 排队读 CASE10	1653 项检查	复制占用时接收请求、地址保持、响应背压及复位	PASS

F 系统联合仿真验证

本节采用板测发布版本配套的自然人脸与零 RGB 空场景，核对真实应用循环中的采集、推理、显示提交及帧槽释放。两组回归的结果、完整日志及配套验收记录集中保存于`docs/verification/soc/face_photo_report_20261104/`。

F.1 完整模型推理

测试台通过实际模型装载器传输六段模型数据及一段 RGBA 头像，共 2,032,640 B、7,940 包，待写入确认及模型、素材封存后启动应用。装载激励连接字节握手接口，未按 115200 baud 逐位模拟，因此本测试不提供串口实物装载时间。输入由数字视频激励提供，DDR 端点采用 native 存储代理；采集预处理、CPU 固件、NPU 和显示路径使用工程实现。自然人脸场景以 RGB 图像作为采集输入，期望检测到 1 个人脸；零 RGB 空场景期望检测结果为空。

每个接纳帧经采集路径写入 1,440,000 B 原图和 307,200 B 模型输入。NPU 从对应输入槽执行模型计算图并写出九个输出头，CPU 读取输出头，按 0.6 分数阈值筛选、解码检测框，并以 0.3 为 IoU 阈值执行非极大值抑制，构建同帧显示记录。两组场景均完成两个不同帧号的推理结果核对，并通过 RAW、BOX、AVATAR 完整画面的像素检查。

F.2 连续帧处理

采集请求依次将帧 1、帧 2 和帧 3 分配到槽 0、槽 1 和槽 2。长时间推理期间，帧池保持计算和显示引用，三个槽占用后等待释放。帧 1 显示期间，HDMI 重复扫描其活动记录；帧 2 的记录生效后，显示端排空帧 1 的旧引用并发布释放事件。CPU 校验释放身份并完成 ACK 后，槽 0 用于采集帧 4。

表F-1列出两组完整回归的事件统计。每组完成 4 次采集，其中帧 1 和帧 2 完成推理、显示记录提交及整幅画面核对；测试在确认一次旧槽释放和复用后结束。

表 F-1 系统联合仿真帧事件统计

统计事件	自然人脸	零 RGB 空场景
采集请求 / 完成	4 / 4	4 / 4
推理结果完成	2	2
新帧显示记录 / 实际提交	2 / 2	2 / 2
旧槽释放 / 复用	1 / 1	1 / 1
完整输出画面的不同帧号数	2	2
覆盖显示模式	RAW、BOX、AVATAR	RAW、BOX、AVATAR

测试台同时检查采集写入不覆盖仍由显示端引用的槽，槽复用发生在 CPU 确认释放之后；显示输出的槽号、帧号、代次和提交序号与已提交记录一致。记录生效后，模式和提交序号在一幅有效画面内保持不变。表中的记录和提交次数按不同采集帧计数。帧 1 先以 RAW 生效，再使用同一帧切换 BOX 和 AVATAR，提交序号依次为 1、2、3；帧 2 以 AVATAR 生效，序号为 4。仅模式切换的两次提交不计作新帧，也不重复释放当前槽。

性能日志中的 `FACE_SYSTEM_NPU_CPU_RESULT` 由帧池 `results` 计数增加触发，表示 CPU 已完成后处理、显示记录构建及 NPU 完成确认。两组场景各有两个结果事件，只形成一个相邻间隔样本；周期差、时间和结果完成率见表3-12。一次性装载位于首个结果之前；每帧图执行前的 79 次算子预检仍计入相邻结果间隔。日志中的 `op=0,10,…,70` 是稀疏算子完成检查点，不能据此得到完整的逐算子耗时；两帧完整画面核对属于数字功能验收，也不替代持续板级帧率测量。

F.3 运行配置

回归在 Windows 原生 ModelSim 中运行，固件由 xPack RISC-V GCC 构建。自然人脸运行标识为`face_system-20261104T011551_456009Z`，零 RGB 空场景运行标识为`face_system-20261104T011551_453685Z`。两组运行均采用板级头像配置和七段数据装载流程，参数见表F-2。

表 F-2 系统联合仿真运行配置

项目	配置
仿真工具	ModelSim 10.5
固件工具	xPack RISC-V GCC 15.2.0-1
CPU/NPU、Video、像素时钟	50 MHz、125 MHz、40 MHz
应用配置	`board_profile=true，boot_load=true，avatar_boot=true`
输入及显示尺寸	800×600 RGB888，RAW、BOX、AVATAR
模型输入	320×320 UINT8 BGR NCHW，上下各补 40 行零值

表 F-2 系统联合仿真运行配置（续）

项目	配置
采集写入	原图与模型输入两路，每帧 1,747,200 B
存储端点	native DDR 存储代理
结束时计算周期数	每组 41,410,743 周期

F.4 结果核对

测试台逐字节核对采集写入的原图和模型输入，检查写入覆盖范围与完成时的端口排空状态。每个已完成推理帧的 26,368 B 输出头与独立参考比较，并核对 CPU 后处理结果有效标志和人脸数量：自然人脸为 1，零 RGB 空场景为 0。

显示检查从有效图像起点累计每帧 480000 个像素，逐像素核对 RGB、坐标顺序和帧身份。自然人脸场景在 BOX 模式检查矩形内两像素宽的红色边框，在 AVATAR 模式按素材采样、整数透明度混合及覆盖范围核对输出；RAW 模式逐像素保持原图。两组回归均确认帧 1、帧 2 的完整 800×600 画面，显示模式覆盖位图为 `111`，覆盖三种模式。自然人脸场景每幅 AVATAR 画面的矩形覆盖区为 148,996 个像素，实际与背景不同的像素为 76,688 个；覆盖数不等于非透明像素数。零 RGB 空场景的覆盖数和改变数均为 0。

两组完整日志均给出 `FACE_SYSTEM_PASS`，错误和警告计数均为 0。通过条件还包含显示欠载和硬件故障检查、帧池无空闲槽等待、释放确认及复用检查，验证了从输入预处理到同帧框及头像输出的数字功能链路。